

82

26th June, 2010

Saturday

Operating Systems

Topics :

(1) Background

(a) Process management

- * Process concepts
- * CPU scheduling
- * IPC & synchronisation
- * concurrency
- * Deadlocks
- * Threads

(b) Memory management

- * concepts
- * RAM chips
- * Techniques
- * Virtual memory

(4) File system & I/O management

- * Interface
- * Device characteristics
- * Implementation issues.

(5) Protection mechanisms

(q). What is an operating system ?

Ans: Interface between user and Hardware

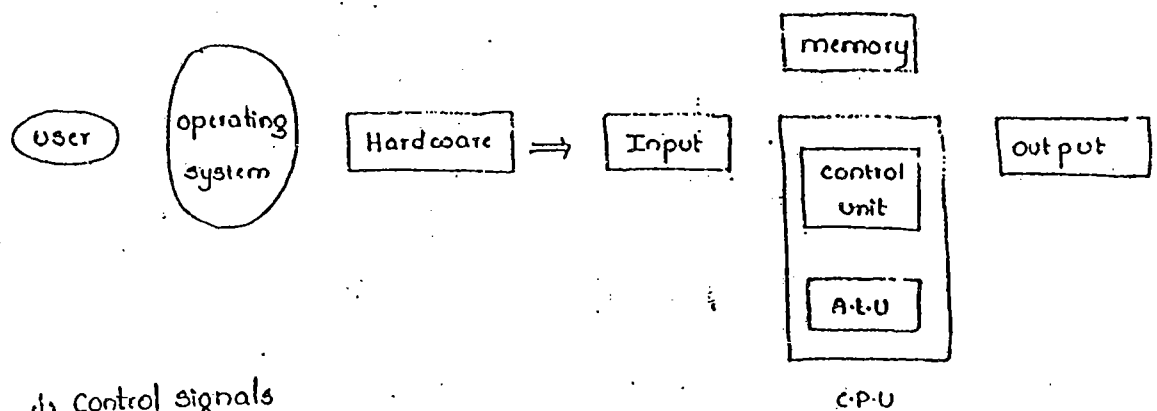
* Resource manager

* control program(s)

* A set of utilities to simplify application development.

* Acts like a government.

Von-Neumann Architecture :



(i) Control signals

(ii) Mathematical operations (Data, Register)

clock cycles.

$$c = a + b$$

load R_1 m[a]

load R_2 m[b]

Memory :

* Primary (main memory, physical $\Rightarrow$ RAM, ROM, cache, Registers)

* Secondary (Auxiliary $\Rightarrow$ HD / DVD / Pen drives)

* Processor cannot fetch data directly from secondary memory. Instead Instruction Register (IR) is used, from where instructions are fetched and executed sequentially.

* If IR requires any operands, then it fetches the operands and executes as :-

* Fetch cycle
 * Decode cycle
 * Execute cycle
 * Interrupt cycle

} → Instruction cycle

* The communication among user and CPU can be represented by Von-Neumann architecture.

Main Objective of operating system :

* It takes the complete control of hardware, and create a platform, which is easy to user to write different applications.

Command Interpreter :

* A program that interprets the commands of the user.

* It acts as Interface between the user and Kernel.

* These are of two types :

* Text based O.S (shell) : DOS, UNIX, NETWARE

* GUI based O.S (Desktop) : WINDOWS, LINUX.

* Operating system is a set of control program(s).

Operating System is visualised as resources.

Hardware



- * CPU
- * memory
- * I/O

Software



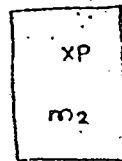
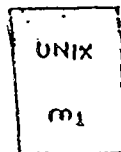
- * Device drivers
- * semaphores
- * messages
- * monitors

Functions & Goals:

Goals:

- (1) convenience (user friendly)
- (2) Efficiency (resource utility)
- (3) Reliability & Robustness
- (4) Scalability (Ability to evolve)
- (5) portability (Different platforms)

Consider an example:



Execution time: 50 sec

75 sec

To find a particular content in "UNIX" O.S, we must write its syntax.

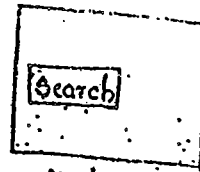
Eg:- `$find`

`/-name "text.c"`

So, that it requires exact syntax, but it takes less time to execute the given statement.

To find the same in "XP" O.S, it is an easy process that any user can

approach through (i.e., clicking "start" button & then "search")



find.exe

- * It takes more time than unix to execute, but it is more familiar to all kinds of user to operate.
- * So, "convenience" is more important than "efficiency".
- * In some real time systems; "Efficiency" is more important than "convenience".

First Generation

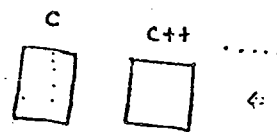
No operating system

Card-Readers, Punch card

Second Generation

Magnetic Tapes

manual Intervention.



Batch processing
↓
clubbing similar program
at one place & processed
one at a time.

Third Generation

Hard Disk Technology

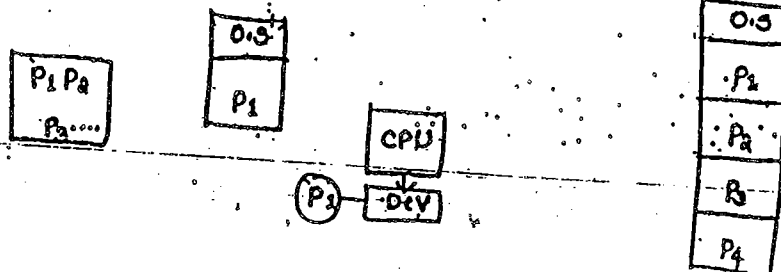
Uniprogramming

multiprogramming

*(wastage of
Idleness of CPU)

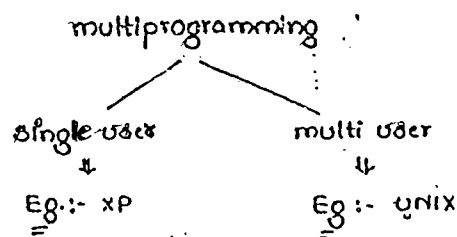
↓ (DOS)
main memory

↓ (UNIX, XP)



Multiprogramming:

- * Ability of operating system to hold multiple ready to run programs in the main memory so that if the running program requires I/O, the CPU can be switched to another ready program.
- * Multiplexing of CPU among ready programs in memory.



Non-Pre-emptive:

- * Running process is released voluntarily (on its own) as follows:
 - * completion of process
 - * I/O request.

Pre-emptive:

- * Running process can be forced to release based on:
 - * Priority
 - * Time sharing

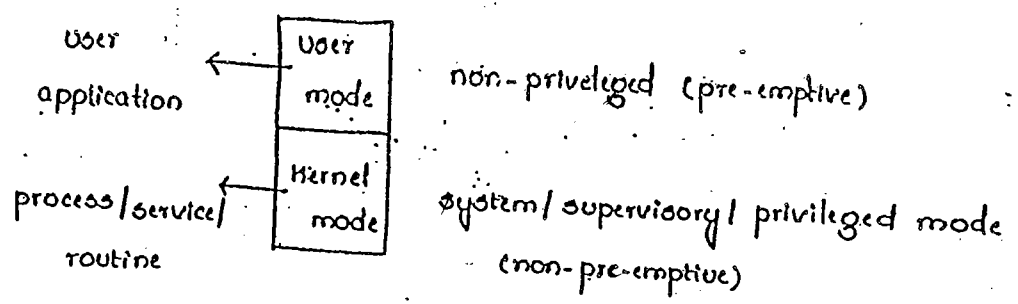
Time sharing Systems (pre-emptive):

Eg:- windows XP

first version of windows : 3.0
3.1
3.11 } non-preemptive

Architecture of multiprocess support:

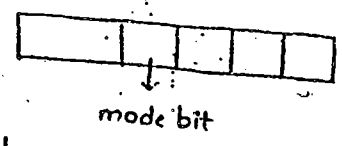
- (1) Direct memory Access (DMA) $\Rightarrow$ Since computation & I/O required, we need DMA.
- (2) Address Translation
- (3) Atleast two modes of CPU execution.



- * In kernel mode, the process have atomic execution (without pre-emption)
- * In user mode, the process have non-atomic execution (with pre-emption)

Program status coord (PSW):

It represents the mode, in which the user operates in.



0 - user mode
1 - kernel mode

System call vs Library call:

main()

```

{
    int a,b,c;

```

```

    scanf("%d %d", &a, &b);  $\rightarrow$  (library call)
    C = a+b;
    fork();  $\rightarrow$  (system call) (sys)
    printf("Hello");  $\rightarrow$  (supervisory call)

```

Stored in std. library

(BSA) $\Rightarrow$ Branch & Save Address (BSA)

User mode

* `fork()` creates child process and the execution of `fork()` is in kernel mode, because it is a part of O.S. and the conversion takes place (from user mode to system mode)

* Execution of supervisory call (SVC) involves the software interrupt, which handles Interrupt Service Routine (ISR).

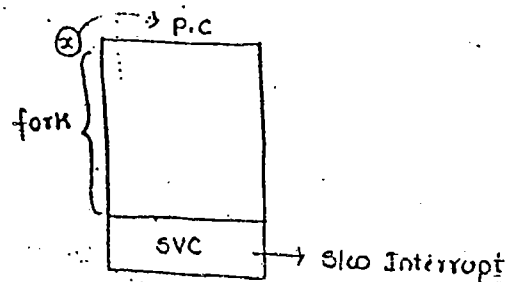
* When a SVC is generated, ISR changes to 1 (non-preemptive mode). Then after the completion of all the functions in kernel mode, again an SVC is generated, where ISR value is changed from 1 to 0 (pre-emptive mode).

System call Interface (SCI) $\Rightarrow$ UNIX

In windows O.S, SCI is considered as API.

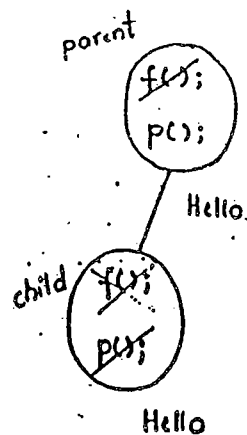
ISR $\Rightarrow$ changes mode bit
&
Dispatch the table.

fork	(x)

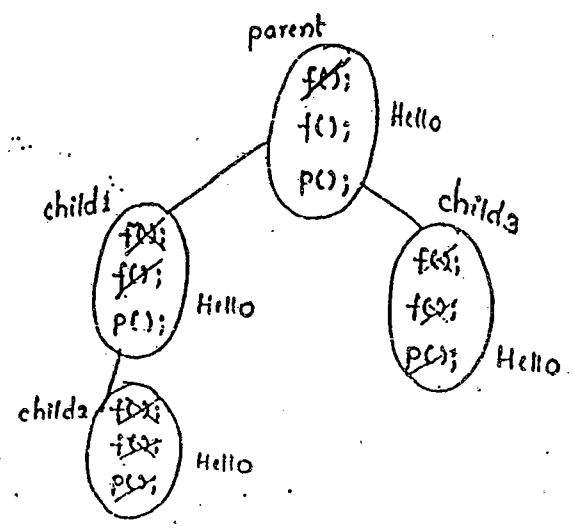


```
1) main()
{
    fork();
    printf("Hello");
}
```

Output : Hello
Hello



```
(a) main()
{
    fork();
    fork();
    printf("Hello");
}
```



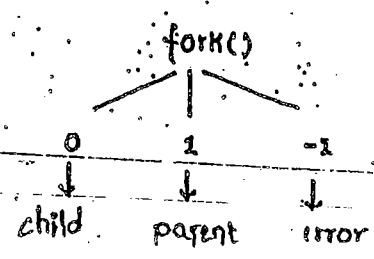
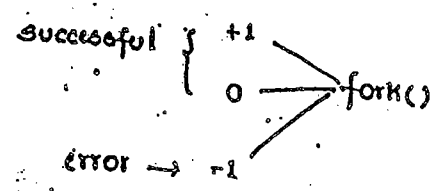
* If 1 fork $\Rightarrow$ 2 prints
 2 forks $\Rightarrow$ 4 prints
 3 forks $\Rightarrow$ 8 prints
 $\vdots$
 n forks $\Rightarrow 2^n = \text{Total}$
 $n_{\text{child}} = 2^n - 1, \text{parent} = 1$

output :
 Hello
 Hello
 Hello
 Hello.

```
(b) main()
{
    int i, n;
    for(i=1; i<=n; ++i)
        fork();
}
```

(a) $n-1$ (b) $n-1$ (c) n^2-1 (d) 2^n-1 (new forks)

* Any system call may return either +1, 0, -1 values.



main()

```
{
    int id;
    id = fork();
    if (id == 0) // child
    {
        // child //
    }
    else if (id > 0)
    {
        // parent
    }
    else printf("Error in creation");
}
```

parent

id 1

```
id = fork();
if (id == 0)
{ }
else if (id > 0)
{ }
```

child

id 0

```
id = fork();
if (id == 0)
{ }
else if (id > 0)
{ }
```

Difference between kernel mode & user mode:

Compilers, editors and similar application-independent programs are not part of the O.S., even though they are typically supplied by computer architecture. This is crucial, but subtle point. The O.S. system is that portion of the software that runs in kernel mode. It is protected from user tampering by hardware.

Compilers & editors runs in user mode. If a user doesn't like a particular compiler, he/she is free to write their own clock interrupt handler, which is a part of O.S.

01st July, 2010

Thursday

Process concepts

Program Vs Process :

* Program under execution

* Unit of execution

* Schedulable / Dispatchable unit

* Instance of program

* Central locus of control (os)

* Animated spirit

Process : Program

1 : 1

1 : multiple

* Executing program resides in secondary memory

Program	Process
* In secondary memory	* In main memory
* Without resources	* Utilization of resources
* Passive	* Active (Alive)

Formal definition of process :

* Developer / Designer views process as a simple Datastructure.

* Every Datastructure is always defined with four parameters :

* Definition

* Representation (process structure)

↳ Implementation

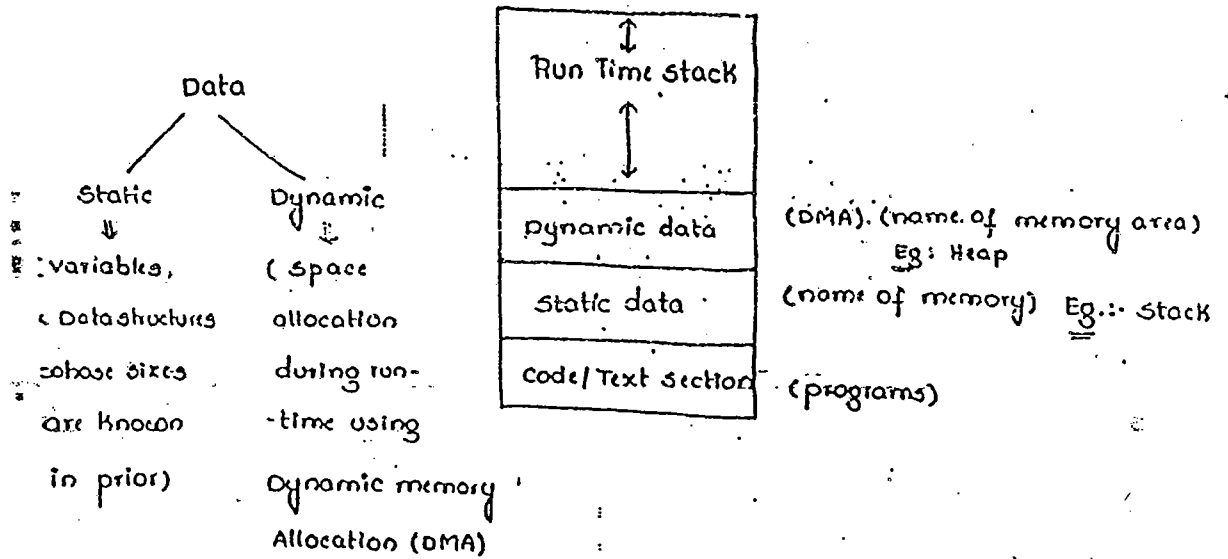
* Operations

* Attributes (push, pop)

Process Structure (Representation/ Implementation) in main memory :

Abstract view of process:

- * program consists of data and instructions.



Run Time stack $\Rightarrow$ Activation records are maintained wherever function calls are generated.
 $\downarrow$
Every process maintains this stack for storing addresses (recursive), we use this stack.

Operations :-

- * Essential resources required for stopping the execution of process, then some resource utilities are used.
- * When a program is loaded in main memory, resources are allocated and then CPU schedules all the process.
- * create (resource allocation)
- * scheduled (CPU)

- * Execution (CPU)
- * Blocked (I/O)
- * Resume
- * suspend

Attributes :-

- * Process Id (pid)
- * Priorities
- * Process state
- * Program counter
- * memory limits
- * list of files
- * list of devices
- * Type & size (in bytes)
- * Protection.

Process control Block (PCB)

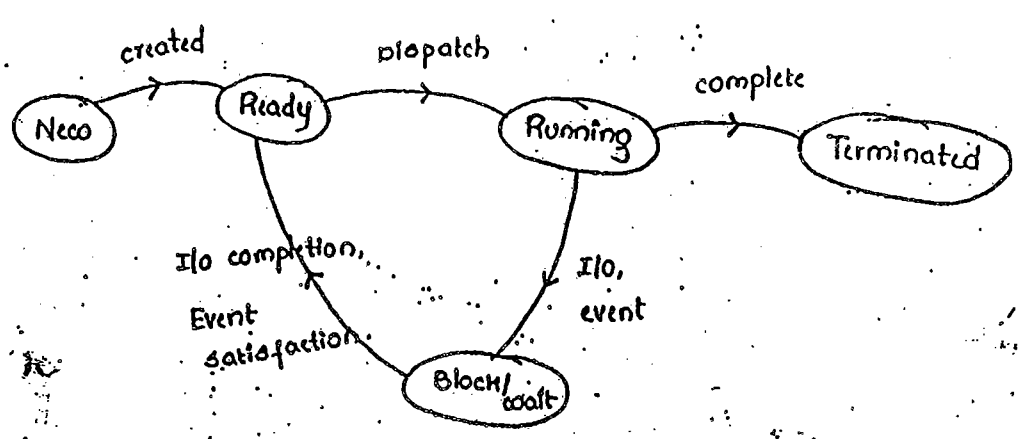
↓
ID of process

Pid	
State	Priority
PC	GPR
mlr limits	files
Devices	...

process context / process environment.

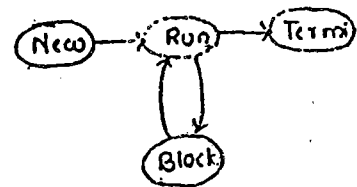
* All the process attributes are stored within the process control block (PCB)

** Process State :-



Above diagram represents :-

(a) Uniprogramming (no "ready" is present)

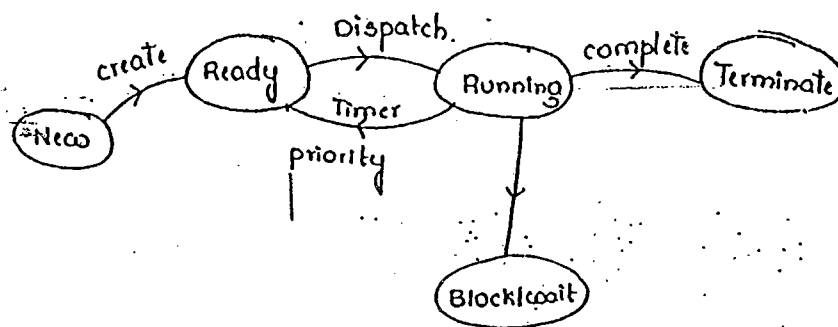


(b) pre-emptive multiprocessing

(c) non-preemptive multiprocessing

(d) multiple cpu based o/s

Pre-emptive multiprocessing:-



Degree of multiprocessing = no. of processes

Degree $\uparrow$, when process $\uparrow$, so, we suspend process whenever required.

process suspension $\Rightarrow$ swapped out on temporary basis and later on resumed.

There is no suspension for the following process states:

$\uparrow$ performance

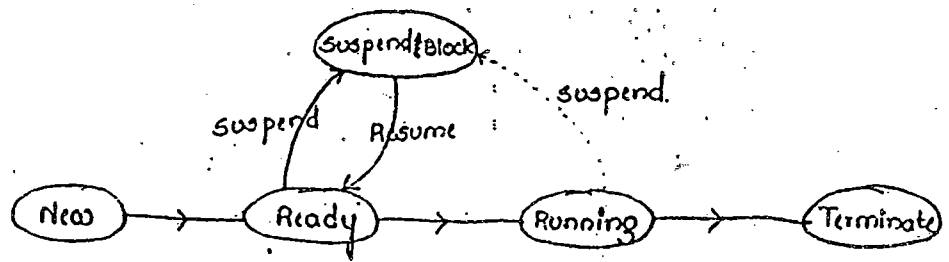
* New

* Terminate

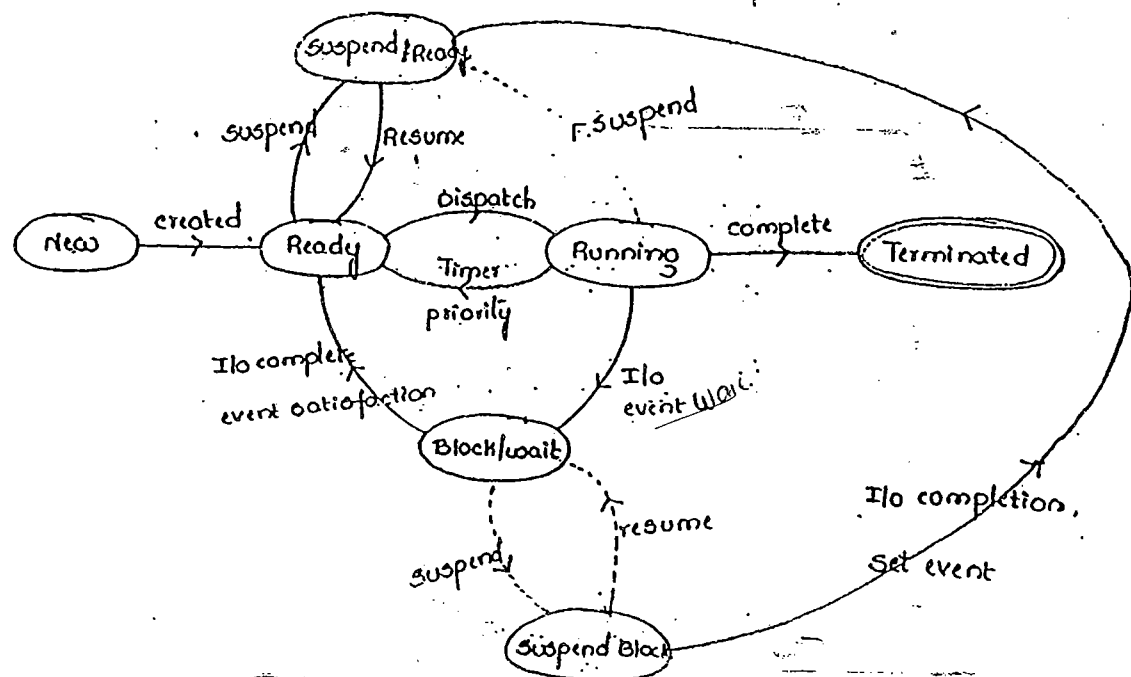
Most desirable state for suspension is : "Ready" state.

Block state - Because, it is waiting for I/O or other events, so, when suspended it get confused.

If "Ready" process is suspended, then it moves to "suspend" state.



Block/wait
When "Ready" state is suspended.



- * Only Ready, running, Block states $\Rightarrow$ can be suspended
- * process, waiting for I/O operations are not suspended (dangerous). If get suspended, it moves to suspend block state.
- * Suspension of running process are not preferred always. If suspended, it releases the resources pre-emptively.
- * So, "Ready" state is the safe to suspend any process, they are being temporarily stored in secondary memory.

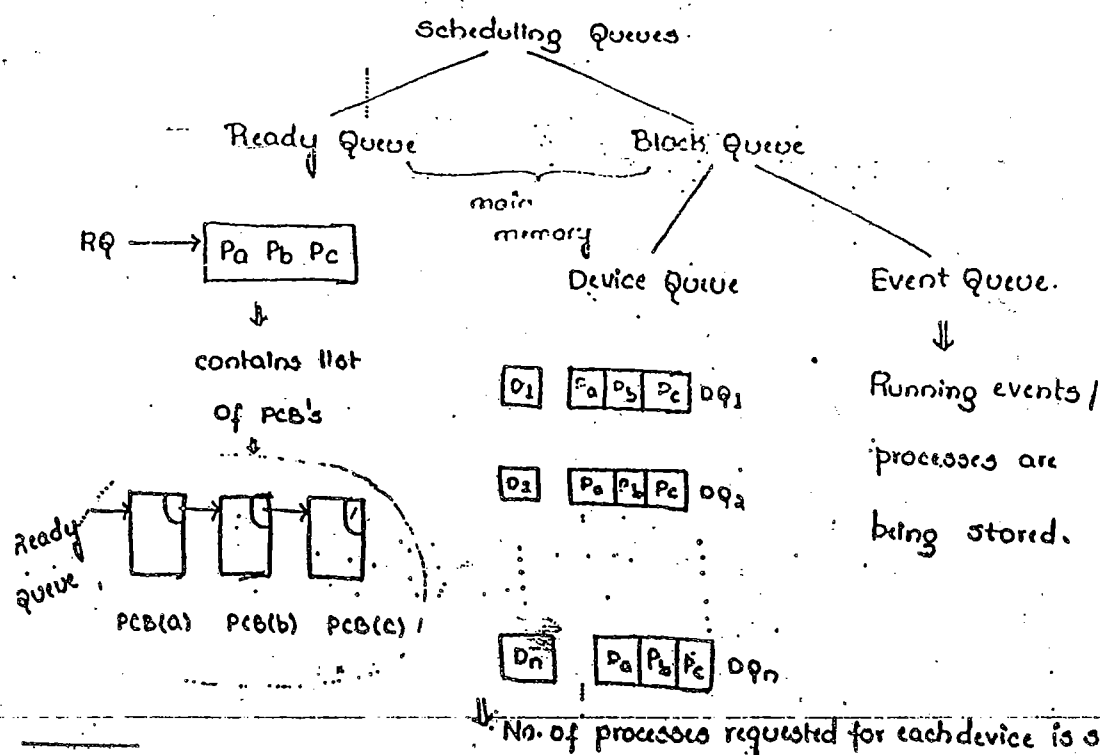
(Q) consider a system with 'n' CPU's and 'm' processes where $n \geq 1$ and $m \geq n$. calculate lower bound and upper bound on the process states.

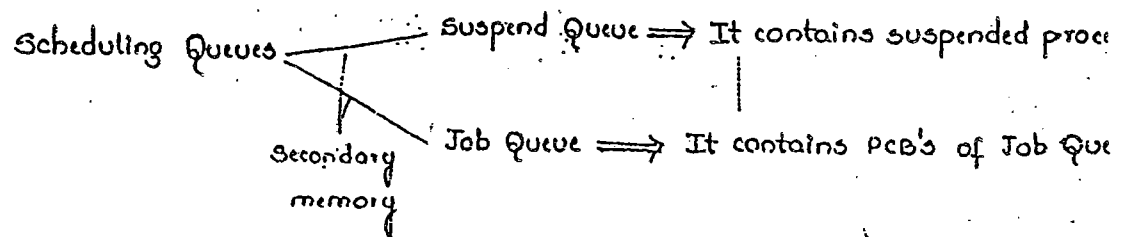
- * Ready
- * Running &
- * Block states.

Ans:- maximum Ready state processes are = m
maximum running state processes are = n.

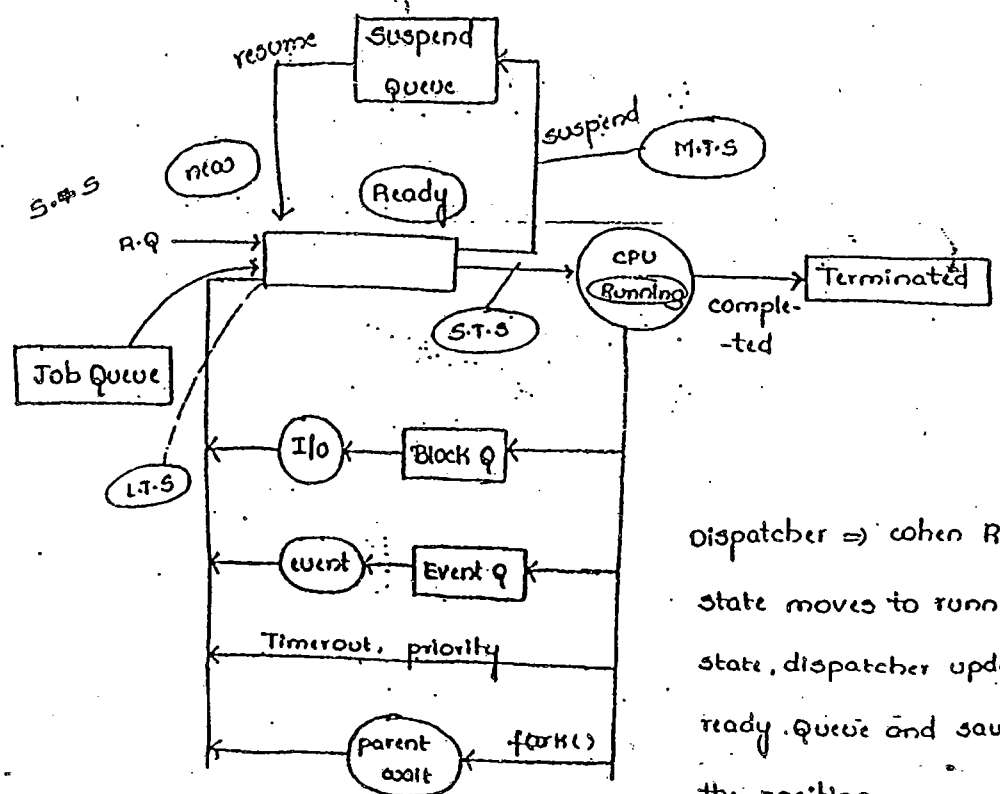
	Min LB	Max UB
Ready	0 ✓	m ✓
Running	0 ✓	n ✓
Block	0 ✓	m ✓

Scheduling Queues & State-Queuing Diagram :-





Process State Queue diagram:-



dispatcher $\Rightarrow$ when Ready state moves to running state, dispatcher updates ready queue and saves the position.

Long term scheduler - loading new programs from secondary to main memory Job Queue.

Short term scheduler (CPU scheduler) :-

It decides that which process (ready processes) should run next.

middle term scheduler (suspended queue).

(It decides that which process should be suspended (or) resumed)

- A process is said to be I/O Bound if it generates minimal or very few I/O requests and does more computation.
- A process is said to be I/O bound if it does more I/O than computation.

- * Switching a process to another on CPU is called "process switching".
- * Saving and loading the PCB's from one process to another is called "context switching".
- * "context switching" has directly impact on volume of information in PCB.

multiprogramming $\Rightarrow$ long term scheduler.

CPU Scheduling

CPU scheduler :

- * Decision of which process to run is taken.

Goals :-

- * Maximize CPU utility
- * Minimize Response time, waiting time.

Process Times :

- * Arrival Time (A.T) - Submission time.
- * Waiting Time (Ready Queue / BQ)
- * Burst Time (CPU) - Service Time (B.T)
- * Completion Time (C.T)
- * Turn Around Time (T.A.T) = Completion Time (C.T) - Arrival Time (A.T)

Response Time (R.T) :

- * Time of submission of request by a process, to obtain a result / response (first response)

Deadline (D):

* No. of processes completed within the deadline:

A.T	R.Q	BT ₁	I/O	RQ	BT ₂	RQ	BT ₃	C.T
	ω _{T1}		ω _{T2}	ω _{T3}		ω _{T4}		

→ clock Time

Notation:

(i) n-processes (P₁.....P_n)

(a) A.T (P_i) = A_i

(b) B.T (P_i) = X_i

(c) C.T (P_i) = C_i

(d) Deadline (P_i) = D_i

(e)

Formulae:

(a) TAT (P_i) = C_i - A_i

(b) WT . TAT (P_i) = $\frac{C_i - A_i}{X_i}$

(c) Avg. TAT (P_i) = $\frac{1}{n} \sum_{i=1}^n (C_i - A_i)$

(d) W.T (P_i) = TAT - BT
= C_i - A_i - X_i

(e) Avg. W.T (P_i) = $\frac{1}{n} \sum_{i=1}^n (C_i - A_i - X_i)$

(f) Schedule length (L) = max (C_i) - min (A_i)

(g) Throughput (μ) = $\frac{n}{L}$

h) Deadline overrun = $C_i - D_i$

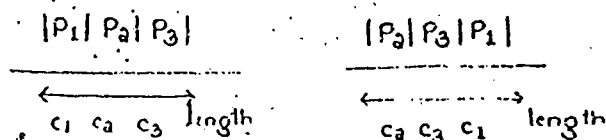
If $(C_i - D_i) < 0$ under run

If $(C_i - D_i) = 0$ on deadline

If $(C_i - D_i) > 0$ overrun.

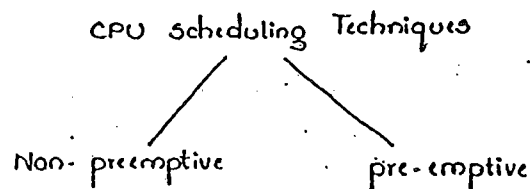
Schedule :-

n-processors. $n=3$. (P_1, P_2, P_3)



07/07/2010

wednesday



(1) First come first serve (FCFS) :

Criteria : Arrival Time (process)

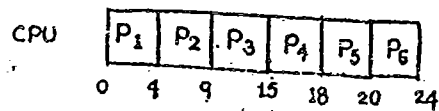
mode : non-preemptive.

P.NO	A.T	B.T (CPU)	CT	TAT	WT
1	0	4	4	4	0
2	1	5	9	8	3
3	2	6	15	13	7
4	3	8	23	20	17
5	4	9	32	28	24
6	5	4	36	31	26

$$TAT = CT - AT$$

$$WT = TAT - BT$$

Gantt chart :



Arrival Time of P₁ = 0

$$P_2 \text{ C.T.} = P_1 \text{ C.T.} + P_2 \text{ B.T.}$$

i.e., at '0' clock time, P₁ arrives

$$\text{Completion Time (P}_n\text{)} = \text{Completion Time (P}_{n-1}\text{)} + \text{Burst Time (P}_n\text{)}$$

Eg. 2 :-

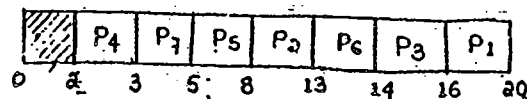
PNO	AT	BT	CT	TAT	WT
1	8	4	20	12	8
2	5	5	13	8	3
3	7	2	16	9	7
4	2	1	3	1	0
5	4	3	8	4	1
6	6	1	14	8	7
7	3	2	6	2	0

$$\text{TAT} = \text{CT} - \text{AT}$$

$$\text{WT} = \text{TAT} - \text{BT}$$

18

Gantt chart :



$$L = 20 - 2 = 18$$

$$L = \max. \text{ of } C_i - \min. \text{ of } A_i$$

$$= 20 - 2$$

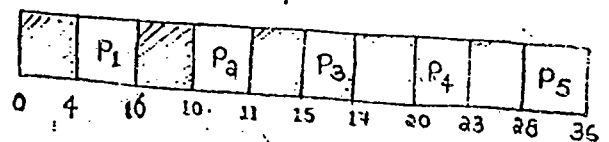
$$= 18$$

Since, there are no processes arriving at clock time = '0'. The process 'P₄' arrives at '2' clk time. So, the time (0-2) is subtracted from the total completion time. And, since 'P₄' arrives earlier than that of process 'P₁',

Process 'P₁' is considered first and then the other processes arrived within the particular burst time.

Eg 3 :-

PNO	AT	BT
1	4	2
2	10	1
3	15	2
4	20	3
5	28	8
		<u>16</u>



$$WT(P) = 0$$

$$L = 36 - 4 = 32$$

Shortest Job First (SJF) :

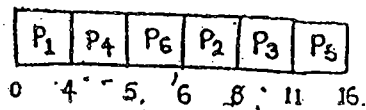
* It is also called as Shortest process next (SPN).

Criteria : Burst Time

Mode of working : Non-preemptive (default).

It also works under pre-emptive mode.

PNO	AT	BT
1	0	4
2	1	2
3	2	3
4	3	1
5	4	5
6	5	1



Initially, Process 'P₁' is ready for execution, since it has arrival time = 0. So, it has been processed.

for processing 'P₁', it requires 4 clock cycles (B.T=4), within the time, process P₄, P₆, P₂, P₃ are also ready for execution.

Within those processes, select the process which have less

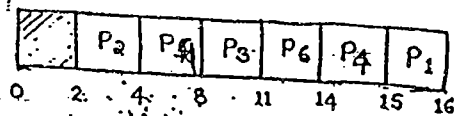
Burst time and process it. Here, P_4 has a B.T = 1. So, it is selected.

$\therefore$ Total clock cycles = $4+1=5$. At the time of 5th clock cycle, process, ' P_6 ' (new) is available on ready Queue. Since, the B.T = 1 of process ' P_6 '. after completion of ' P_4 ', ' P_6 ' is get executed, even though it added recently in queue.

Eg. 8 :-

PNO	AT	BT
1	5	1
2	2	2
3	3	3
4	2	4
5	4	1
6	3	3

14



$$16 - 2 = 14$$

(3) Shortest Remaining Time First (SRTF) :-

In pre-emptive mode, STF is known as shortest Remaining Time first.

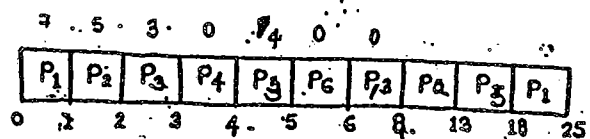
"Pre-emption of running process is based on the arrival of a new shorter process".

Criteria : - Burst Time

mode : pre-emptive.

Eg :-

PNO	AT	BT
1	0	8
2	1	6
3	2	4
4	3	1
5	4	6
6	5	1

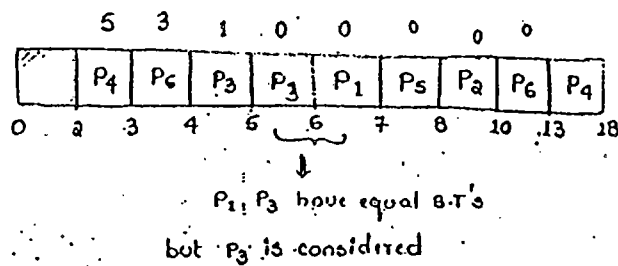


$$TAT(P_3) = 8 - 2 = 6$$

Initially, process P_1 has arrived, and within 1 clock cycle time P_2 arrives, then P_3 & soon. So, only 1 clock cycle is considered for P_1 within $B.T = 8$, so the remaining $B.T = 7$ remains. In P_2 , $B.T = 5$, $P_3 \Rightarrow B.T = 3$, $P_4 \Rightarrow B.T = 0$.

At clock cycle = 4, process P_5 also arrives, but if we consider it, it has more burst time than that of process P_3 . So, we consider P_3 as it is SRTF. Next, within 5 clock cycles P_6 also arrives, which have $B.T = 1$, so we consider P_6 after P_3 , and so on.... upto the end.

PNO	AT	BT
1	5	1 ✓
2	7	2 ✓
3	4	2 ✓
4	2	6
5	6	1 ✓
6	3	4



Performance of Shortest Job First (SJF):

Burst Time

Advantages:

- * Enhances Throughput
- * Average W.T & TAT reduces.
- * It is implementable, with predicted/ Estimated Burst times.

Disadvantage:

- * Starvation to longest job.
- * It is non-implementable, because burst times of processes are not mentioned.

Prediction Techniques for Burst Time :

(1) Process Size : (Bytes)

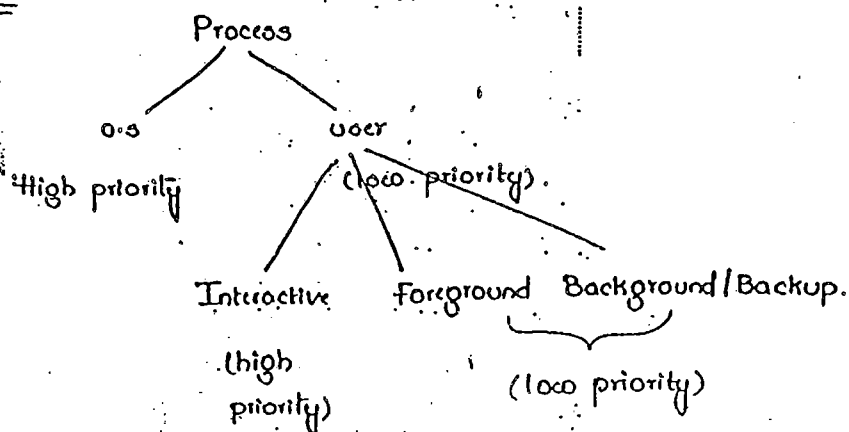
Based on past records, burst times, New burst times are predicted for new records. functional change may present.

Assume $P = 101 \text{ KB}$

$P_{old} = 101 \text{ KB} \text{ --- (20)}$

$P_{new} = 100 \text{ KB} \text{ --- (20)}$

(2) Process Type :



(3) Next cpu burst is the average of its previous Bursts :

Let P_i be the process. Let t_i be the real known Burst Time.

Let γ_i be the predicted Burst time.

$$\text{Then } \gamma_{n+1} = \frac{1}{n} \sum_{i=1}^n t_i$$

$$\gamma_i = c \text{ (guessed)}$$

P_i	RQ	CPU	I/O	RQ	CPU	RQ	CPU	RQ	...
	w_{r_1}	t_1	w_{r_2}	w_{r_3}	t_2	w_{r_4}	t_3	w_{r_5}	

(4) Prediction based on Exponential Averaging / Aging Algorithm:-

let $T_i \rightarrow$ predicted B.T

$t_i \rightarrow$ Real B.T

completed $\rightarrow$ n-CPU Bursts.

$$T_{n+1} = \alpha t_n + (1-\alpha) T_n \rightarrow (1)$$

$$0 \leq \alpha \leq 1$$

If $\alpha = 0$; $T_{n+1} = T_n$.

If $\alpha = 1$; $T_{n+1} = t_n$

If $\alpha = 0.5$; $T_{n+1} = \frac{1}{2} (t_n + T_n)$

$$T_{n+1} = \alpha t_n + (1-\alpha) T_n \rightarrow (1)$$

$$T_n = \alpha t_{n-1} + (1-\alpha) T_{n-1}$$

$$T_{n+1} = \alpha t_n + (1-\alpha) [\alpha t_{n-1} + (1-\alpha) T_{n-1}]$$

$$= \alpha t_n + \alpha (1-\alpha) t_{n-1} + (1-\alpha)^2 T_{n-1} \rightarrow (2)$$

$$= \alpha t_n + \alpha (1-\alpha) t_{n-1} + \alpha (1-\alpha)^2 t_{n-2} + (1-\alpha)^3 T_{n-2} \rightarrow (3)$$

(4) Highest Response Ratio Next (HRRN):-

Criteria : Response Ratio (RR) = $\frac{w + s}{s}$

where $w \rightarrow$ waiting time of process so far.

$$s = \frac{\text{Service time}}{\text{Burst time.}}$$

when $w=0$, $R.R = s/s = 1$.

All processes must have $RR \geq 1$.

mode of scheduling: non-preemptive.

Non-preemptive SJF :-

P.NO	AT	BT
1	0	3
2	2	6
3	4	4
4	6	5
5	8	2

HRRN :-

P_1	P_2	P_3	P_5	P_4	
0	3	9	13	15	20

NP-SJF :-

P_1	P_2	P_5	P_3	P_4	
0	3	9	11	15	20

$$RR_3 = \frac{5+4}{4} = 9/4$$

$$RR_4 = \frac{3+6}{5} = 8/5$$

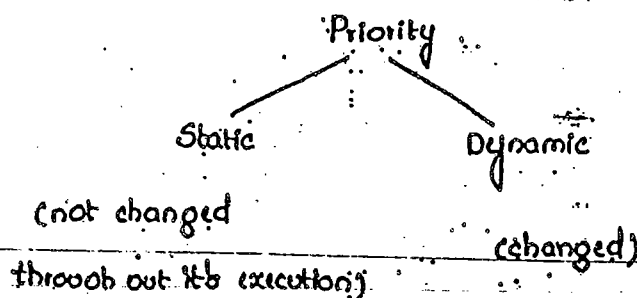
$$RR_5 = \frac{1+2}{2} = 3/2$$

$$RR_4 = \frac{7+5}{5} = 12/5$$

$$RR_5 = \frac{5+2}{2} = 7/2 \text{ (high)}$$

* process having high R.R. it is first considered.

* At clock cycle 9, Process P₅ is in Ready Queue, which have very little B.T. & Process P₃ executed, instead of P₅ because of long time waiting. This scheduling algorithm gives importance to long waiting processes as well as B.T processes.

(5) Priority based scheduling :-Criteria : priorityMode : non-preemptive

Increasing the priority level regularly at the runtime is called as
"Aging Algorithm".

Drawback :

Starvation.

* Higher Integer

high priority

Eg:-

Priority	PNO	AT	BT
4	1	0	4
5	2	1	5
6	3	2	8
8	4	3	6
2	5	4	3
12	6	5	5

NP- priority :-

P_1	P_4	P_6	P_3	P_2	P_5	
0	4	10	15	23	28	31

preemptive priority :-

	3	4	7	5	2	9						
CPU	P ₁	P ₂	P ₃	P ₄	P ₄	P ₆	P ₄	P ₃	P ₂	P ₁	P ₅	
	0	1	2	3	4	5	10	14	21	25	28	31

2a:

Prio.	PNO	AT	BT
12	1	5	4
9	2	6	5
7	3	2	4
6	4	1	6
4	5	4	2
8	6	3	3

	5	3	2	1								
	P ₄	P ₃	P ₆	P ₆	P ₁	P ₂	P ₆	P ₃	P ₄	P ₅		
	0	1	2	3	4	5	9	14	15	18	23	25

Round - Robin (m.p / Time sharing) :

Criteria : A.T + Time Quantum (TQ)

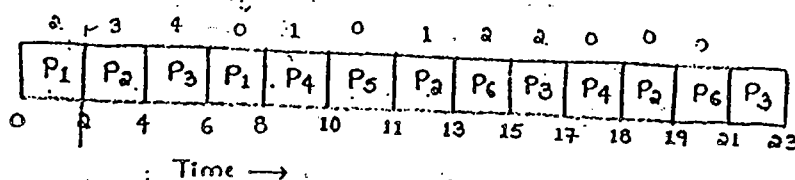
Slice

mode : pre-emptive

TQ = 2

PNO	AT	BT
1	0	4
2	1	5
3	2	6
4	3	3
5	4	1
6	5	4

R.Q : $P_1 P_2 P_3 P_1 P_4 P_5 P_2 P_6 P_3 P_4 P_2 P_6 P_3$



* Initially, P_1 arrives and executes till B.T = 2, (since Time slice = 2). Then, within 2 clock cycles, processes P_2, P_3 also arrives, then considered their B.T at TQ = 2 and hence P_1 requires another 2 clk cycles it must again be maintained in Ready queue.

* At time = 8, all the processes arrives, based on their b.T's, ready queue is maintained.

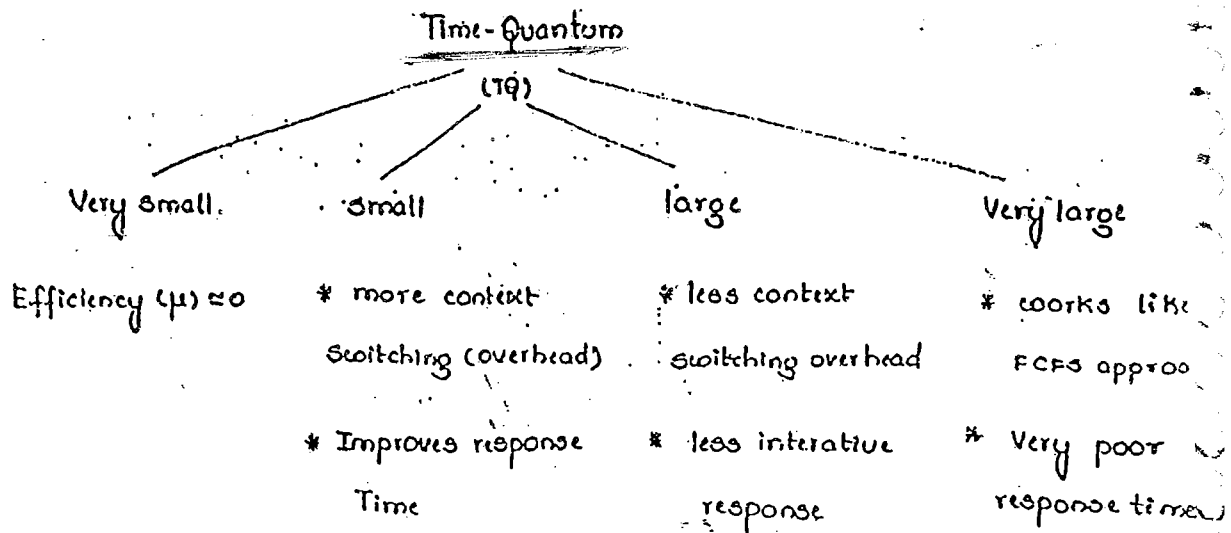
* If TQ is very small, efficiency (μ) = 0.

* If TQ is low $\Rightarrow$ context switching occurs.

* If TQ is high $\Rightarrow$ FCFS approach is followed

} so TQ value must be chosen moderately.

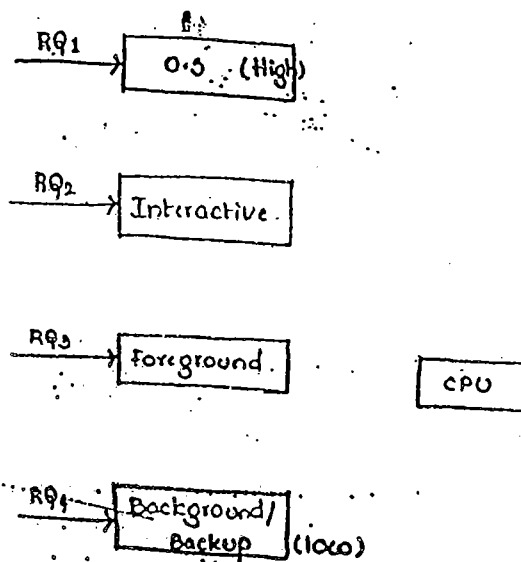
Performance of Round Robin:



08/07/2010

Thursday

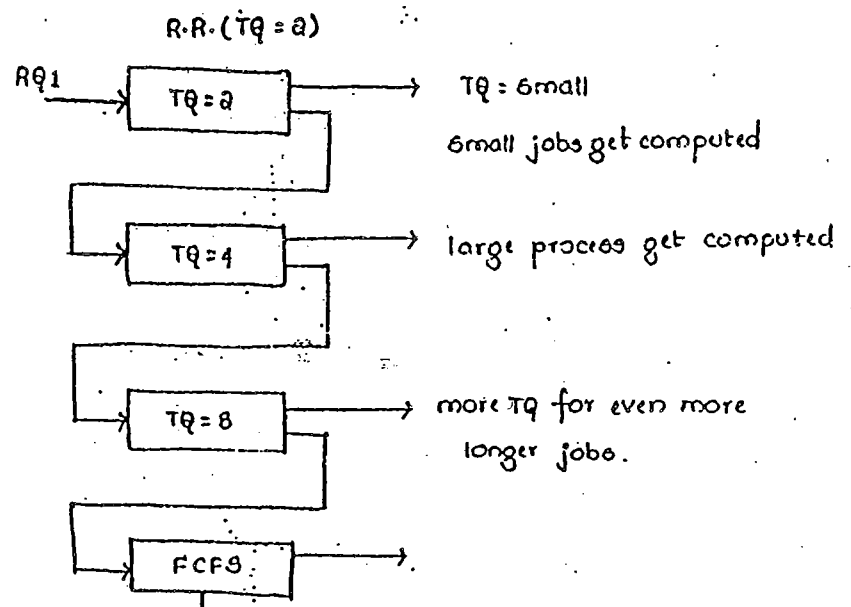
Multi-level Queue Scheduling



* In multi-level queue, the drawback is that, the second queue is serviced only when first queue is empty / complete.

* Here, the feedback is pre-empted to the same Queue.

* So, we have multi-level feedback queue to overcome this drawback.



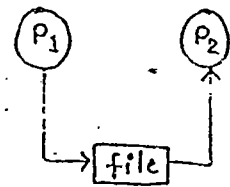
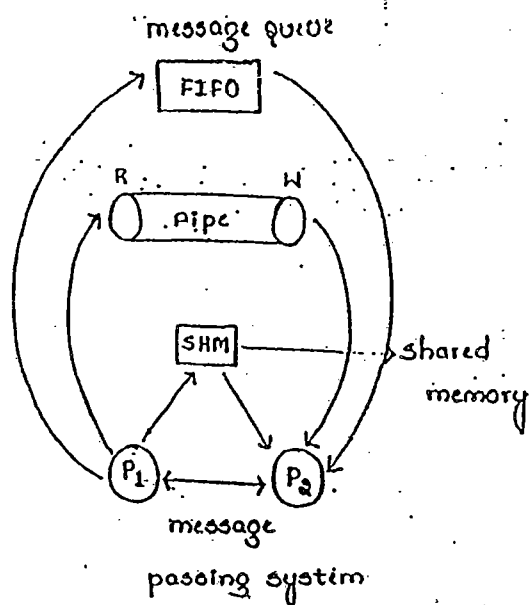
Yet, all the jobs are not computed,
Using FCFS all the jobs get completed
one-by-one.

Inter Process communication & Synchronization:

Inter process communication (I.P.C):

* For the communication among any two process, there must be a media to communicate.

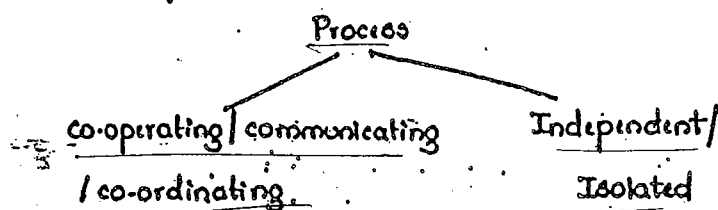
Eg.:- file (It is not always used)



Problems due to lack of synchronisation:-

- * Loss of data
- * Inconsistency (wrong results)
- * Dead locks.

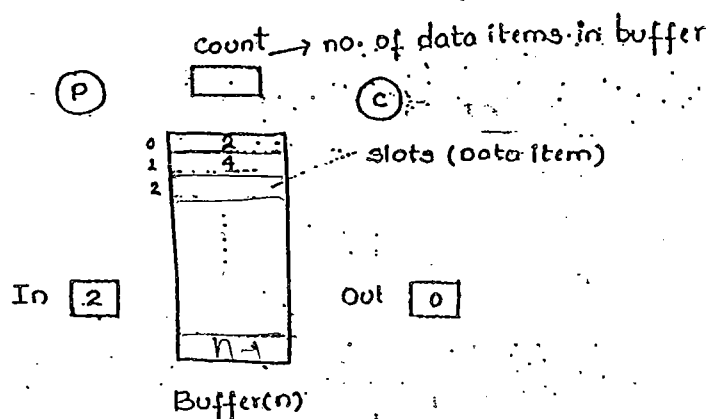
How to achieve Synchronization?



Co-operating process :-

- * Two (or) more processes are said to be co-operative, if they get affected (or) affects the execution of other process.
- * Otherwise, they are said to be Independent.

Producer-Consumer problem :-



- * If Buffer is full, producer get affected.
- * If Buffer is empty, consumer get affected.

```
#define n 100
```

```
int Buffer[n];
```

```
void producer(void)
```

```
{
```

```
    int item, in = 0;
```

```
    while(1) {
```

```
    {
```

```
        ProduceItem(item);
```

```
        while(count == n); // Busy wait.
```

```
        Buffer[in] = item;
```

Ram
Mond

```

    in = (in+1) mod n;
    count = count + 1;
}
}
void consumer(void)
{
    int itemc, out = 0;
    while(1)
    {
        while (count == 0);
        itemc = Buffer[out];
        out = (out+1) mod n;
        count = count - 1;
        processitem(itemc);
    }
}

```

Lack of Synchronisation in Producer-consumer problem :-

Inconsistency :-

In producer,

count = count + 1;

// This step needs to fetch

(1) load Rp, m[count]

the variable "count" and

(a) Inc Rp

then perform the operation.

(b) store m[count], Rp

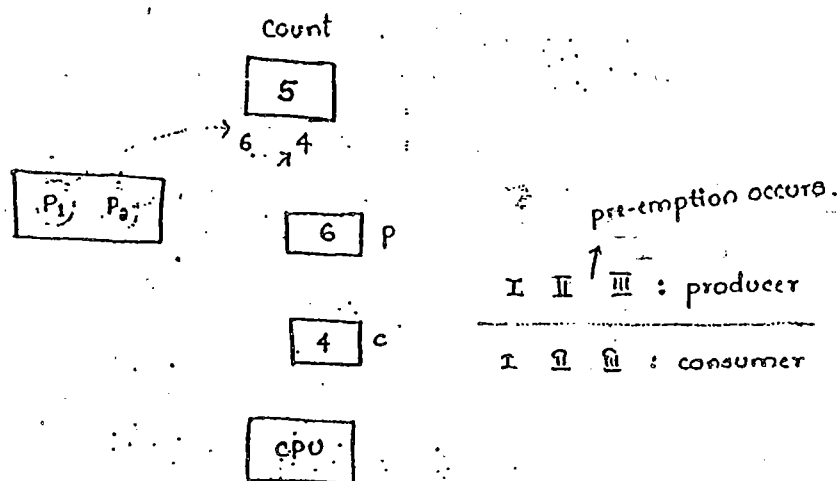
In consumer,

count = count - 1 ;

(1) load R_c, m[count]

(2) Dec R_c

(3) Store m[count], R_c



* In producer,

while first two steps get executed, pre-emption occurs. i.e.,

consider the count = 5 (initially). In step 1, the value is loaded into register 'R_p'. and then in step 2, the value in the register get incremented. so, the value of count = 6.

* Before storing the "count" value from the register, it got pre-empted. so, the producer have the count value = 6.

* In consumer,

count value = 5 is stored in a register 'R_c', and then decremented. Before transferring the value from 'R_c' it get pre-empted. so it have count = 4. so, there is inconsistency, because count value is never equal to 5.

13/07/2010

Tuesday

Synchronization Mechanisms.

Producer - Consumer :

Lack of Synchronization :

- * Inconsistency
- * Loss of data
- * Deadlocks

Terminology :

1) Critical Section() & non-critical section() :-

- * Critical section is that part of a program where shared resources are accessed.
- * Non-critical section is that part of a program where no shared resources are accessed.

CS $\Rightarrow$ shared resource (any variable/device)

(a) Race conditions (concurrency) :

processes racing to get into critical section.

(b) pre-emption :

A process get executed in the middle of other process.

count = count + 1

(1) load

(2) Inc.

* (3) store.

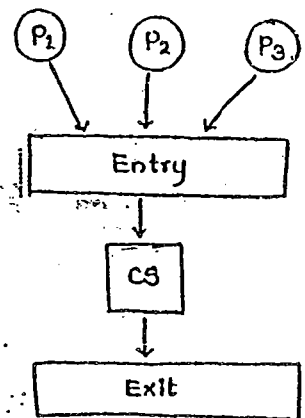
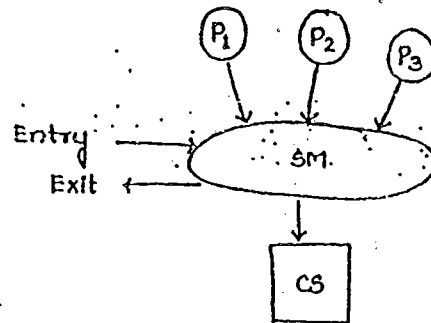
(4) mutual exclusion :

No two processes may be present in the critical section at same time.

Architecture / model of Synchronisation mechanism:

Synchronisation mechanism:

- * To ensure mutual exclusion so that there is no problem of Race condition among processes.

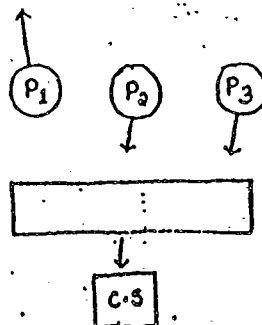


Requirements of Synchronisation mechanism:

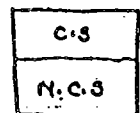
(1) mutual exclusion must always be guaranteed.

(a) progress:

process running outside critical section should not block (the process or) the other interested process from accessing / entering the critical section.

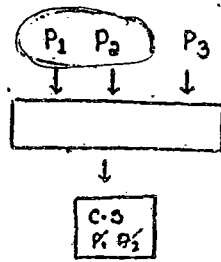


program.



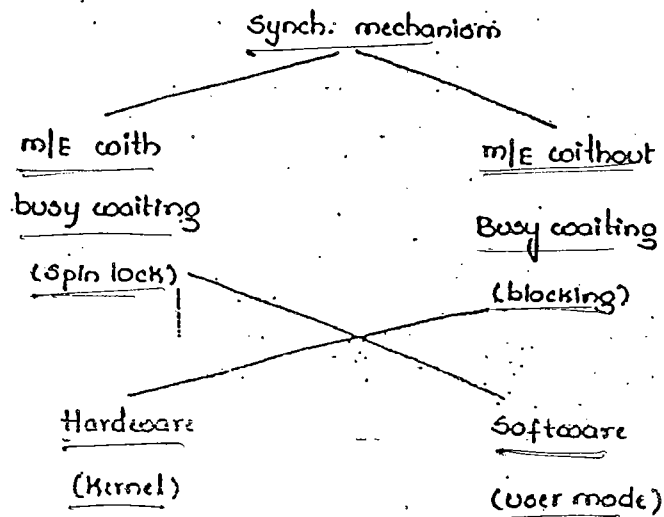
(3) Bounded waiting:

- * No process has to wait forever to access its critical section.



If P_1 enters C.S. and after leaving, if P_2 wants to enter C.S., then it can enter and after performing operations, it leaves. Then, if again P_1 wants to enter along with P_3 , where P_1 has high priority than P_3 , then P_1 enters & then P_3 and so on, where there is no chance for P_3 to enter C.S. (i.e., P_3 remains idle forever).

(4) No Assumptions about the speed (or) no. of CPU's may be assumed.



Disabling Interrupts :-

- * m/e without busy waiting.
- * multi-process
- * kernel mode.

* The main cause of pre-emption is "interrupt".

* Disable the source of pre-emption in the critical section at entry level and enable the interrupts at exit level.

* It guarantees m/e because if P_1 running in c.s., other interrupts are disabled.

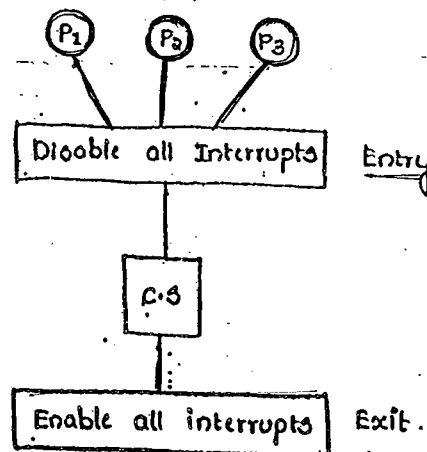
* critical section

* Race condition

* pre-emption.

* Enabling/ disabling of interrupts done only in kernel mode.

So, o.s process can only enable/disable the interrupts and user process cannot enable/disable $\Rightarrow$ Restriction to o.s process.



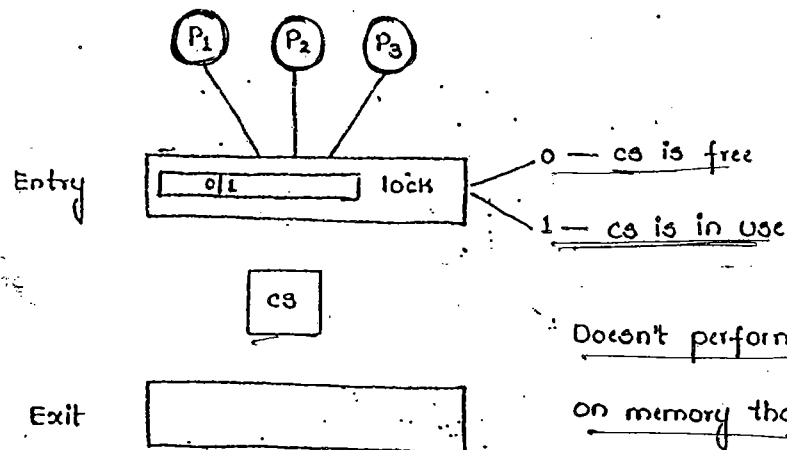
*

(a) Lock variable :

* Semaphore mechanism implemented in user mode.

* M/E with busy waiting.

* multiprocess.



Doesn't perform direct manipulation
on memory that load into ALU
reg.

void Entry-section (int process)

```
{
    while (lock != 0); // Busy waiting.
    lock = 1;
}
```

cs

Here, initially the lock value is '0' (lock=0)

when, P_1 needs to enter c.s, it follows
all the steps.

void Exit-section (int process)

```
{
    lock = 0;
}
```

Step 1: The lock value is stored in the register
 R_i .

Step 2: It is compared whether lock is '0' or not.

Step 3: If lock value is not zero (i.e., lock=1), then again process same steps (1&2)

Step 4: If lock=0, then the process enters c.s and while, entered then it changes
lock=1.

Assembly language :-

Entry section :

1. load R_i, m[lock]

2. cmp R_i, #0 ;

3. JNZ steps

4. store m[lock], #1 ;

c.s

6. Exit section :

store m[lock], #0 ;

R₀ : P₁ P₂ P₃

lock

0

Entry section

R₁ 0

R₂ 0

CPU
P₁

c.s P₂
P₁

c.s P₂
P₁

P₁ : 1 2 3 preempt

P₂ : 1 2 3 4

5

P₁ : 4 5

* It fails to guarantee mutual exclusion.

* Busy waiting results in wastage of CPU cycles / CPU time.

Hint for testing either supports to guarantee m/E :-

(1) Identify the shared variable (lock)

(2) Read the value of shared variable (load)

(3) compare (optional)

(4) Update / action (based on result of comparison)

⇒ If process 'P₁' entered c.s, and before changing lock=1, it get pre-empted.

Since the lock=0, another process 'P₂' wants to enter c.s and get resides in-CPU, and gets into c.s.

* Since P₁ and P₂ gets into c.s simultaneously, it fails to guarantee mutual exclusion.

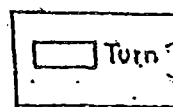
(3) Strict Alternation :

* slow at user mode

(P₀, P₁) or (P₁, P₀)

* 2-process

* M/E with busy waiting



CS

* Strictly on alternate basis, processes (P₀ & P₁) takes turn to enter c.s.

void P₀(void)

```

{
  while(1)
  {
    Non_cs()
    Entry while(turn != 0); // busy waiting
    CS
    Exit turn = 1;
  }
}

```

If turn is not zero then P₁ must wait

Because "turn" value is not immediately updated, but updated at exit section, so, it guarantees M/E.

Void P₁(void)

```

{
  while(1)
  {
    Non_cs()
    while(turn != 1); busy wait
    CS
    turn = 0;
  }
}

```

wait P₁ until turn become 1

* Guarantees M/E.

* Not Guarantees progress

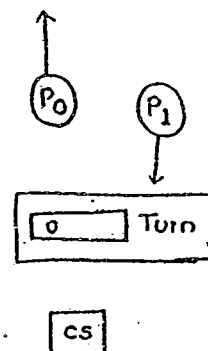
Direct progress

Indirect progress

direct progress

direct progress :-

Initially, the $turn=0$, and if P_0 is not interested in c.s, and P_1 is interested, then the turn must be equal to '1' to enter into c.s. But, unless ' P_0 ' enters, "turn" can't be changed, and ' P_0 ' not interested to do so. Hence, progress is not guaranteed.

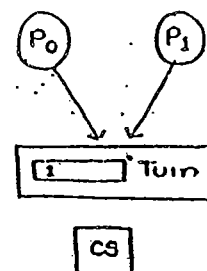


' P_0 ' which is outside block

' P_1 ' directly not to enter c.s.

Indirect progress :-

If ' P_0 ' enters c.s and while exiting turn changed to '1', so that ' P_1 ' need to enter. If again ' P_0 ' wants to enter (i.e., interested) and P_1 is not interested. Then, P_0 is discarded to enter c.s because $turn=1$ and it needs P_1 to change $turn=0$. Hence, progress is not guaranteed.



P_1 is blocking P_0 indirectly.

(4) Peterson's solution (Dekker's Algorithm):-

- * m/e with busy waiting
- * slow at user mode.
- * 2-process solutions (P_0 & P_1)

↓
disadvantage.

```
#define N 2
```

```
#define TRUE 1
```

```
#define FALSE 0
```

```
int interested[N] = {false};
```

```
int turn;
```

```
void Entry-section(int process)
```

```
{
```

```
    (1) int other;
```

```
    (2) other = 1 - process;
```

```
    (3) interested[process] = TRUE;
```

```
    (4) turn = process;
```

```
    (5) while(interested[other] == TRUE && turn == process); // busy waiting
```

```
}
```

```
void Exit-section(int process)
```

```
{
```

```
    interested[process] = false;
```

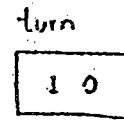
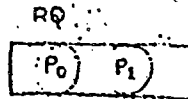
```
}
```

* progress is guaranteed, m/e guaranteed.

* Bounded waiting is guaranteed (since, it is not guaranteed for more than a process).

Drawback:

* wastage of CPU time.



P₀ : 1, 2, 3

P₁ : 1, 2, 3, 4, 5

P₀ : 4, 5

P₁ :

interested[0] = F

int'd[0] = ~~F~~ T

int'd[0] = F T

Interested

turn

Hardware Solution

(5) Test & Set lock instruction (TSL) :-

(privileged instruction $\Rightarrow$ atomically)

- * Slows at user mode
- * mle with busy waiting
- * multi-process (advantage over peterson)

Eg :- TSL register, flag ; $\left\{ \begin{array}{l} 0 - \text{cs is free} \\ 1 - \text{cs is busy} \end{array} \right.$

** "copies the value of flag into register and sets the value of flag to one (always)".

Entry-section :

- (1) TSL regi, m[flag];
- (2) cmp regi, #0;
- (3) JNZ step1
- (4) CS
- (5) store m[flag], #0;

Priority Inversion (P/TSL) :

- * Preemptive priority based CPU scheduling is used.

If $H > L \Rightarrow$ Deadlock

when a high priority process enters the CPU. And P_L requires CPU to perform its instructions, in order P_H to enter into c.s. So, P_H enters in CPU and needs to enter c.s & P_L is in c.s and needs CPU to exit. Hence, Deadlock occurs.

RQ :

$P_H P_L$

CPU
 $P_H P_L$

P/TSL Entry
Section

CS
 P_L

Producer & Consumer :

* M/E without Busy waiting (Blocking)

(i) sleep() and wakeup() :-

```
void producer(void)
```

```
{
```

```
    int itemp, in = 0 ;
```

```
    while(1)
```

```
    {
```

```
        produce item(itemp);
```

```
        if (count == N) sleep();  $\Rightarrow$  If Buffer = full
```

```
        Buffer[in] = itemp;
```

producer $\Rightarrow$ sleep()

```
        in = (in + 1) mod N ;
```

```
        count = count + 1 ;
```

```
        if (count == 1) wakeup(consumer);  $\Rightarrow$  when item is placed in
```

Buffer consumer $\Rightarrow$ wakeup

```
    }
```

```
void consumer(void)
```

```
{
```

```
    int itemc, out = 0 ;
```

```
    while(1)
```

```
    {
```

```
        if (count == 0) sleep();
```

```
        itemc = Buffer[out];
```

```
        out = (out + 1) mod N ;
```

```
        count = count - 1 ;
```

```
        if (count == N - 1) wakeup(producer);
```

```
    }
```



N = 3

~~0~~ ~~1~~ ~~2~~ 3



0	X
1	Y
2	Z

Buffer (0-a)

(c)

Preemption

sleep(2

⑤ : x

Inconsistency Y
Z

sleep(); 7 deadlock

```

(c): sleep();

```

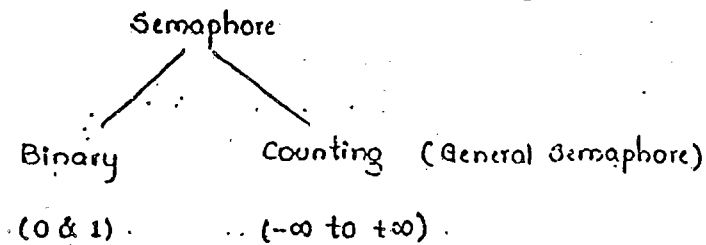
* while consumer executing instructions, before "sleep()" gets executed, pre-emption takes place and producer executes instructions and after buffer is full, producer goes to sleep() state assuming that consumer could wake-up, when consumer starts executing, sleep() instruction gets executed so, consumer expects that producer could be wake up. Hence, it represents the situation of "Deadlock".

17/07/2010

Saturday

Semaphores

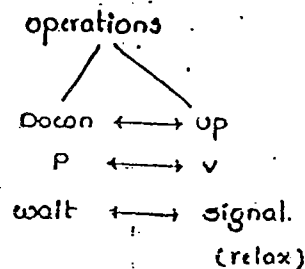
- * A variable (semaphore) that takes on integrated values.
↓
(Integers)



↓
Critical section problem (mutex)

↳ mutual exclusion.

- * It is an operating system resource.
- * Semaphore operations executed in kernel mode (automatically)
↓
(no preemption)
- * Implementation by Dijkstra (variable)



struct SEMAPHORE

{
 int value;

 Queue Type L;

};

// list of PCB's of those processes that get blocked while performing "down" operation unsuccessfully.

SEMAPHORE s;

value = 4;

DOWN(s);

<c.s>

"Semaphore" gets executed in kernel mode.

* If the value after decrementing is negative, then it is unsuccessful.

* If value is zero (or) positive then it is successful.

(+) value $\rightarrow$ Successful down processes.

(-) value $\rightarrow$ Blocked processes.

Eg:-

s-value = 4

Down.

4 - 1 = 3

3 - 1 = 2

2 - 1 = 1

1 - 1 = 0

0 - 1 = -1

-1 - 1 = -2

1

1

1

1

-1

-2

4 Successful processes [(+)ve. value of counting available in stack Semaphore always indicate no. of down operations successful]
2 blocked processes (unsuccessful processes) $\Rightarrow$ waiting for semaphore.

DOWN(SEMAPHORE s)

s.value = s.value - 1;

if (s.value < 0) //unsuccessful.

put this process (PcB). in SL() & Block it (sleep());

After Blocking the process, just it moves off (i.e., no down operations are performed)

UP(SEMAPHORE s)

s.value = s.value + 1;

if (s.value < 0)

select a process from S.L() and wakeup();

wakeup process gets into c.s but not

If s = -2 perform "down" operation

After $P_1 = -2 + 1 = -1$

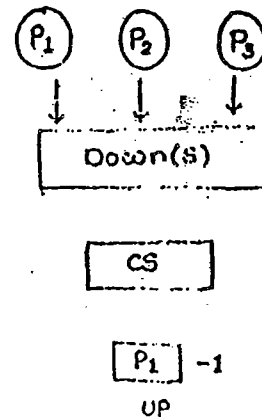
$P_2 = -1 + 1 = 0$

$P_3 = 0$

$= 0 + 1$

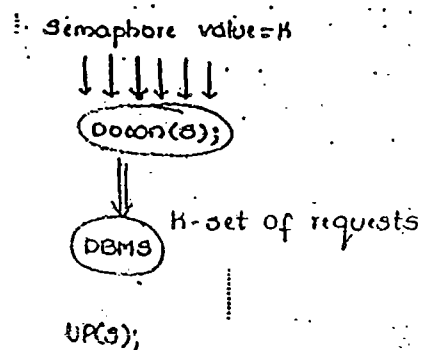
$= 1$

* When we have 3 processes, semaphore value starts with 1 $\Rightarrow$ performing down operation on it, the value becomes '0'. (i.e., the down operation is successful, & it enters into critical section). After P_1 enters into c.s, all the other processes perform down operation
($P_1 = 0, P_2 = -1, P_3 = -2$)



* Process never get blocked while performing "up" operation and it gets blocked while performing "Down" operation.

Allowing K processes into c.s



After process wakes up, don't perform any down(s) operations, simply execute the critical section.

Eg:- $S = 12$

16 p, 2 v, 8 p, 1 v, 4 p, 3 v, 8 p, 2 v

S.L() $\rightarrow$ Semaphore Queue consists list of processes

$S = 12$ (16 down) p $\rightarrow$ down
 $= -4$ (2 up) v $\rightarrow$ up
 $= -2$ (8 down)
 $= -10$
 $= -9$
 $= -13$
 $= -10$
 $= -16$

Binary (mutex) Semaphore (0/1):

```
struct BSEMAPHORE
```

```
{
```

```
    enum value (0,1);
```

```
    Queue Type L;
```

```
};
```

```
BSEMAPHORE S;
```

```
S.value = 1;
```

```
DOWN(S);
```

```
<C.S>
```

```
DOWN (BSEMAPHORE S)
```

```
{
```

```
    if (S.value == 1) // Successful.
```

```
        S.value = 0;
```

```
    else //unsuccessful.
```

```
    { put this process (PCB)
```

```
      in S.L() & Block it
```

```
      sleep();
```

```
    }
```

```
}
```

After performing down() operation
on B semaphore,

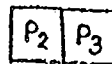
if value = 0 $\Rightarrow$ unsuccessful

value = 1 $\Rightarrow$ successful.

(1-1)

If $S=1$, $\Rightarrow P_1 = 0$ (After down)

operation, semaphore
must be 1).



P₂ checks semaphore value. Since the
value = 0, no down() operation is per-
formed. Simply it moves to blocked
state. Similarly, by having semaphore
value, P₃ also gets blocked.

Condition-I: If there are blocked processes.

(or)

Condition-II: If there is a process in critical section

Then, value of Binary Semaphore must be zero.

After completion of all processes, semaphore value gets incremented.

(Because chance given to newly entered processes).

UP(BSEMAPHORE S)

```

{
    If (S.L() is not empty)
    {
        select a process from
        S.L() and wake up()
    }
    else
    {
        S.value = 1;
    }
}

```

user doesn't know no. of processes present in the Queue by just having semaphore value, one can't decide no. of processes. So, we perform operation on blocked Queue.

Semaphore value = 0 $\Rightarrow$ It represents that there is no completion to present blocked processes.

After completion of processing all the jobs in blocked Queue, increment semaphore value. It represents that don't release semaphore until all the jobs are completed.

Initial semaphore = 0, no blocked processes

S = 1

seq 10p, 3v, 15p, 7v, 8p, 2v, 3p, 1v S=0

0 1 0 1 0 1 0 1
9 9 6 21 14 21 10 15 22

p = down

v = up

Blocked process

9	6	21	14	22	20	23	22
---	---	----	----	----	----	----	----

For Cases of Binary Semaphore:-

Case (i) :-

$S = 1$

S value becomes = 0

$P(S);$

Status = successfully.

Case (ii) :

$S = 1$

$S = 1$

$V(S);$

Status = successful.

Case (iii) :

$S = 0$

$S = 0$

$P(S);$

Status = unsuccessful.

Case (iv) :

$S = 0$

first value (initial value).

$V(S);$

(Up operation)

$S = 1$

Status = successful.

Classical Interprocess Communication problems.

(1) producer-consumer :-

```
#define N 100
```

```
int Buffer[N];
```

```
Semaphore empty = N; // no. of empty slots in Buffer
```

```
Semaphore full = 0; // no. of full slots
```

```
Bsemaphore mutex = 1; // used to ensure m/e between P & C on  
buffer.
```

```
void producer(void)
```

```
{
```

```
    int itemp, in = 0;
```

```
    while(1)
```

```
    {
```

```
        producer_item(itemp);
```

(Blocking producer)

```
        down(empty); // check for Buffer is full or not.
```

```
        down(mutex)
```

If Empty = 0, down(empty) = 0-1
= 0

```
        Buffer[in] = itemp;
```

↓
Buffer = full, then block producer.

```
        in = (in+1) mod n;
```

```
        Exit ← { up(mutex); // release the critical section.  
                up(full);
```

```
        } consumer_item(itemp);
```

```
    }
```

```
void consumer(void)
```

```
{
    int itemc, out=0;
```

```
while(1)
```

```
{
```

```
    consumer-item(itemc);
```

```
Enter { Down(full); } Interchange if
        Down(mutex); } deadlock occurs to
```

```
    itemc = Buffer[out]; up(full)
```

```
    out = (out+1) mod N;
```

```
    up(mutex);
```

```
    up(empty);
```

```
    producer-item(itemc);
```

```
}
```

```
}
```

```
void producer(void)
```

used for maintaining synchronization between producer & consumer.

Entry of cs

```
{
    Down(empty);
```

used for updating "count" values of buffer. It is c.s for maintaining consistency of information.

```
    Down(mutex);
```

```
    up(mutex);
```

```
    up(full);
```

```
}
```

```
void consumer(void)
```

```
{
```

```
    Down(full);
```

```
    Down(mutex);
```

```
    up(mutex);
```

```
    up(empty);
```

```
}
```

Consumer calls
wakeup producer

(producer wakes up consumer)

↓
whenever multiple semaphores

are properly used (or else

it leads to deadlock.

(because of "mutex" & "full" values)

(a) Reader-writer problem :-

(1) 1 Reader - 1 writer

(2) Multiple Readers - 1 writer

(3) Multiple Readers - Multiple writers

* we can perform multiple "Read" operations at a time.

Database enters into critical section, when there is a writer. so, at a time, only one write operation can be performed.

int RC=0; // Readers count

Semaphore mutex = 1; // used for mutual exclusion b/w readers)

RC must be in c.s.

Semaphore db = 1; // used to ensure m/e between readers & writers
on DBMS

void Reader(void)

{

 cobble(1)

{

 Down(mutex); // To increment readers count, check "mutex" value

 RC = RC + 1;

 if (RC == 1) // If first reader enters into DBMS, It's responsibility
 DOWN(db); is to check whether any writer present in c.s.
 UP(mutex); (on not using (RC == 1) condition.

 CS < READ-DBMS >

 DOWN(mutex);

 RC = RC - 1;

```
if (Rc == 0)
```

```
UP(db);  $\Rightarrow$  release db
```

```
UP(mutex);  $\Rightarrow$  release mutex
```

supposed db have writer, its val becomes = 0. so, no reader enter in the first stmt.

```
void writer(void)
```

```
{
  while(1)
```

```
{
  down(db);
```

```
< update - demo >
```

```
up(db);
```

If first writer is successful to enter into a critical section, it release the mutex b/w the readers for allowing many no. of readers. Again decrementing the 'Rc' count, use critical section of Readers.

(3) Dining philosopher's problem:-

Normal case :-

```
void philosopher(int i)
```

```
{
  while(1)
```

```
{
  think(i);
```

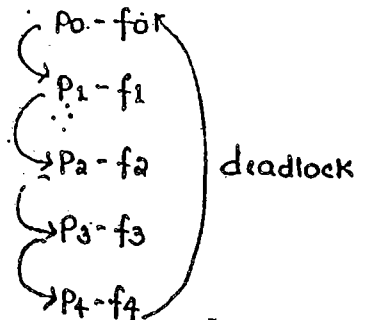
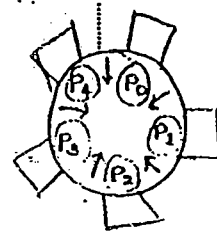
```
take_fork(i);
```

```
Take_fork((i+1) % N);
```

```
eat(i);
```

```
put_fork(i);
```

```
put_fork((i+1) % N);
```



• To avoid the problem of deadlock, in this, philosopher breaks the rule :-

i.e., Even position member follow one approach &

odd position members follow another approach.

Actual problem :-

• Philosophers take the position on a table, they first try to take left hand side fork, if they succeed, then they take right fork. If both the forks are available, then they start eating & then put off the forks.

• But, at a time, all philosophers can't have both forks, where dead lock situation occurs.

Eg:- If P_0 first takes f_0 , P_1 takes f_1 , P_2 takes f_2 , P_3 takes f_3 , P_4 can't have its left fork because P_0 's right is P_4 's left, hence, dead lock occurs. So, no one wants to release their left forks.

State[N] // stores state of philosopher either thinking
(or) eating

Semaphore mutex = // used for releasing & taking the fork $\Rightarrow$ It is C.S.

Semaphore $s[N] = \{0\};$

```
#define N 5
```

```
#define THINKING 0
```

```
#define HUNGRY 1
```

```
#define EATING 2
```

```
#define LEFT (i+N-1) % N
```

```
#define RIGHT (i+1) % N
```

```
int state[N] = {0}; // Either thinking or hungry or eating.
```

```
Semaphore mutex = 1; // To handle forks either by taking or releasing
```

```
void philosopher(int i)
```

```
{
    while(1)
```

```
{
```

```
    Think(i); // initial state
```

```
    take_forks(i);
```

```
    eat();
```

```
    put_forks(i);
```

```
}
```

```
}
```

```
void take_forks(int i)
```

```
{
```

```
    Down(mutex); // taking forks.
```

```
    state[i] = HUNGRY;
```

```
    test(i);
```

```
    Up(mutex);
```

```
} DOWN(state[i]); // fails to take forks so blocked.
```

```
void put_forks(int i)
```

```
{
```

```
    DOWN(mutex);
```

```
    state[i] = THINKING;
```

```
    test(LEFT);
```

```
    test(RIGHT);
```

} state again changed
to THINKING

```
    UP(mutex);
```

```
}
```

```
void test(int i) // testing forks are available or not
```

```
{
```

```
    if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] !=
```

EATING)

```
    {
```

```
        state[i] = EATING
```

```
        UP(s[i]); // Eating is completed & set s[i]=1
```

```
    }
```

```
}
```

Concurrency & Concurrent programming

Concurrency

Real
physical
concurrency

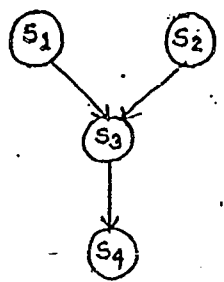
Pseudo
concurrency
(achieved by processor)

(achieved through multiprocessor,
array processor, vector processor)

Precedence Graph ^{dictates} ~~what~~ :-

* It indicates which statements executes
concurrently.

$S_1: a = b + c;$
 $S_2: d = e + f;$
 $S_3: k = a + d;$
 $S_4: l = k * m;$



There is no precedence.
Any statement can be executed first.

Concurrency conditions :-

$S_1: a = b + c;$
 $S_2: d = e * f;$
 $S_3: k = a + d;$
 $S_4: l = k * m;$

Read set $\rightarrow$ The values read into equation

$R(S_1) = \{b, c\}$

write set $\rightarrow$ update value in equation

$W(S_1) = \{a\}$

$a = \dots b + c \dots$ all a, b, c updated. so,

$R(S) = W(S) = \{a, b, c\}$

- (1) $R(s_i) \cap W(s_j) = \phi$ If Reading & writing set of two different processes equal, then we can't perform
 (2) $W(s_i) \cap R(s_j) = \phi$
 (3) $W(s_i) \cap W(s_j) = \phi$ (concurrent execution) at a time.

$a = c * d$

$a = 1 * m$ (not supported)

if (count $\neq$ 0) $\Rightarrow$ all processes are not completed.

fork & Join :

fork L_1 ;

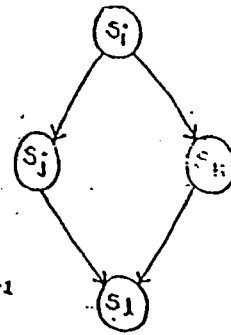
S_j ; // after completion of S_j go to S_1 .

go to L_1

$L_1 : S_k$ // it executes immediate instructions L_1

$L_1 : \text{Join count};$ $\Rightarrow$ in

S_1



Join (int count)

{

count = count - 1;

if (count $\neq$ 0) // all the processes are not

exit;

completed. so, there is

no execution of S_1 .

}

Count = 3

S₁;

fork L₁ (S₁ followed by fork)

S₂;

S₄;

fork L₂

S₅;

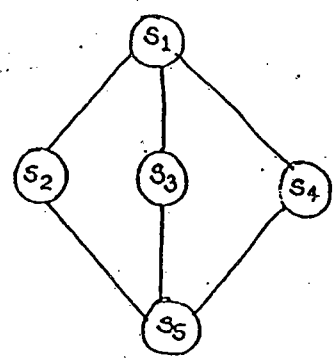
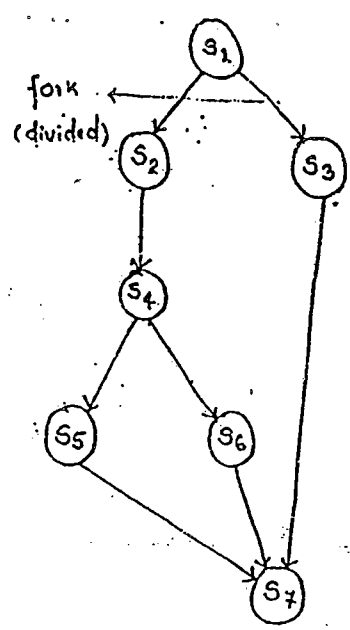
L₁: S₃

go to L₃

L₂: S₆

L₂: Join count

S₇;



Count = 3

S₁;

fork L₁

S₂;

go to L₁;

go to L₂;

L₁: fork L₂

S₃;

go to L₂;

L₂: S₄;

go to L₃

L₃: Join count (check for all processes either complete (or) not).

S₂, S₃, S₄ are independent processes so, they can be accessed in any way. But S₅ is purely dependent on S₂, S₃ and S₄.

Eg:- If S₂, S₃ accessed & completed, and S₄ needed to be accessed now, even though there is S₅ to be accessed, we won't access. After the completion of all the processes at the upper level (S₂, S₃, S₄), then only the last line process (S₅) must be computed. For such checking, "Join".

condition is used which counts the completed functions that are executed, used for completion of S_5 .

Parbegin - Parend & co-begin - Coend :-

begin({)

$S_1;$
 $S_2;$
 $S_3;$
end }

$\Rightarrow$



$S_0;$

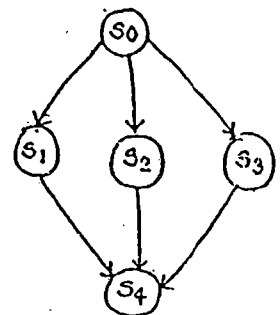
parbegin({)

$S_1;$
 $S_2;$
 $S_3;$

parend

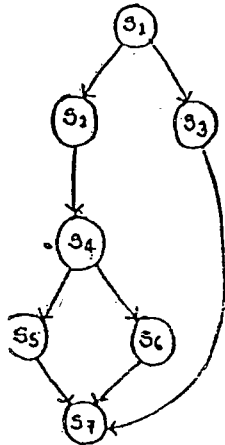
$S_4;$

$\Rightarrow$ They support parallelism



Begin-end $\Rightarrow$ represents sequential actions

Parbegin - Parend $\Rightarrow$ represents parallel actions.



$\Rightarrow$

$S_1;$

Parbegin

begin

$S_2;$
 $S_4;$

Parbegin

$S_5;$
 $S_6;$

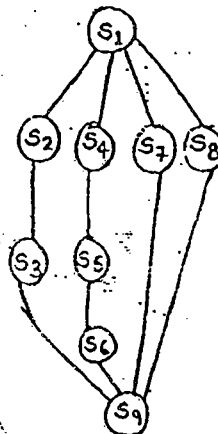
Parend

end

$S_3;$

Parend

$S_7;$



$S_1;$

Parbegin

begin

$S_2;$
 $S_3;$

end

begin

$S_4;$
 $S_5;$
 $S_6;$

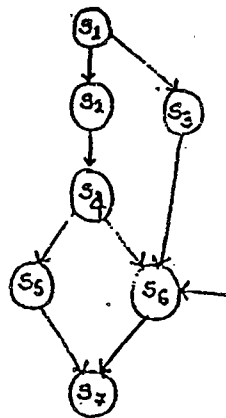
end

$S_7;$

$S_8;$

Parend

$S_9;$



For this graph, we cannot write a code because for execution of s_6 , we need s_4 and s_3 . With the help of code, we cannot impose such condition. We can overcome this problem by using "semaphore".

Parbegin - Parend with semaphore :-

Semaphore a, b, c, d, e, f, g = {0};

Parbegin

begin s_1 ;

Parbegin

$v(a)$;

$v(b)$;

Parend

end

begin $P(a); s_2; s_4$;

Parbegin

$v(c)$;

$v(d)$;

Parend

end

begin $P(b); s_3$;

$v(e)$;

end

begin $P(c); s_6$;

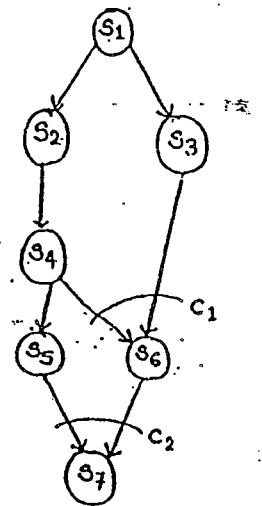
$v(f)$;

end

begin $P(d); P(e); s_6; v(g)$; begin $P(f); P(g); s_7$; end.

Parend

end



Implementation with fork & Join :-

$C_1 = 2; C_2 = 2$;

s_1 ;

fork l_1 ;

s_2 ;

s_4 ;

fork l_2 ;

s_5

go to l_3 ;

$l_1: s_3$;

$l_2: \text{Join } c_1$;

s_6 ;

$l_3: \text{Join } c_2$;

s_7 ;

```

Eg:- void P(void)      void Q(void)      main()
      {                {                {
          A;              D;              Parbegin()
          B;              E;              P();
          C;              }              Q();
      }                  }              parend()
                                }

```

What are the possible outputs for the above code :

- ~~(a)~~ A,B,C,D,E ~~(b)~~ D,E,A,B,C ~~(c)~~ A,B,C,E ~~(d)~~ D,A,C,E,B
~~(e)~~ E,C,B,D,A

A,B,C and D,E are sequential actions. C,B cannot be happen. So, only first three are possible outputs.

DEADLOCK (lockingup)

* Two (or) more processes are said to be involved in deadlock, iff they wait for the happening of an event, which would never happen.

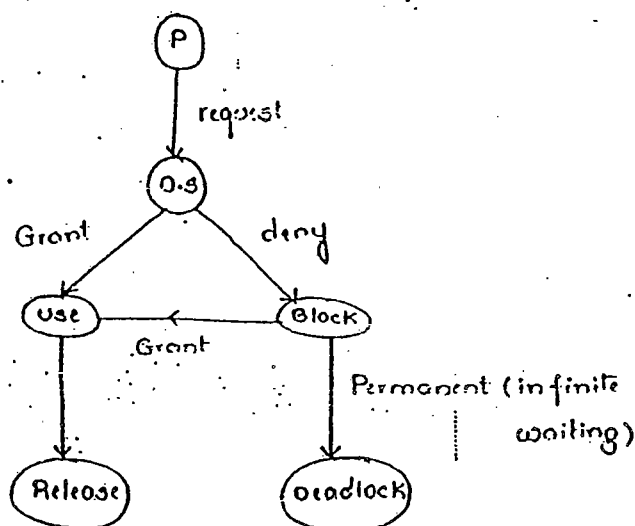
* System model (for avoiding deadlocks)

* 'n' processes ($P_1, \dots, P_n$)

* 'm' resources

* Starvation $\Rightarrow$ long waiting.

* Deadlock $\Rightarrow$ Infinite waiting.



* Processes make a request for resources. If the resources are available, then it is granted by O.S. If it takes some time for resource allocation, then process gets blocked. Blocked processes request for the resources, after certain time, and if again not granted, then the process moves to deadlock state.

A deadlock

Necessary conditions for occurring deadlock :-

situation can arise if the following four conditions hold simultaneously in the system.

(1) Failure of mutual exclusion.

(2) Hold & wait.

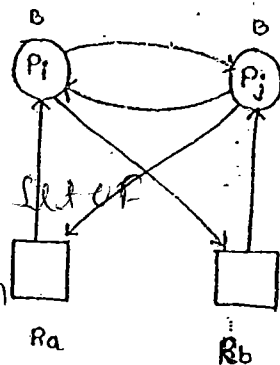
(3) No-preemption (process won't release their resources).

Deadlock Condition

(4) circular wait.

① If a resource type only one instance, then a cycle then must be a deadlock.

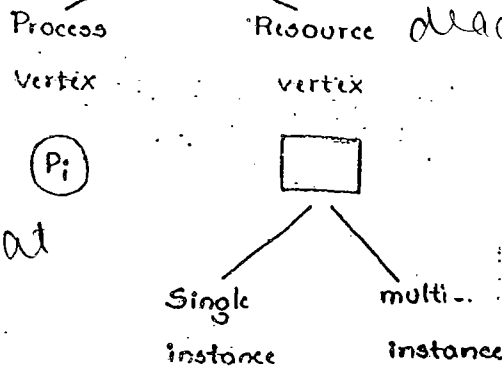
② If a cycle involve only a set of resource type and each has single instance then deadlock occurred.



③ each process involve in the cycle is deadlock.

④ Cycle in the graph both a necessary and sufficient condition for the existence of Resource deadlock.

If the resource has multiple instance then the cycle is necessary but not sufficient.

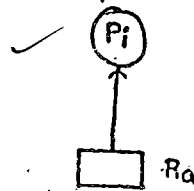


Ra []
(single copy)
Instance
processor

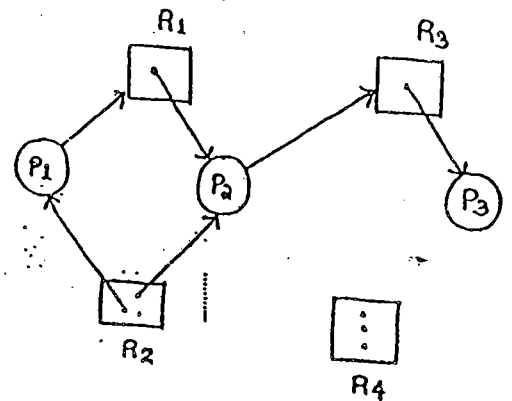
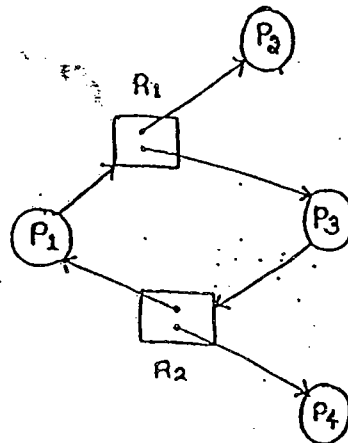
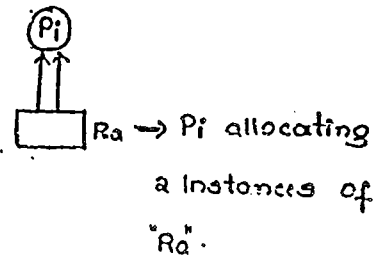
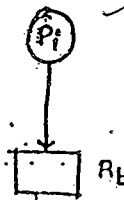
Rb []
registers
(multiple copies)

Edges:

Assign edge



Request edge



P_1 gets blocked, since it requests ' R_1 ', which in turn R_1 gets accessed by ' P_2 '. Similarly, P_2 also gets blocked.

* If RAG has single instance and multiple instance.

* In multiple instances, there is a chance of occurring deadlock, because they form cycles (some times).

* Single instances may also cause deadlock, if the processes accessed in the cycle form.

if the graph contain no cycle then no process in the system is in deadlock. if the graph contain a cycle then a deadlock may exist.

method for handling deadlock: three way

① use a protocol to prevent or avoid deadlock ensuring that the system will never arise in the deadlock

Strategies for deadlock handling :-

② we can allow a system to enter a deadlock state detect it and recover.

③ we can ignore the problem altogether and routing. Type-1 the deadlock Type-a new occurrence in system Type-3

This solution

is used by most operating system

* Deadlock prevention
* Deadlocks avoidance
(Banker's algorithm)
No deadlock happens/ occurs.

* Deadlock
Detection and
Deadlock
Recovery
Deadlock happens

* Deadlock Ignorance
(Ostrich Algorithm)
↓
(Disasters)

Deadlocks prevention :-

* Try to dissatisfy one / more of the necessary conditions.

(1) mutual exclusion :- Not required for sharable resource
must hold for non sharable resource

(2) ! (Hold & wait):

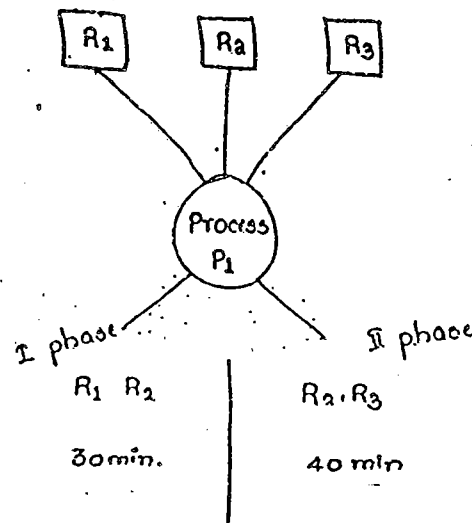
Either "hold" (or) "wait" (but not both)

* Requests & de-allocation of all the resources done between the process commencement.

Starvation Inefficiency

mutual Exclusion :-

* Holding only one process within the critical section.



Consider,

- * A process ' p_i ' which needs to access the resources ' R_1 ', ' R_2 ' and ' R_3 '.

If a resource ' r_1 ' and ' r_2 ' are accessed and another time, again ' R_2 ' needs to be accessed. So, the following situation holds :-

Hold & wait :-

- * Process ' p_i ' in first phase, requires R_1 and R_2 resources and the time span allotted to phase-I is 30min to complete.

* After the completion of phase-I, process ' p_i ' releases the resource ' R_1 ' but holds the resource ' R_2 ' (because it again requires the same resource at phase-II also), and requests for the resource ' R_3 '. This situation is considered as "Hold & wait".

- * In order to avoid this problem, we have two types of solutions :-

1st Approach :-

- * Process, in phase-I, requests all the resources whatever it needs.

Eg:- $P_1(R_1, R_2, R_3)$

- * They doesn't wait for all the resources to be accessed.

2nd Approach :-

- * Release all the resources before making a new request.

Phase - I $\Rightarrow P_1(R_1, R_2)$

Phase - II $\Rightarrow P_1(R_2, R_3)$

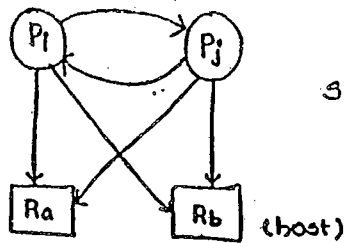
- * After the completion of phase-I, release all the resources even though, it needs the same resource at phase-II.

- * Then, again in phase-II, the resources are requested newly (if already used in phase-I).

(c) No pre-emption :-

- * Allow the processes to get preempted.

Self forcefully



Self release $\Rightarrow$ It requests the resources that are not available, so that process releases other resources that are already available

Consider, P_1 , which already accessed R_a , it requests for R_b , but R_b is not available, at that time. So, P_1 releases R_a resource (in the sense, it may be useful for others) after completion of other process, it will execute.

22/07/2010

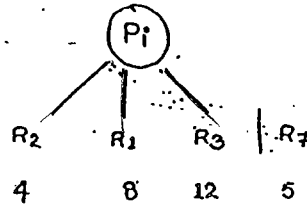
Thursday

Prevention (restrictive):

(c) circular wait:

"Total order relation on all process for requesting of resources."

Incl dec.

 $f(R) \rightarrow 'I'$ $R_1 - 8$ $R_2 - 4$ $R_3 - 12$ $R_4 - 18$ $R_5 - 25$ $R_6 - 9$ $R_7 - 5$  P_i requests in any order
(i.e., either in incl dec).If no order is followed,
any sequence is followed, P_i
must release all resources.Starvation occurs when there
is a release on "

Avoidance :-

(Banker's Algorithm):

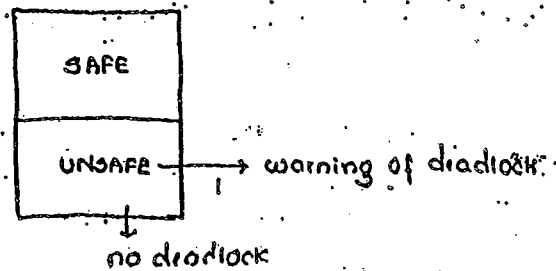
* Safety Algorithm

* Resource- Request.

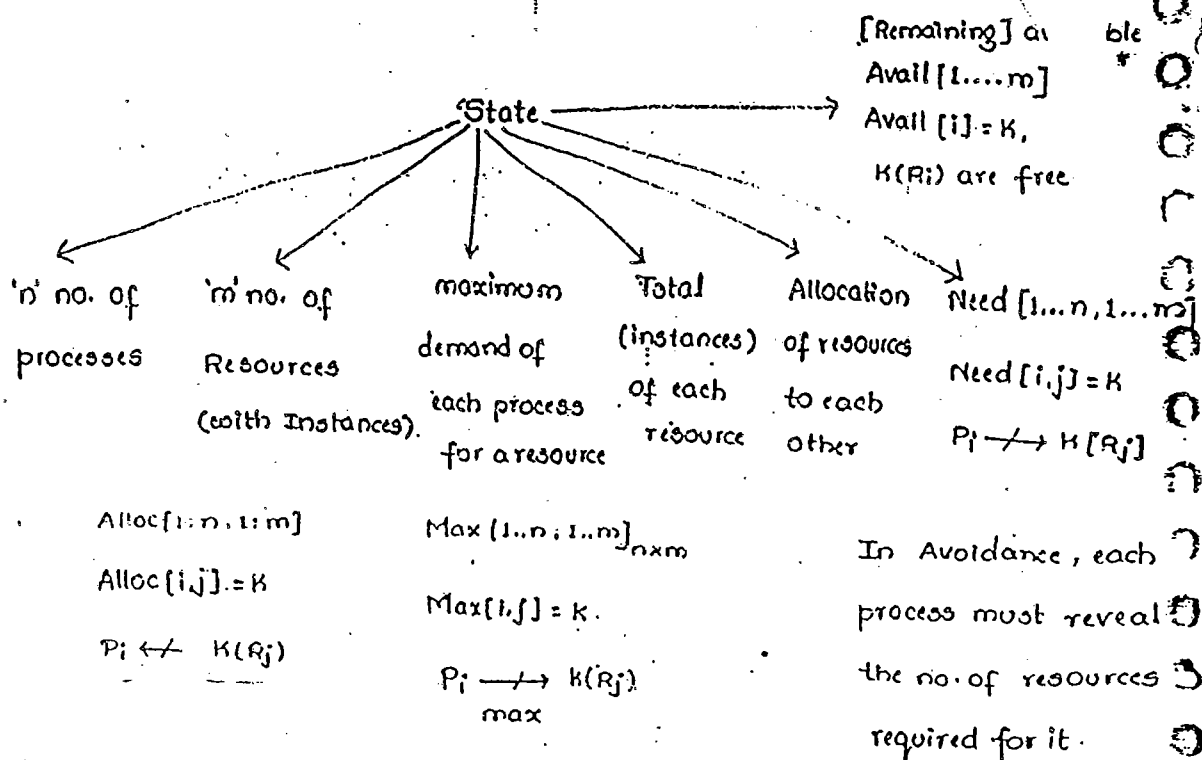
Objective :-

* Obtain a system always in a safe
mode.

* It is an "A priori Info" algorithm, It is based on state of the system



⇒ Banker algo is less efficient than the resource allocation graph



Eg :- n = 5 (P₁ to P₅).

m = 1, R = 28 (Total)

Need = max - Alloc.

n	R (max)	R (Alloc)	(Need) R	(Avail) R
P ₁	10	5	5	3
P ₂	20	10	10	4
P ₃	8	1	7	6
P ₄	8	4	4	13
P ₅	12	5	7	18
				28

T₀ : < P₃, P₄, P₅, P₁, P₂ >

Safe-sequence

Avail = current avail
+ allocation.

* System is said to be SAFE, iff the needs of all the processes satisfied with the available resources in some or the other order (is called SAFE sequence) otherwise, the system is said to be

But not warning

really a deadlock to indicate

- ① mxn2 operation to check state is safe or not
- ② detecting a cycle in the RRG require n² operation

$n=5, m=3 \quad (A,B,C) = (10,5,7)$

P	max			Alloc			need			Avail			Alloc (8+3+2) π
	A	B	C	A	B	C	A	B	C	A	B	C	
P ₀	7	5	3	0	1	0	7	4	3	3	3	2	$10 - 7 = 3$
P ₁	3	2	2	2	0	0	1	2	2	5	3	2	$5 - 2 = 3$
P ₂	9	0	2	3	0	2	6	0	0	7	4	3	$7 - 6 = 1$
P ₃	2	2	2	2	1	1	0	1	1	7	4	5	
P ₄	4	3	3	0	0	2	4	3	1	7	5	5	
										<u>10 5 7</u>			

$T_0 : \langle P_1, P_3, P_4, P_0, P_2 \rangle \Rightarrow \text{safe.}$

Initial availability = (3,3,2), 0's have enough resources to grant to P₁, but it first calculates any problem if user gives this resource. It runs the "safety algorithm" in this situation.

T_1 : Resource Request algorithm :-

"Request by the resources are granted or not" is being identified by this algorithm.

$T_1 : P_1 : \longrightarrow (1,0,2) \text{ (Req 1)} \longrightarrow \langle P_1, P_3, P_4, P_0, P_2 \rangle \Rightarrow \text{safe.}$

$T_1 : P_4 : \longrightarrow (3,3,0) \text{ (Req 4)} \longrightarrow \text{Blocked.}$

It is blocked because it doesn't satisfy the Availability.

$T_1 : \langle 0,0,0 \rangle \rightarrow P_0$

n	max	Alloc	need	Avail
	A B C	A B C	A B C	A B C
P ₀	7 6 3	0 1 0	7 4 3	2 3 0
P ₁	3 2 2	3 0 2	0 2 0	
P ₂	9 0 2	3 0 2	6 0 0	
P ₃	2 2 2	2 1 1	0 1 1	
P ₄	4 3 3	0 0 2	4 3 1	

Resource Request Algorithm:-

1. Request $\leq$ Available

2. Request $\leq$ need

3. (Assume request is satisfied)

Available = Available - request

Allocation = Allocation + request

Need = Need - Request

4. If result is safe, allocate resource.

P₁ $\rightarrow \langle 1,0,2 \rangle$ request

P₄ $\rightarrow \langle 3,3,0 \rangle$

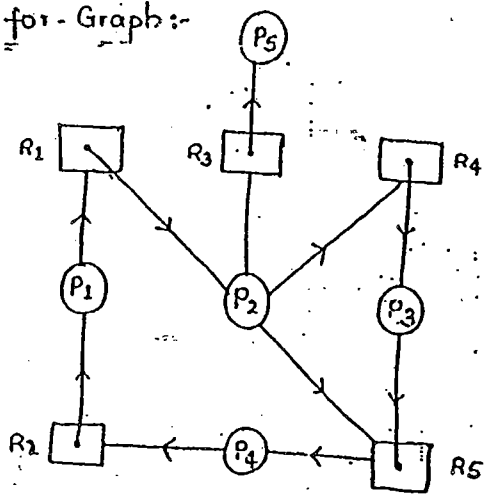
P₀ $\rightarrow \langle 0,2,0 \rangle$

III Deadlock Detection & Recovery :

occurs when system performance is low.
If many blocked processes are present.

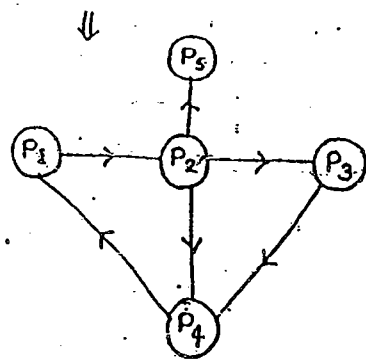
i) Single Instance of a resource :-

wait-for-Graph :-



Among 5 processes, 4 are blocked. So, we apply deadlock detection algorithm.

Cycle Detection
Algorithm
is used



$P_1 - P_2 - P_3 - P_4 - P_1$
↓
cycle $\Rightarrow$ Deadlock

II multi-Instance of a resource :-

$$n=5, m=3, (A, B, C) = (7, 2, 6)$$

⇒ request is again processed after allocation

T ₀	Alloc			Req	Avail		
	A	B	C		A	B	C
P ₀	0	1	0	0	0	0	0
P ₁	2	0	0	2	0	1	0
P ₂	3	0	3	0	3	1	3
P ₃	2	1	1	1	5	2	4
P ₄	0	0	2	0	5	2	6

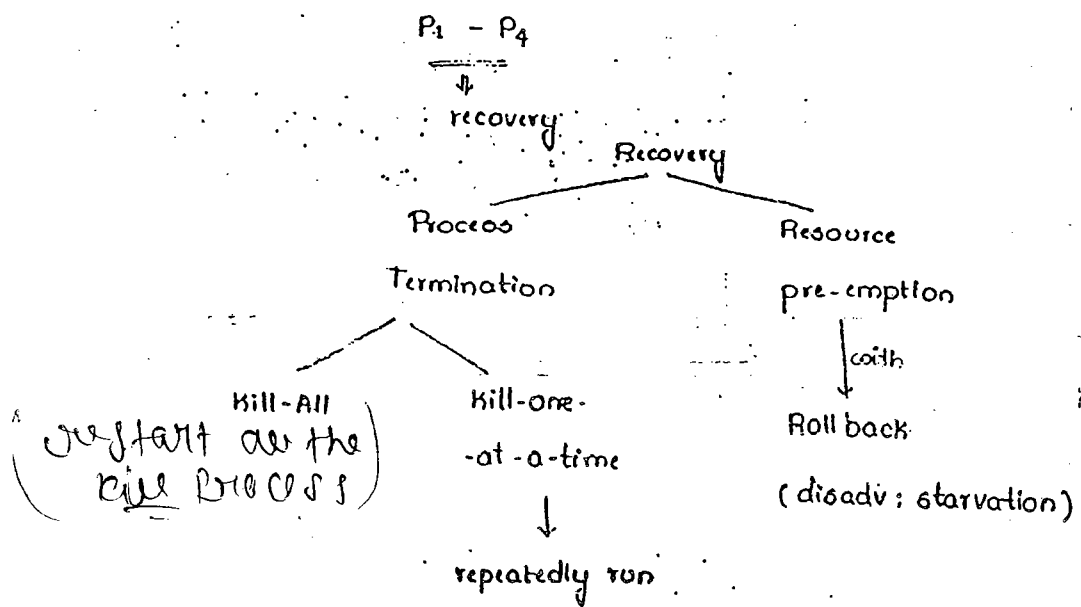
$7-7=0$
 $2-2=0$
 $6-6=0$

Initially, resource available is '000' but P₂ requests 001, It can't satisfy 0.3.
 P₁ → P₄ ⇒ Input's to recovery module.

To check safe mode : $\langle P_0, P_2, P_3, P_4, P_1 \rangle$

T₅ : P₂ → $\langle 0, 0, 1 \rangle$

$\langle P_0, \dots \rangle$ X Deadlock



Kill-All :-

Adv :-

* No need for detection again.

Disadvantage:-

* Again, it must be started from beginning

* To overcome, this, we have "killing gracefully".

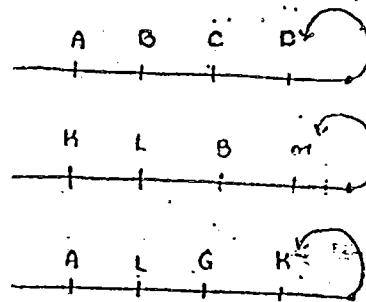
One-at-a-time:-

Adv:-

* Save the other processes to resume. * Need for detection algorithm again

Disadv:-

Resource pre-emption:-



Roll-back.

Repeatedly, a process is detected to have deadlock and re-start then the starvation occurs.

Deadlock handling strategies are not used on desktop systems and "Deadlock Ignorance" is considered:-

* Robustness.

not important tasks

* follow obitric algorithm (in missile, real time, Industries o.s)

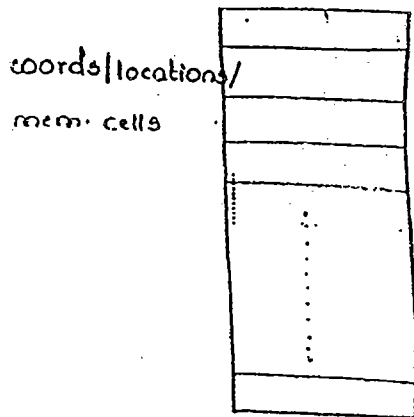
The strategies are used in real time systems where there is no single bit errors.

↓
follow Deadlock prevention

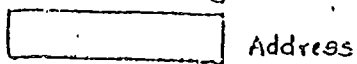
Memory Management

Abstract view of memory

(1-D Array of words)



Size/width/length (bits)

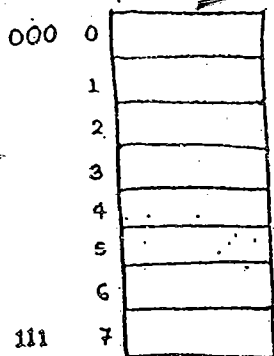


(Fixed length
binary bits)

Word

- N = Total no. of words in memory (Capacity of memory)
- n = Address (in bits)
- m = width/size (bits) of words

$N = 8$
 $n = 3 \text{ bits}$
 $N = 2^3 \text{ words}$



$N = 8 \text{ words}$

$n = 3 \text{ bits}$

$N = 64 \text{ words} = 2^6 = 2^6$

$n = 6 \text{ bits} (2^6)$

$N = 512 \text{ Mwords}$

$n = 9 + 20$

$= 29 \text{ bits}$

$N = 4 \text{ GW}$

$n = 2^2 + 30$

$= 32 \text{ bits} \Rightarrow 2^{32} = 4 \text{ GW}$

$N = 2 \text{ GB}$

$n = 30 + 1 = 31 \text{ bits}$

$N = 500 \text{ GB}$

$n = 30 + 9$

$= 39 \text{ bits}$

$2^{10} = 1 \text{ KB}$
 $2^{20} = 1 \text{ MB}$
 $2^{30} = 1 \text{ GB}$
 $2^{40} = 1 \text{ TB}$

$2^{30} = 1 \text{ GB}$
 $2^{31} = 2 \text{ GB}$
 $2^{32} = 4 \text{ GB}$

$$2^6 = 32 \text{ K}$$

$$2^6 = 64$$

$$2^7 = 128$$

$$2^8 = 256$$

$$2^9 = 512$$

$$2^{10} = 1024 \approx 10^3 = 1 \text{ K}$$

$$2^{20} = 1024 * 1024 \approx 10^6 = 1 \text{ M}$$

$$2^{30} \approx 10^9 = 1 \text{ G}$$

$$2^{40} \approx 10^{12} = 1 \text{ T}$$

$$N = 2^n$$

$$512 \Rightarrow 2^9$$

$$n = \lceil \log_2 N \rceil$$

$$p = 2^n \text{ words}$$

$$b = \lceil \log_2 N \rceil$$

memory specification.

word Byte Bit

1

$$n = 23 \text{ bits.}$$

$$m = 16 \text{ bits (1 word = 2 bytes)} \approx 16 \text{ bits}$$

	Capacity	Address
words	8 MW	$n = 23 \text{ bits}$
Bytes	$8 \text{ M} \times 28$ $= 16 \text{ MB}$	$n = 24 \text{ bits}$
Bits	$16 \text{ M} \times 8 \text{ bits}$ $= 128 \text{ Mbits}$	$n = 27 \text{ bits}$

$$8 = 2^3 \text{ MW}$$

↓

$$3 + 20 = 23$$

$$16 = 2^4 \text{ MB}$$

$$= 4 + 20 = 24$$

$$128 = 2^7 \text{ Mbits}$$

$$7 + 20 = 27$$

2

$$N = 256 \text{ MW}$$

$$m = 32 \text{ bits (1 word = 4 bytes)} \Rightarrow 256 \text{ M} \times 4 \text{ B} \Rightarrow 1 \text{ G.}$$

	Capacity	Address
words	256 MW	$n = 28 \text{ bits}$
Bytes	$256 \text{ M} \times 4 \text{ B}$ $= 1 \text{ G.}$	$n = 30 \text{ bits}$
bits	8 G bits	$n = 33 \text{ bits}$

$$32 \text{ bits}$$

$$2^5$$



$N = 256 \text{ Gbits}$

$m = 64 \text{ bits}$

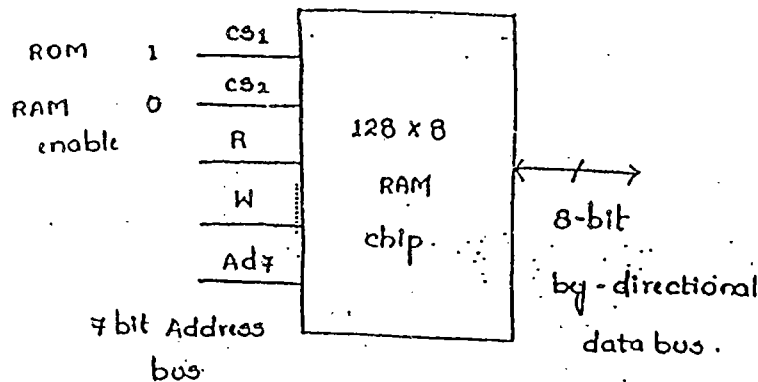
(1 coord = 8 bytes) $\Rightarrow 256 \text{ G} \times 8 \text{ B}$

$$2^8 \cdot 2^{30} \times 2^8 \times 2^3$$

$$\frac{2^{38}}{2^6} = 2^{32} = 4 \text{ GW}$$

	capacity	Address
coords	4GW	32 bits
Bytes	$4 \text{ G} \times 8 \text{ B}$ $= 32 \text{ GB}$	36 bits
Bits	256 Gbits	38 bits

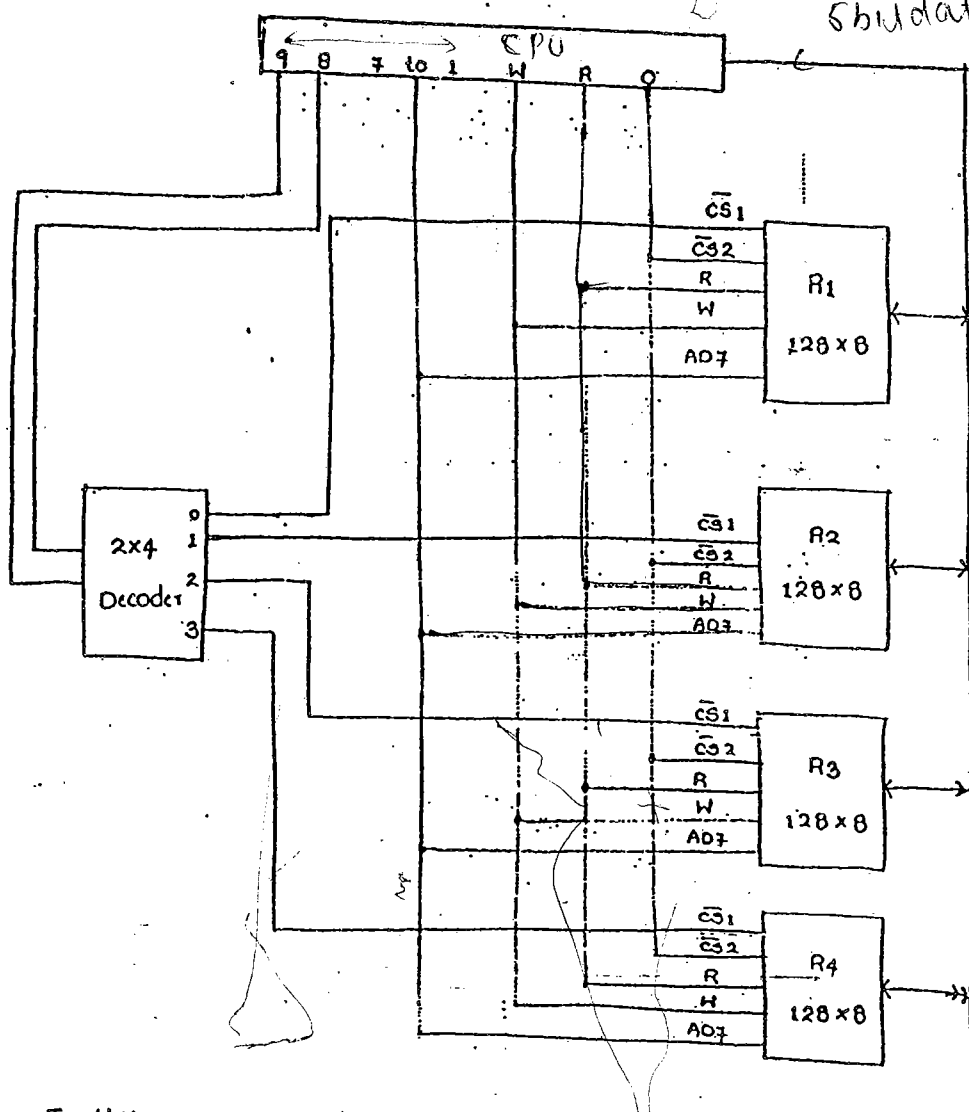
RAM chip organisation :-



$$\text{No of chips} = \frac{\text{memory desired}}{\text{chip size}}$$

$$\frac{512 \text{ MB}}{128 \text{ MB}} = \frac{2^9}{2^7} = 2^2 = 4$$

5 bit databus



I Using 128 x 8 RAM

16 KB = ?

↓ no. of words increasing

(i) chips = 128

(ii) n = 14 bits

(iii) Decoder = 4 x 128

$$\frac{16 \text{ KB}}{128} = \frac{2^{14}}{2^7} = 2^7 = 128$$

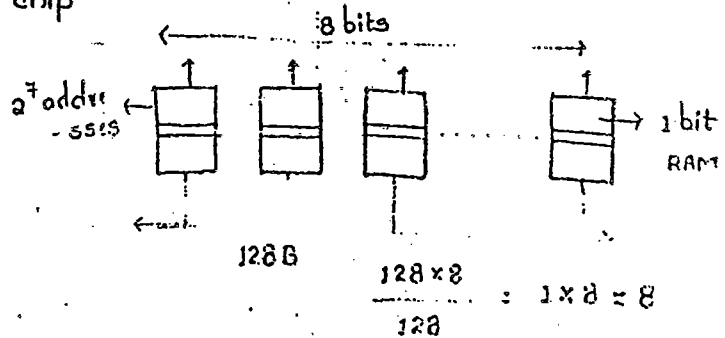
II Using 128 x 1 RAM chip

128 bytes

(i) chips = 8

(ii) $n = 7$ bits

It contains each word $\Rightarrow$ 1 bit



Since, memory and data size are same, there is no need of decoder.

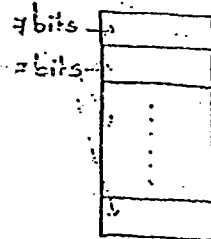
II Using 128 x 1 RAM chip

16 KB

(i) chips = 1024

(ii) $n = 14$ bits

(iii) Decoder = 7×128



128 x 8

$$\frac{16 \text{ KB}}{128 \times 1}$$

$$= \frac{2^{14}}{2^7} = 2^7$$

$$= 128 \times 8 \text{ bits}$$

128 x 1 RAM

$$128 \times 8 \text{ words} = 1024 \text{ bits}$$

$$\frac{16 \text{ KB}}{128 \times 1} = \frac{2^4 \times 2^{10}}{2^7} = 2^7 = 128 \times 8$$

$$= 1024$$

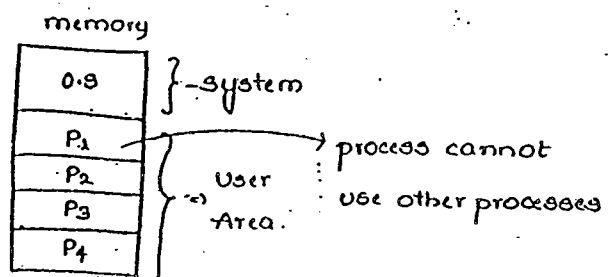
Memory Manager

Functions :

- * Allocation
- * Protection
- * Free space management
- * Deallocation

Goals :

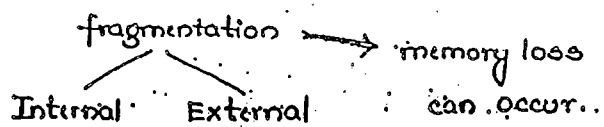
- * Efficient utilization of space.
- * Run larger program in smaller memory



Goals :-

(1) Space utilization:-

Keeping fragmentation at each level, so that there is a chance of effective utilization of space.



(2) Run larger program in smaller memory

Program = 100 KB (Disk)



using virtual memory/overlays.

Eg.:- Run the data within, pendrive (eg:- 60KB)

8.10.11

Memory management

Techniques

Contiguous (CS)

- * centralized

program, as a unit is completely stored

- * Overlays
- * partitioning

Non-contiguous (NCA)

- * Decentralised

program portions are distributed in memory.

- * paging
- * segmentation
- * Segmentation-paging
- * virtual memory.



24/07/2010

Saturday

(1) Overlays (larger programs run in smaller space) :-

2-pass Assembler.

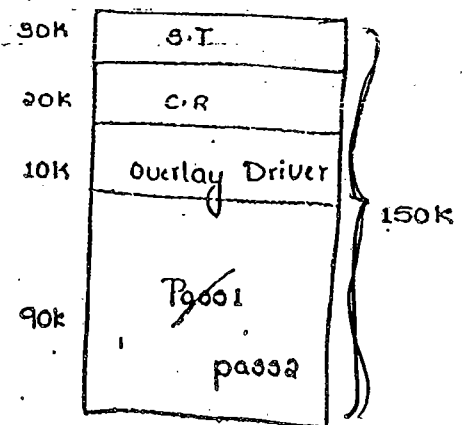
pass 1 : size - 70 KB

pass 2 : size - 80 KB

Symbol Table.

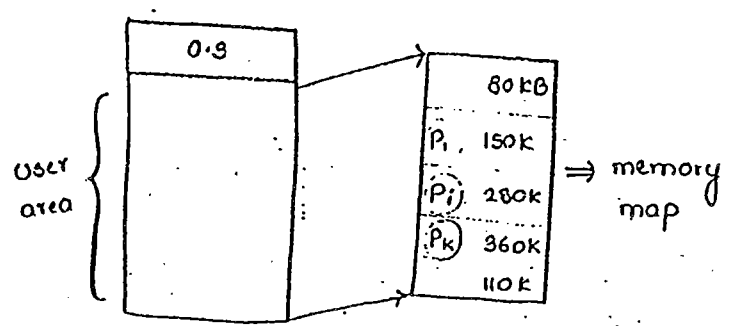
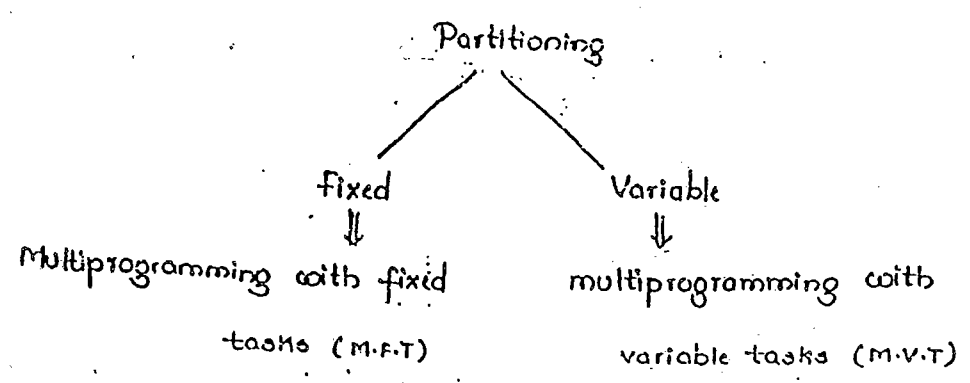
for construction of passes : 30 KB

C-R (Common Routines) : 20 KB
200 KB

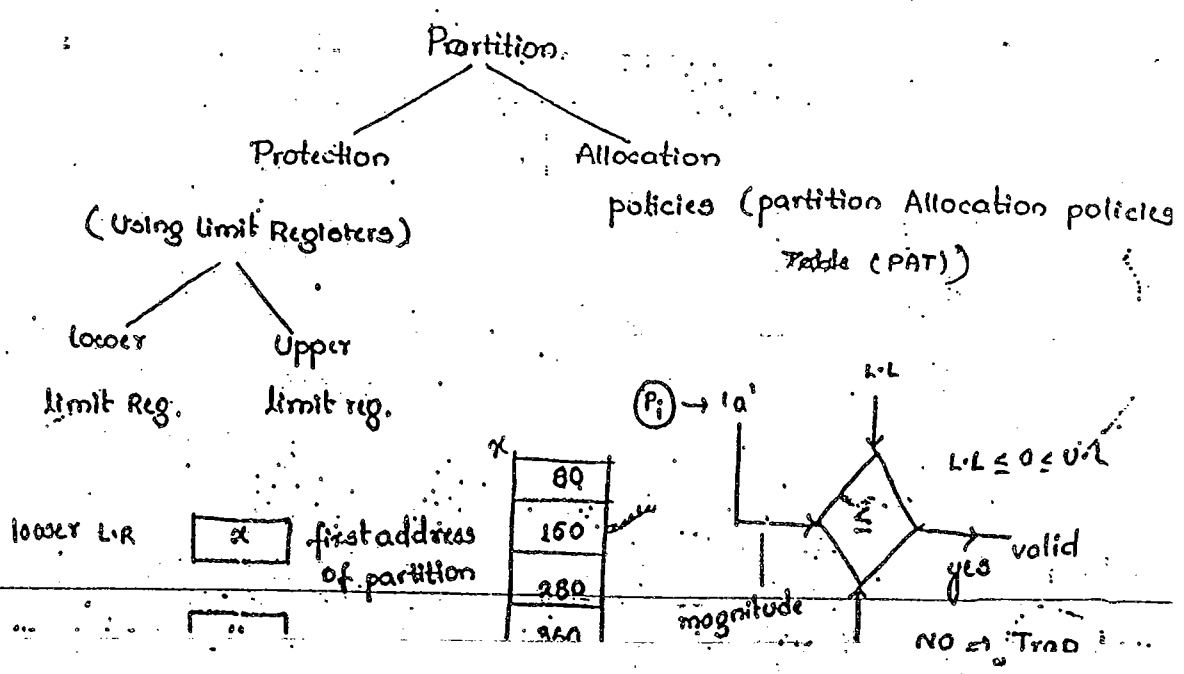


loading programs → overlay driver

(2) partitioning :-



fixed (M.F.T) ⇒ no. of partitions are fixed and of different sizes since 1 partition = 1 process



Eg: $P_{new} = 100 \text{ KB}$

80
150
280
360
50

Partition Allocation policies :-

(i) First fit (F.F) :-

* Allocate the process in the first free begin of the partition.

To include $P_{new} = 100 \text{ KB}$, the partition of $P = 150 \text{ KB}$ is selected as it is the first partition of required size.

(i) Best fit (B.F) :-

* Smallest begin of free partition.

* Best available fit

* Best fit only.

Best available fit :-

Among available partitions, select the smallest best available.

Best fit only :-

It selects for best fit only, if it is not available, it waits for that partition, and after available it takes.

(i) Next fit (N.F) :-

Same as first fit, but it doesn't start from first partition, but it starts from the place where the last partition ended up (i.e., last process's partition ended point).

Advantage :-

* Searching becomes faster than first fit.

(iv) coolest fit (c.o.f):

largest free partition $\Rightarrow$ selecting the largest available free partition and get placed in that partition.

Performance of MFT :-

(1) fragmentation :-

* Internal

* External $\Rightarrow$ when unable to accommodate a process of size even though we have sufficient size of memory.

1 partition $\Rightarrow$ 1 process

(contiguous allocation)

Eg:- Process = 180 KB

80 KB
150 KB
* 280 K
* 360 K
110 KB

$80 + 150 + 110$

$= 340 \text{ KB}$

If we have a new process of 180 KB to place into memory, but there is no certain partition to get fit into this process even though it has total of 340 KB free. So, it is called as External frag.

(2) Maximum process size (or) maximum partition size $\Rightarrow$ no flexibility.

*** (3) Degree of multiprogramming:-

Restricted to no. of partitions (can't be $> n$).

$\downarrow$
no flexibility.

(4) Best fit is superior to all.

In order to overcome all these problems, we have m.v.t.

m.v.t (variable partitions):-

⇒ Dynamic
approach

⇒ No user area for
partitioning

P ₁	80K
P ₂	110K
P ₃	75K
P ₄	180K
P ₅	260K
P ₆	320K
300K	

PCM (Partition Control memory)

used for storing the control infor-
-mation of particular partition.

* No use of limit registers in m.v.t for protection.

* Limit value
Size
Status
L.L, U.L } stored in PCM.

Eg:- If a new process of 70KB is needed
to fit into memory, then if we take
first fit, the 80KB partition is divided
as 70KB for the new process & rest
of 10KB. So, we overcome the frag-
-mentation problem here.

Process (new) = 70KB

Performance of mvt :-

(1) fragmentation:-

- * External
- * No Internal.

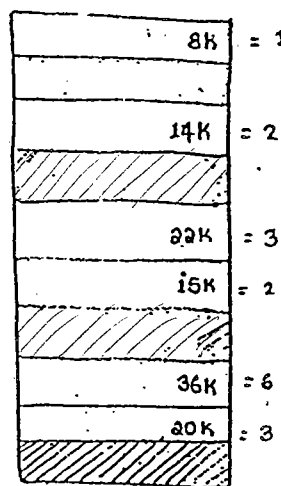
(2) Max. process size ⇒ flexible
no max. part. size.

400KB		PCM
	80KB	
free	10KB	
	P ₂ 110K	
	75K	
	P ₄ 180K	
	260K	
	P ₆ 320K	
	300K (free)	

(8) Degree of multiprogramming :-

- * No restriction to no. of partitions.
- * flexible

(4) worst fit performs better, and best fit performs worst.



4) 6K ?

First fit = 8K

Best fit = OK

next fit = 36K

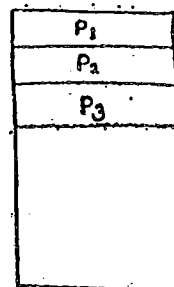
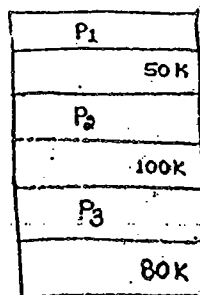
woordt fit = 36k

(ii) How many successive requests
of size $6K$ can be satisfied.

Ans: 17.

External fragmentation:-

(i) Compaction (dynamic relocation of processes):-



2. All the processes at different memory locations are compacted to one side and the free memory is now available.

- * process should be dynamically relocatable and can be processed / continued.

If cannot be run on processed, then no compaction is followed.

Disadvantages:-

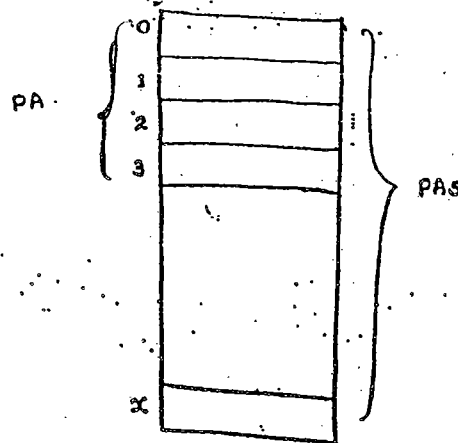
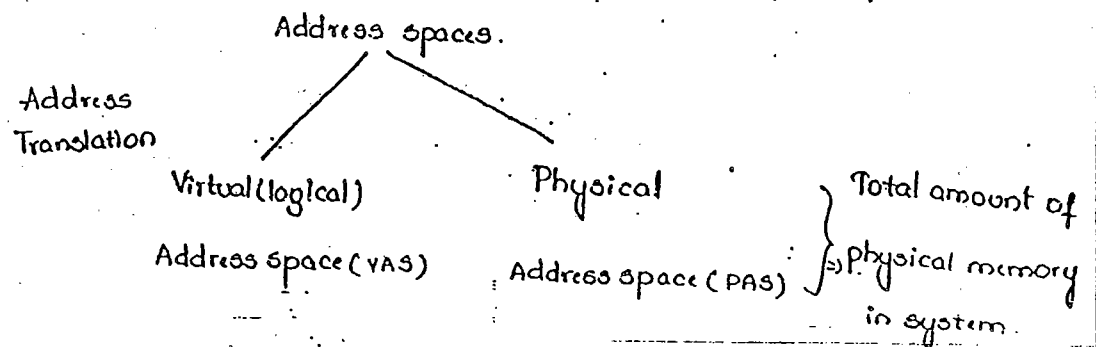
- * longer time (Since everytime relocation is done).
- * If done frequently, time is consumed.

(3) Non-contiguous Allocation:-

- * To overcome external fragmentation, we use non-contiguous allocation policies.

Address space:-

- * space of words / series of words (set)
- * A set of locations of memory is address space.



$$PAS \times PA \Rightarrow PAS = 2^{PA} \Rightarrow PA = \log_2 [PAS]$$

Logical Address Space :-

* Every process within the CPU has an assumption that it can access the whole memory available.

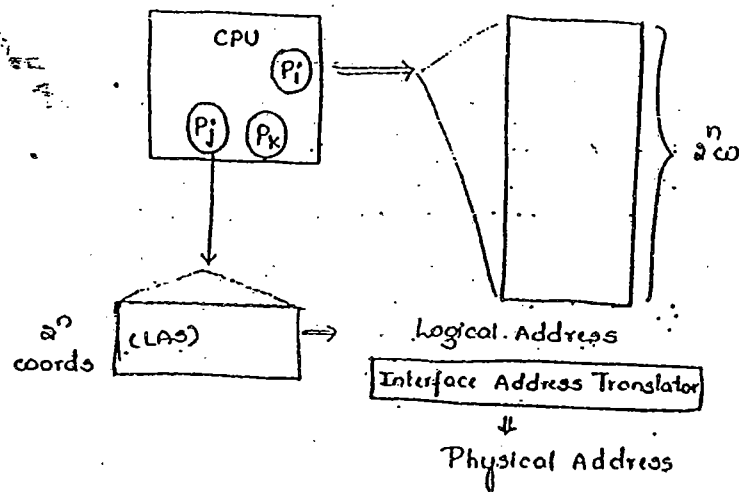
* LAS must be translated to physical for instruction/data fetch.

If LA = 13 bits (2^{13} K)

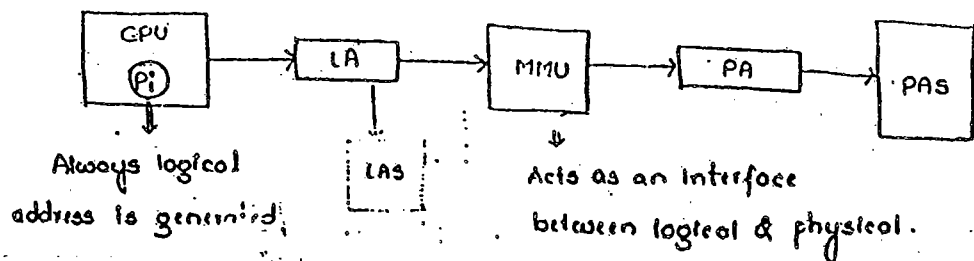
$$LA = \log_2(LAS)$$

Then, LAS = 8 KB

$$LAS = 2^{LA}$$



Architecture of non-contiguous Allocation policies :-

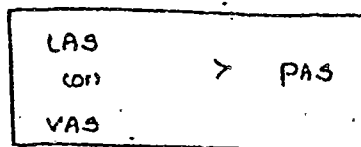


(1) Organisation of LAS

(2) Organisation of PAS

(3) Organisation of MMU (mfr mgmt unit)

(4) Translation process



(practically)

Because, it must have to support the concept of larger program storage in shorter memory.

Theoretically, all of them have different relation.

Paging:-

Simple paging:-

LAS = 8 KB, PAS = 4 KB, LA = 13 bits, PA = 12 bits.

(1) organisation of LAS:-

(i) LAS is divided into equal size pages

(ii) Page size is a power of 2.

(iii) No. of pages (N) = $\frac{\text{LAS}}{\text{Page size}}$

(iv) page no. (p) = $\lceil \log_2 N \rceil$

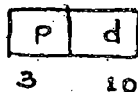
$$N = 2^p$$

(v) page offset (d bits) = $\lceil \log_2 P_s \rceil$

$$P_s = 2^d$$

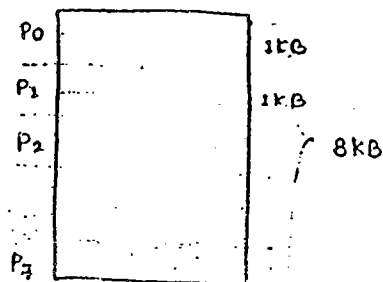
(vi) Format of logical address:

LA = 13 bits



words

pages.



$N = 2^p$
 $\log_2 N = \log_2 2^p$
 $\log_2 N = p \log_2 2$

(a) Organisation of PAS:

(i) PAS is divided into equal size frames (page frames)

(ii) Frame size = page size

Frame size is same as page size

(iii) No. of frames (m) = $\frac{PAS}{PS}$

(iv) Frame no. ('f' bits) = $\lceil \log_2 M \rceil$

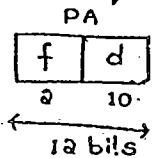
$$N = 2^f$$

(v) Frame offset = page offset

(vi) Frame holds the page, therefore, frame offset is equal to page offset.

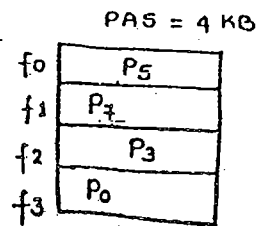
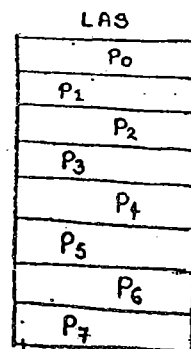
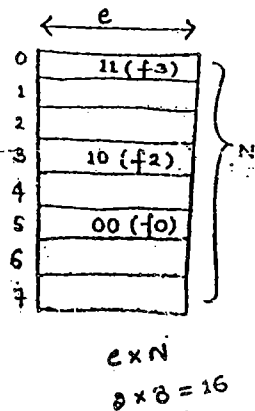
PAS = 5 KB

fA	1KB
FA	1KB
FA	1KB
FA	1KB
FA	1KB



(a) Organisation of MMU (page table):

page map Table $\Rightarrow$ Associated with processes.



* No. of entries in P.T (P.T size) = No. of pages in LAS.

* PT entries contain frame numbers.

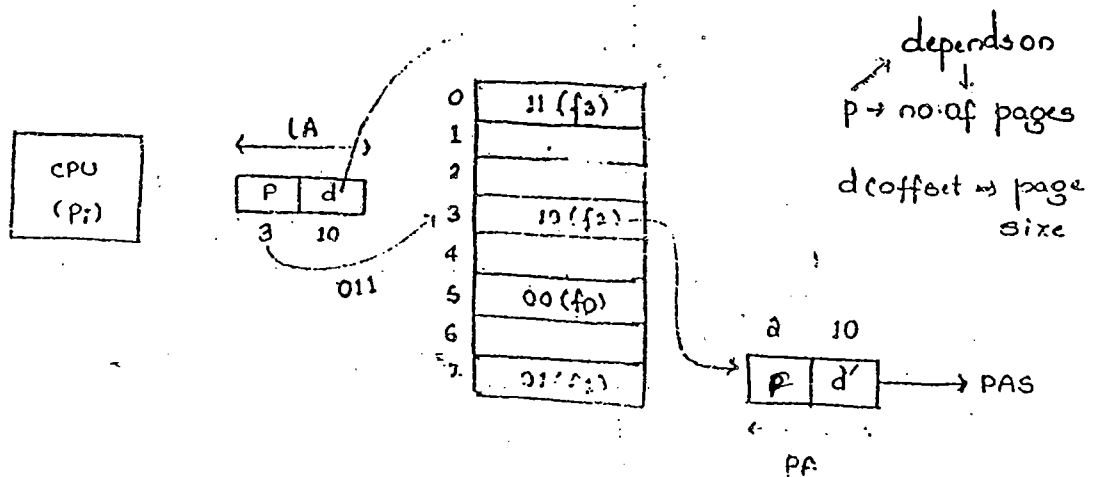
* Associated with processes (Each process has its own page table)

* page tables are stored in main memory.

* P.T size (in bytes) = $N * e$ bytes.

$e \Rightarrow$ page table entry size in bytes

Address Translation:



eg:-

LA = 32 MB, PAS = 256 KB

$N = ?$, $m = ?$, $p = ?$, $f = ?$, $d = ?$

page size = 4 KB

P.T size = ?

sol:-

$$N = \text{no. of pages} = \frac{32 \text{ MB}}{4 \text{ KB}} = \frac{2^{25}}{2^{12}} = 2^{13} = 8 \text{ K}$$

$$M = \frac{256 \text{ KB}}{4 \text{ KB}} = \frac{2^{18}}{2^{12}} = 2^6 = 64 \text{ K}$$

$p = \text{page no. (depends on no. of pages)} = 13 \text{ bits}$ ($N = 8 \text{ K} = 2^3 + 10 = 13$)

$f = \text{frame no. (depends on } m) = 6 \text{ bits}$

$d(\text{offset}) \text{ depends on page size} = 12 \text{ bits}$ ($4 \text{ KB} = 2^2 + 10 = 12$)

P.T size = no. of pages = 8K

If $e = 4 \text{ B}$ then P.T size = $e * N$

$= 8 \text{ K} * 4 \text{ B}$

$= 32 \text{ KB}$

Eg:- 2k pages & 512 frames calculate

If $PAS = 4MB$, $e = 4B$, $N, M, P, f, LA, PA, d, P.T.S$

Sol:-

$$N = 2k$$

$$M = 512$$

$$P = 11 \text{ bits } (2k \Rightarrow 2^{11} = 2048)$$

$$f = 9 \text{ bits } (512 \text{ frames } \Rightarrow 2^9)$$

$$LA = 24 \text{ bits } = (16MB)$$

$$PA = 22 \text{ bits}$$

$$d = 13 \text{ bits } (PA = 22, f = 9)$$

$$d = PA - f = 13$$

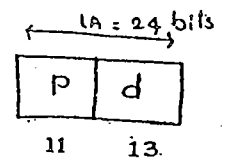
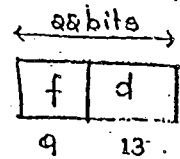
$$P.T.S (Bytes) = N * e$$

$$= 2k * 4B = 8KB$$

$$PAS = 4MB$$

$$PA = (2^2 + 20)$$

$$= 22 \text{ bits}$$



Performance of paging:-

* main memory access time (mmat) = 'm' ($\mu s/nos$)

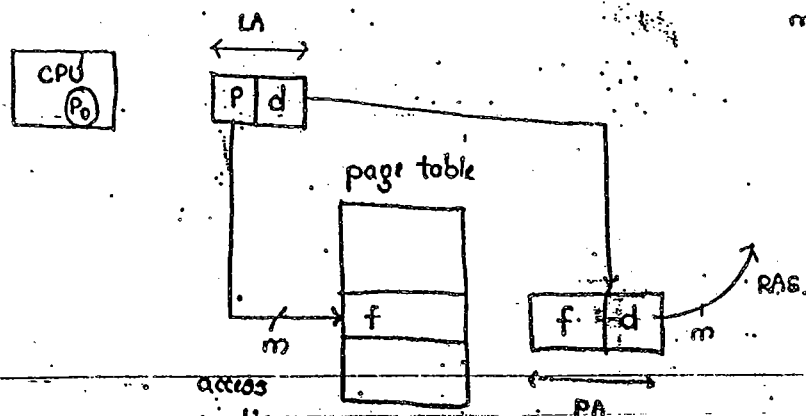
* Effective memory Access time (EMAT)



Decrease/Reduce of EMAT from am d bring closer to 'm'.

Access Time:

Amount of Time to read (or) write to memory.



(ii) paging with TLB (Translation look-aside Buffer) :-

If generated-LA is not found within the cache (or TLB), it is called "TLB miss". If found, it is called "TLB hit".

T.L.B Access time = c

where $[c \ll m]$

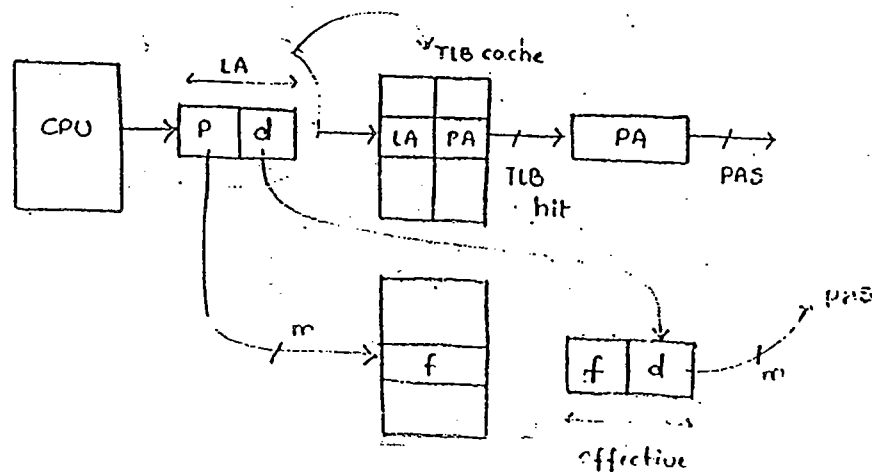
TLB is a cache contains

list of physical & logical addresses

If generated LA is present, then it is TLB hit, or it goes to page table & generates physical address

$$\text{T.L.B page hit ratio } (\alpha) = \frac{\text{No. of hits}}{\text{Total references}}$$

$$\text{T.L.B miss ratio } (\gamma) = 1 - \alpha$$



$$EMAT_{(SP+TLB)} = \underbrace{\alpha(c+m)}_{\text{Success}} + \underbrace{(1-\alpha)(c+am)}_{\text{failure}}$$

eg: (i) $m = 100 \text{ ns}$ $c = 20 \text{ ns}$ $\alpha = 90\%$

$$EMAT = 0.90(20+100) + 0.1(20+200) \\ = 9(120) + 22 = 1100 \text{ ns}$$

(ii) $\alpha = 10\%$

$$EMAT = 0.10(20+100) + 0.9(20+200) \\ = 12 + 9(22) = 210 \text{ ns}$$

(additional time)
 Effective memory overhead : = m.
 (Simple paging)

SP - Simple
 paging

$$EMOH (S.P + TLB) = x(c) + (1-x)(c+m)$$

* By using page table, every time, we require "am" memory access time 'm' is overhead for minimising. This introduce TLB that access time is 'c' which is less than 'm' overhead

* In hit rate, Translation require "c+m", if miss occurs it takes c+m+r
 c+am

** Multi-level paging :-

$$\text{Lr } LA/VA = 32 \text{ bits, } P.S = 4KB$$

$$PT \text{ size} = 1m, B = 4MB$$

$$N = \frac{2^{32}}{2^{12}} = 2^{20} = 1M$$

$$e = 4B$$

Reduction of page
 table

Increasing

Multi-level.

page size



Drawback :-

Increase Internal fragmentation.
 (wastage of space)

Large :- P.S = 1KB

= 2 pages

Program (1024)

= 1024 ≈ 1 page

Small :- P.S = 4B

= 512

512

= 0B

Larger page tables
 1m are not desired.

Optimal page size = ?

Internal fragmentation
P.T overhead } $\Rightarrow$ must be minimum

* $LA3/VAS = 's'$ coords.

* page Table entry = ' e ' bytes.

* page size ' p ' = ?

$$\text{P.T overhead (bytes)} = \left(\frac{s}{p} \right) * e$$

$$\text{Internal fragmentation} = \frac{p}{2}$$

$$\text{Total overhead} = \left(\frac{p}{2} + \frac{s}{p} * e \right)$$

$$\text{To minimize/optimize} \Rightarrow \frac{d}{dp} = \left(\frac{p}{2} + \frac{s}{p} * e \right)$$

$$(\text{min.}) = 0 = \left(\frac{1}{2} - \frac{1}{p^2} s * e \right)$$

$$p^2 = ase$$

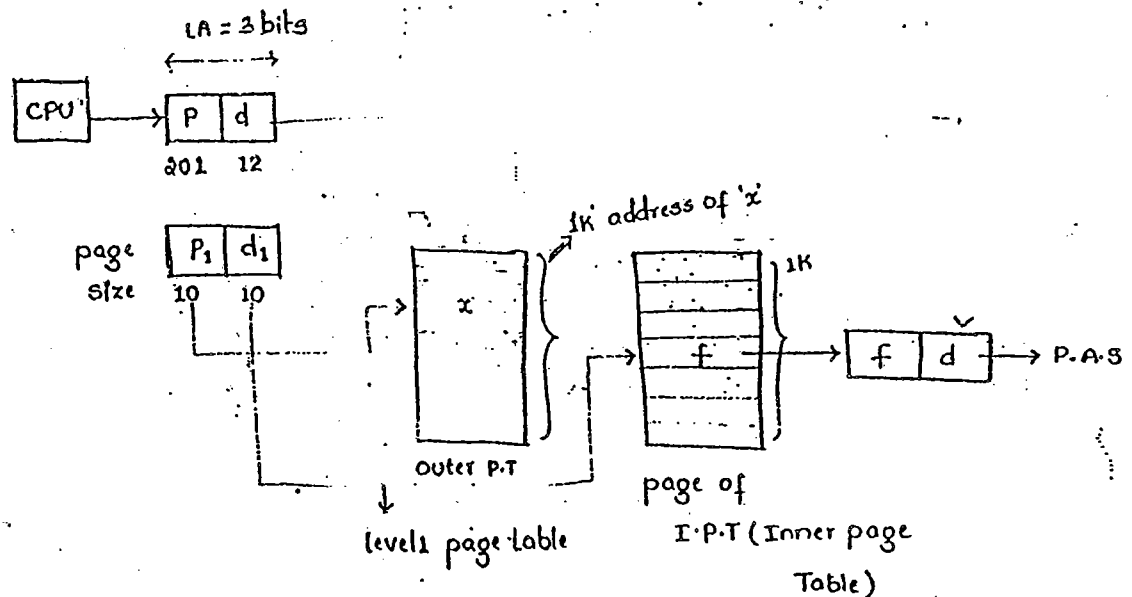
$$p = \sqrt{ase}$$

28/07/2010

wednesday

Paging :-

* Dividing any address space into equal size units (pages)



Total virtual memory = 2^{32}

Each page size = 4KB

Total pages are = $\frac{2^{32}}{4KB} = 2^{20} \Rightarrow 1M$ pages.

Each page size is 4KB. So, again apply paging on page table of each

1K. Total no. of entries = $\frac{1M}{1K} = 1K$.

Two level addresses = 20 → no. of pages

12 → offset.

Divide again pages of each size = 1K $\Rightarrow$ It stores address of outer page for storing address it

require 10 bits

Performance :-

* Main memory access time = "m"

* Effective access time = $m + m + m = 3m$

$\downarrow \quad \downarrow$
 Inner Outer

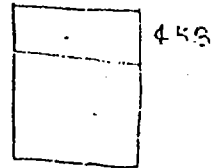
* Effective access time with 'n' level paging = $(n+1)m$.

For reducing access time using TLB :

TLB hit ratio = 'x'

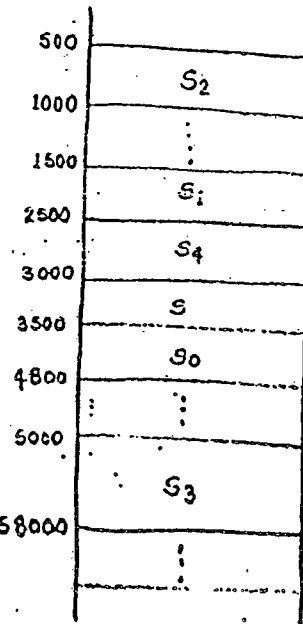
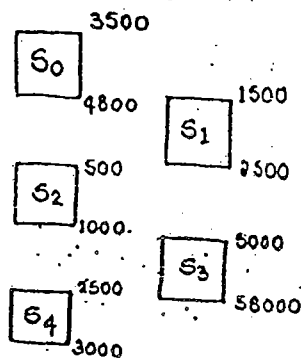
$$= x(c+m) + (1-x)(c+3m)$$

For n-level pages = $x(c+m) + (1-x)(c+(n+1)m)$



Segmentation

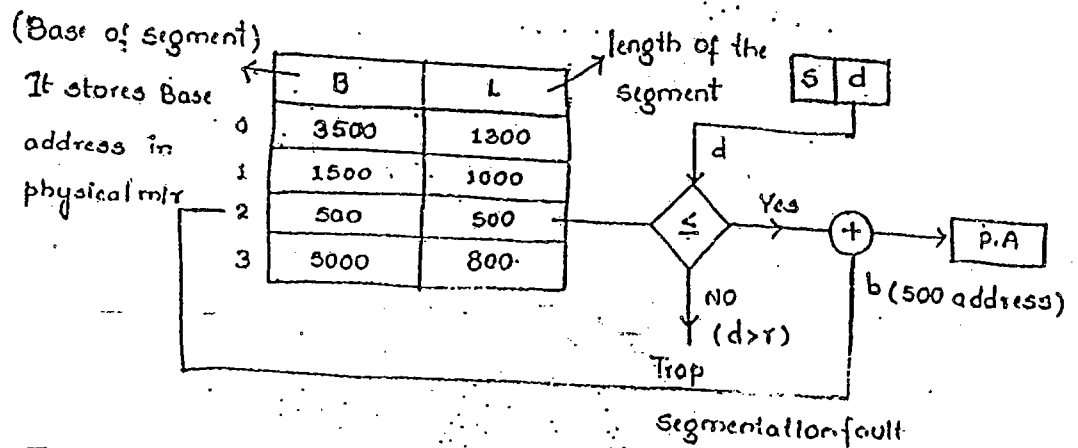
Paging doesn't preserve user's view of memory allocation.



Segment is a piece of code.

(program, divided into unequal sizes is called Segment (for equal size is called pages)).

- (1) Divide the program into segments (VAS organisation).
- (2) Using policies place the segments.



In page table, all are equal sizes. So, no need to compare the required word present within limit (or not). Here, unequal no. of partitions for checking the offset present in within the limit use magnitude comparator.

Segmented paging :-

Introducing paging (or) segmentation

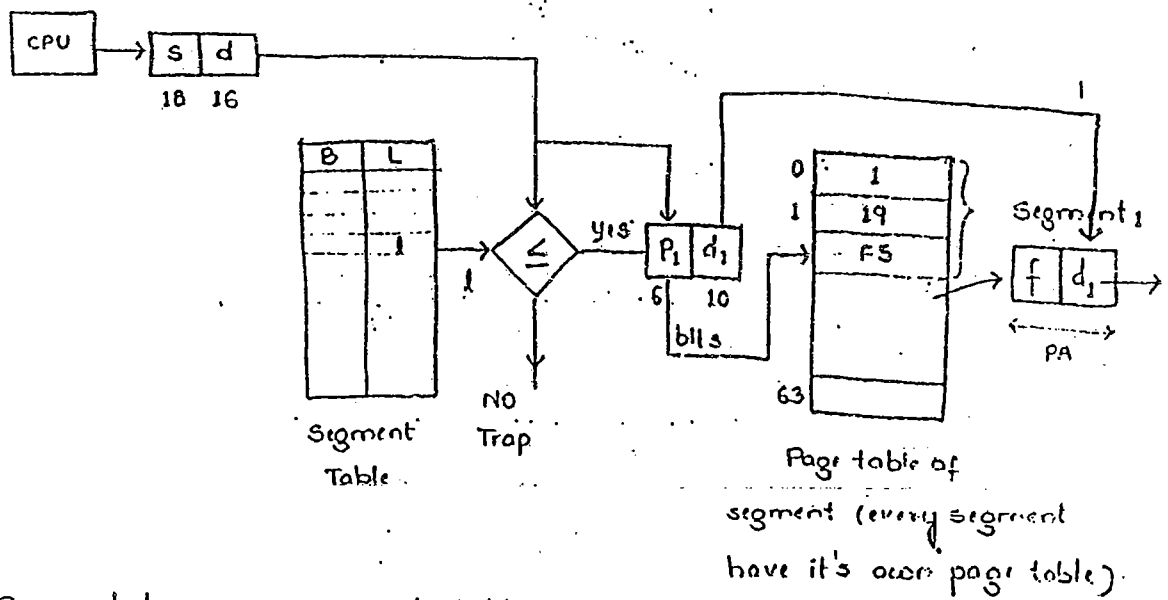
$$\text{Virtual size} = 34 \text{ bits} \quad \langle s, d \rangle = \langle 10, 16 \rangle \text{ bits}$$

$$\text{Max. size of seg. size} = 2^{16} = 64 \text{ KB}$$

offset

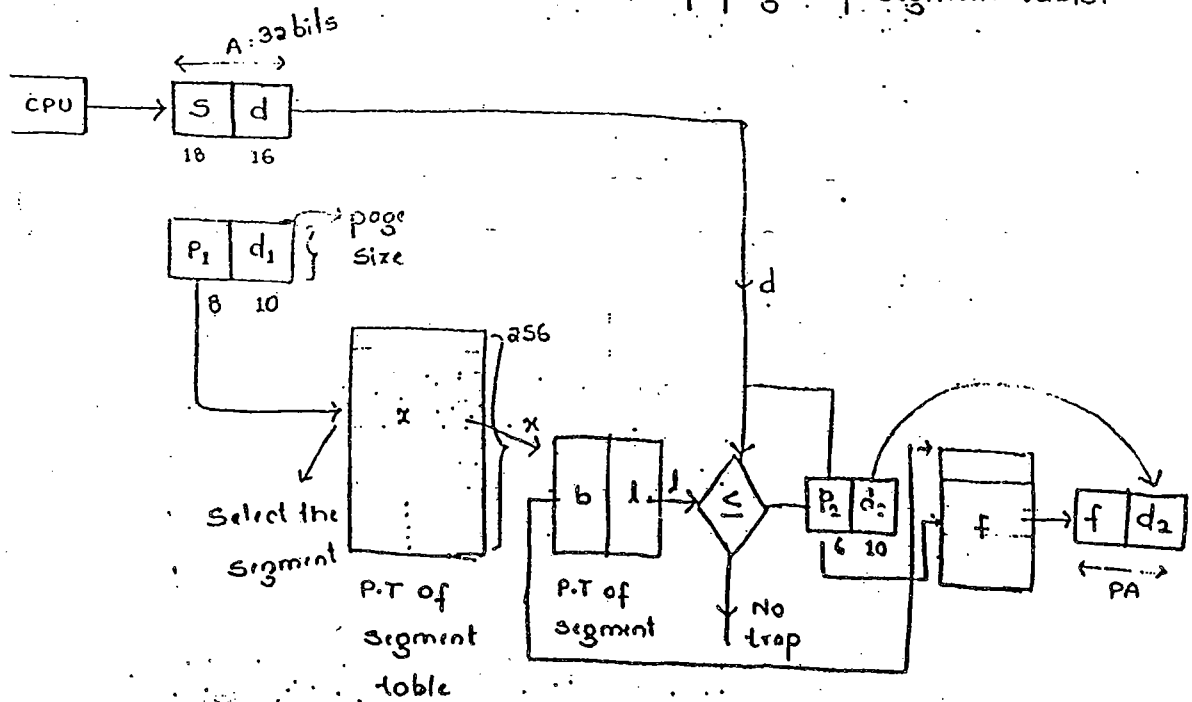
$$\text{No. of segments} = 2^{18} = 256 \text{ K}$$

Segmentation of Segment Size :-



Segmentation of Segment table:

x = Address of page of segment table.



Paging Vs Segmentation :-

	Internal Fragmentation	External Fragmentation
Paging	✓	✗
Segmentation	✗	✓

Virtual Memory

Virtual memory gives an illusion to the programmer that a huge amount of memory available for writing programs greater than the size of available physical memory.

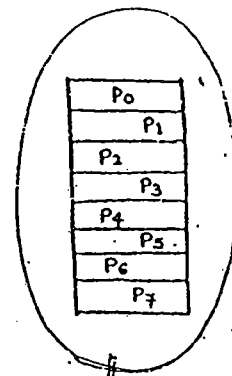
Virtual memory implemented in secondary memory.

Main memory < virtual memory < secondary memory

Demand paging :

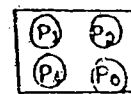
Virtual memory is implemented with demand paging and demand segmentation.

f ₀	P ₃
f ₁	P ₅
f ₂	P ₆
f ₃	P ₇



virtual memory

Implemented in S.S. maximum virtual memory is limited by capacity of secondary storage



Demand paging

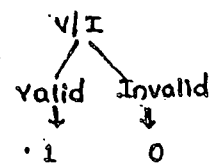
Pure Demand paging

Started with all the frames empty right from the first demand for page.

Protected Demand paging.

prefetch load is low

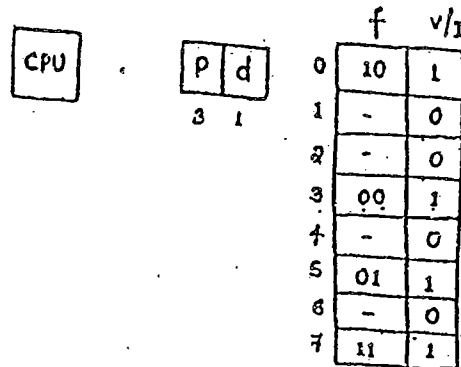
if all the frame were filled



Page fault rate is higher in pure demand paging and less in prefetched demand paging

pure demand paging is connection less. It takes less
 less time, and buffered is connection oriented it takes
 more time for execution.

54

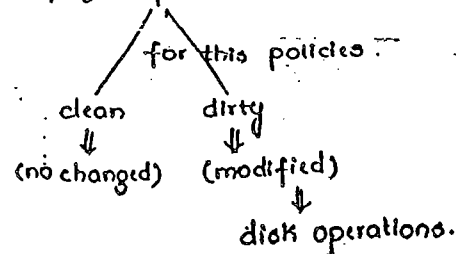


when page fault occurs, duty of virtual handler:-

- * Process is blocked
- * Virtual handler $\rightarrow$ CPU
- * v.m handler $\rightarrow$ Device manager
- * Device manager $\rightarrow$ Device controller
- * Device controller $\rightarrow$ Device buffer.

page fault service time
 $\Rightarrow$ generally 10^{-3} sec. for
 memory accessing, it
 takes $\frac{10^{-6}}{10^{-9}}$ sec.

- * D-B $\rightarrow$ m.m (DMA) transfer the page.
- * (i) Empty free is available $\Rightarrow$ (1) disk operation is enough.
- (ii) No empty free $\Rightarrow$ page replacement.



- * Update page table
- * Unblock the process

address

--	--

Maximum Virtual memory size is less than
 disk i.e. not more than disk

Performance

Main memory Access Time (M.M.A.T) = 'm'

P.E.S.T = 's' ($s \gg m$)

Page fault rate = 'p'

Page hit ratio = $1-p$

$$\boxed{EAT = (1-p)m + p \cdot s}$$

$$\boxed{E.A.T = (1-p)am + p(m+s)}$$

Hit ratio is more $\Rightarrow$ follow the units of main memory access times.
 $\Rightarrow 1.9999 \mu s$

$$E.T.T = \frac{1}{K} (i+j) + \left(1 - \frac{1}{K}\right) \cdot i$$

$$= \frac{1}{K} + \frac{j}{K} + i - \frac{i}{K}$$

Page replacement :-

1) frame Allocation policies :

total no. of frames = M

no. of processes = 'n'

demand of each processes for

frames = S_i (it depends on

total no. of pages of program)

total demand of process:

$$S = \sum_{i=1}^n S_i$$

(Allocation policies) 50%
 ref. str.

S_i Equal proportionate

P_1 10 10

5

P_2 25 10

12 only given to

P_3 3 $\rightarrow$ 10

1 50% of resource

P_4 12 10

6

P_5 20 10

10

70

It is applicable for all, which have equal demand.

If $M = 60, n = 5$

$$\frac{M}{n} = \frac{60}{5} = 12 \quad \frac{n_i}{3} \times m \text{ (and policy)}$$

$$P_1 = \frac{10}{70} \times 50 = \frac{500}{70} = 7.14 \approx 7$$

$$P_2 = \frac{26}{70} \times 50 = 18.57 \approx 19$$

(b) Page Reference string :-

Set of successively unique pages referred in the given list of logical addresses :-

463, 182, 184, 195, 435, 963, 967, 834, 128, 534, 765, 784, 012, 634, 686

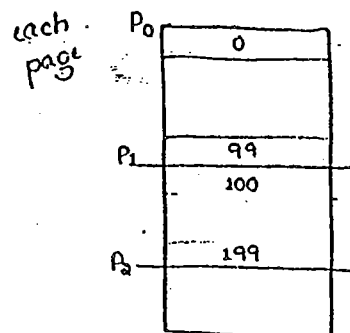
page size = 100 words (consider).

463 $\Rightarrow$ It belongs to
page address

$$\frac{463}{100} = 4 \text{ (every page contains 100 words)}$$

$$463 \% 100 = 63$$

63 $\rightarrow$ Offset.



$= \{4, 1, 1, 1, 4, 9, 9, 8, 1, 5, 7, 7, 0, 6, 6\}$ // If occurred successively, then consider only once:

ref. string $= \{4, 1, 4, 9, 8, 1, 5, 7, 0, 6\}$ // '4' occurs again but not successively, so consider only once.
length = 10

No. of unique pages referred in the given list of logical addresses :-

$$n = \{0, 1, 4, 5, 6, 7, 8, 9\} = 8$$

Eg. :-

reference string = 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

length = 20, unique = $\{0, 1, 2, 3, 4, 7\}$ 3 frames

$$n = 6$$

Frame size = 4

$$3F \rightarrow 16$$

page fault = 10

$$4F \rightarrow 10$$

7	7	0	7
0	3	0	4
1	2	3	1

7	2
4	7
0	1
1	0

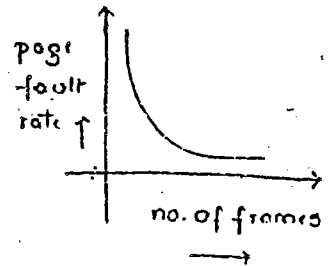
4 frames

Pure demand page (PDP):-

(1) page table policy decides which page is replaced.

* conventional policy is FIFO.

criteria : A.I (It maintains A.I in page table)



Reference string :-

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

FIFO :

3F $\rightarrow$ 9

4F $\rightarrow$ 10

↓

As frames increases,

page faults also increase $\Rightarrow$ Anomaly

Belady's Anomaly $\Rightarrow$ unusual behaviour.

3 frames

1 5
2 3
3 4

4 frames

1 6 4
2 5
3 2
4 3

* with the increase of page frame to a process, the page fault rate also

"sometimes" increases, it is called Belady's Anomaly.

(FIFO & LRU based policies suffer from Belady's Anomaly)

Optimal Replacement :-

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

F₄ $\rightarrow$ 8

F₃ $\rightarrow$ 9

7 1
0
1 7
2

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 6

F₃ $\rightarrow$ 7

1 3
4 4
5 5

1 4
2
3
4 5

4F $\rightarrow$ 6 (But it is not implementable)

3010712010

Friday

Page Replacement policies

Reference string : 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.

(i) FIFO :

3 frames $\Rightarrow$ 15

4 frames $\Rightarrow$ 10

(ii) Optimal replacement :

3 frames $\Rightarrow$ 9

4 frames $\Rightarrow$ 8

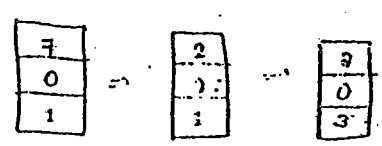
(iii) Least Recently used (LRU) :-

Criteria : Time of reference (TOR)

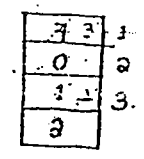
Least TOR $\Rightarrow$ is considered for victim.

P.T

P	V/I	AT	TOR



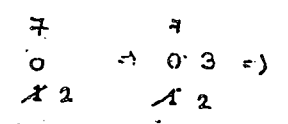
7 is used for more longer time, so '1' gets replaced with '2'



(iv) Most Recently used (MRU) :-

3 frames $\Rightarrow$

4 frames $\Rightarrow$



(v) Counting Algorithms :-

* Least Recently used (L.R.U)

* Most Recently used (M.R.U)

* After loading page in main memory, how many times a page is referred so far.

performance of LRU is most closest to Optimal.

2	1 $\Rightarrow$ first dropped in
0 $\Rightarrow$	2 is replaced
1	1

$\therefore$ OS uses LRU (or) LRU approximations.

LRU Approximations:-

* works like LRU.

* Approximate to the behaviour of LRU

} They are not pure LRU but works like LRU

Reference bit (R) Algorithm:-

* Every page table contains a reference bit within it.

Page Table				
P	f	V/I	AT	R
0	a	1	3	1
1	b	1	2	0
2	-	0	-	-
3	d	1	0	1
4	k	1	1	0

R = 0 - page has not been referred so far during the present epoch.
 1 - page has been referred atleast once during the present epoch.

* "At the end of epoch, reference bits are cleared to zero".

Epoch $\Rightarrow$ duration of time $\Rightarrow$ Time divided into discrete intervals (Time Quantum) called epochs.

During current epoch, what pages are referred (or) not referred are indicated by reference bit (R).

	R
P _i	0
P _j	0
P _k	0

} Initially

	R
P _i	1
P _j	0
P _k	1

} page referred in new epoch.

End of old epoch & starting new epoch has Initial value = 0

Start searching the page fault (victim) from initial, using ref. bit of LRU approximation is Reference bit algorithm.

If all reference bit values = 1, then this algorithm fails. So, we introduce a new algorithm called second chance.

(2) Second chance / clock Algorithm:

Criteria: Arrival time + reference bit
A.T + R

FIFO based algorithm.

Start the search of pages from the A.T, and if it is already referred, then give a second chance to it (i.e., value changed from 1 to 0).

P	f	V/I	AT	R
0	a	1	3	X 0
1	b	1	2	X 0
2	-	0	-	-
3	d	1	0	X 0
4	k	1	1	X 0

(3) Enhanced Second chance: Avoids unnecessary page back.
(not-recently used)

Criteria: (R) + (M) (modified bit)
+
(dirty bit)

modified bit $\Rightarrow$ checks whether the contents of page are modified (or) not.

0 $\rightarrow$
1 $\rightarrow$ write back.

R=0, M=1 :-

R	M	
0	0	not R, not m
X	0	not referred but modified
1	0	Referred but not modified
1	1	Referred & modified

Suppose $P_i \rightarrow$

R	M	R	M
1	1	0	1
0	0		

It is referred in old epoch and it is not referred in new epoch but modified.

when page fault occurs, consider the value of "00".

0 $\rightarrow$ LRU

we look for '01' combination next, since, it represents LRU

Then '10' is considered & at last '11' is considered.

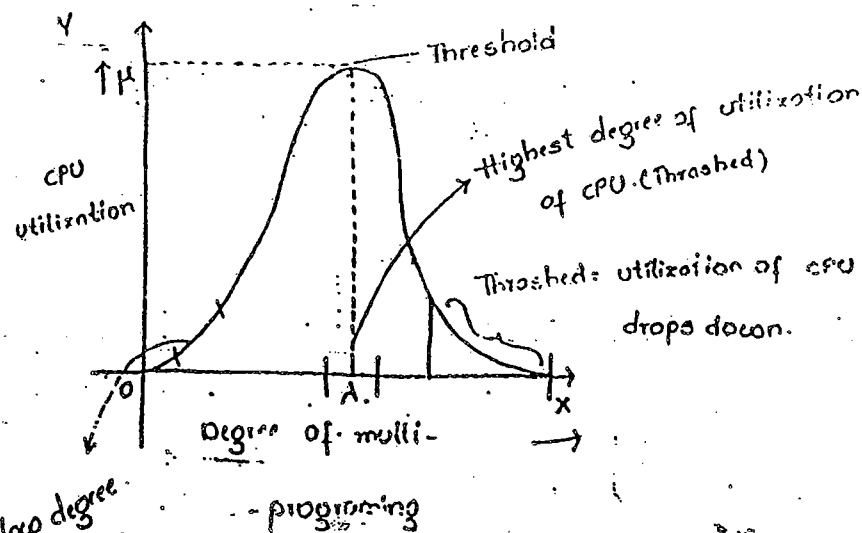
Thrashing :- (High paging activity) (page fault rate).

* Excessive / High page fault rate is called Thrashing

* It is also an undesirable state of the system like deadlock.

Reasons for Thrashing:-

Primary	Secondary
(1) Lack of memory (frames) i.e., if frame allocation to a page get reduced.	(1) page replacement algorithm.
(2) High degree of multiprogramming	Drawback: fragmentation (a) page size small (2B) large (1004B) $\rightarrow$ 2 pages $\rightarrow$ 2 page faults (512 pages) less pages (then less page faults) when reduces thrashing * (3) program structure:



Thrashing control Strategies :

I. prevention & Avoidance

- * Controlling degree of m.p in and around " μ ".

page size:

Temporal (Time) $\rightarrow$ Thrashing

Spatial (space) $\rightarrow$ Internal fragmentation

Program Structures :-

int A(1...128, 1... 128)

page Size = 1280

i) for $i \leftarrow 1$ to 128
for $j \leftarrow 1$ to 128
 $A[j, i] = 5$

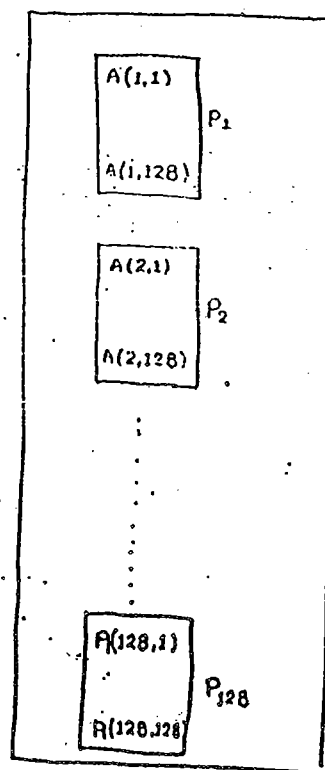
ii) for $i \leftarrow 1$ to 128
for $j \leftarrow 1$ to 128
 $A[i, j] = 5$

CPU Allocated

P_1

P_2

P_{127}



II. Detection & Recovery

- * low CPU utilization, along with high degree of m.p.

- * paging disk utilization must be high.

Recovery:

- * By applying concept of "process suspension" (swapping out some processes to secondary memory)

Column $\leftarrow A[j, i]$
major order $A[1, 1]$

Diagram illustrating the total number of page faults for a 128x128 matrix. The matrix is divided into 128 rows and 128 columns. The first row is labeled P_{128} , and the first column is labeled P_1 . The matrix is divided into 128 blocks of size 128x128. The first block is labeled P_1 , and the last block is labeled P_{127} . The total number of page faults is calculated as $128 * 128 = 16384$. The diagram shows that for every 'i' only 1 page fault occurs, so total 128 page faults occur.

* Based on locality of reference.

$g^{(1)}$

{

$\vdots$

$hc)$ — AK —

$\vdots$

}

locality

$hc)$ ←

{

$\vdots$

— 15 K

$\vdots$

}

By knowing, no. of frames
required based on Guess/
Estimate reference string
and divide.

Total = 32 H

Objective :-

* The main objective is to reduce page fault rate.

* Locality may be a function, block, class (or) module.

* Hold the pages of the locality at which the present function is performing.

Guess / Estimate : Estimating the ref. string ^{size} in each function.

Reference string (pri):

7, 6, 8, 9, 6, 8, 9, 7, 12, 18, 20, 22, 18, 20, 12, 23	28, 29, 34, 38, 27, 28, 29, 38, 34
main()	g()

working set window (WSW):

* Set of unique pages referred in the given list of reference string during the past ' Δ ' references.

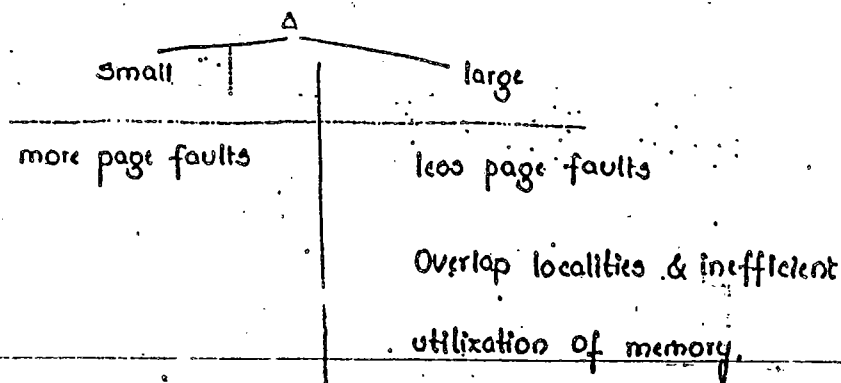
↓
Integer (Guess)

Let $\Delta = 10$ (Guess) $\Rightarrow$ no. of integers in $g()$ = 10 (length)

WSW = { 23, 27, 28, 29, 34, 38 } = 6.

↓
Unique reference string from the
Set of $g()$ function.

Success will depend on the value of Δ .



* If we guess the value of Δ to be small, then there is a chance of occurring more page faults.

Eg:- If $\Delta = 10$

W3W3 = { 23, 28, 29, 34, 38, 24, 28, 23, 29, 38, 34 }

page faults = 6.

If $\Delta = 5$

W3W3 = { 28, 23, 29, 38, 34 }

page faults = 5.

If $\Delta = 5$

W3W3 = { 23, 28, 29, 34, 38 }

page faults = 5.

for the same set of reference string, if $\Delta = 10$ (more no. of integers are consider, then page faults = 6. other than if $\Delta = 5$; it have page faults = 10.

* Let $p_i = 1, \dots, n$

$(w_3w_3)_i$ where $i = 1, \dots, n$

Total demand (s) = $\sum_{i=1}^n (w_3w_3)_i$

Total frames available = "m".

* If $m \approx s$ (no thrashing, system is perfectly balanced).

* If $m \gg s$ (no thrashing)

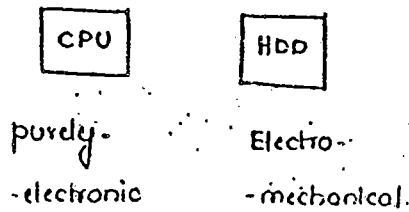
(scope of increasing degree of multiprocessing).

* If $m \ll s$ (Thrashing)

1000 10,000

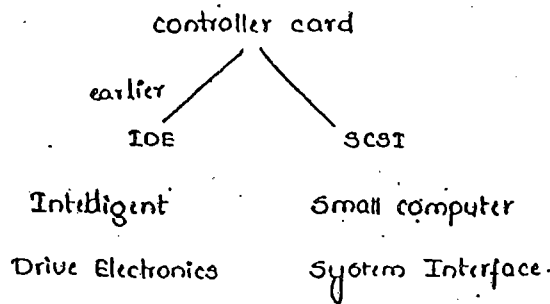
File System & Device management

Interface :



Every secondary/ Tertiary media must have their own file system.

(1) We need a controller interface as controller card / chip. $\Rightarrow$ Hardware requirement

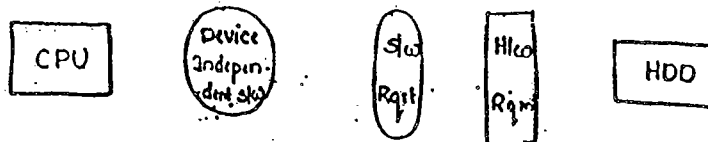


(2) Software requirement :-

* Device driver.

3) Device Independent software :

* It is called as file system. (Third party device drivers).



Files & Directories :-

File :- Collection of logically related set of records of an entity.

* It is considered for data structure.

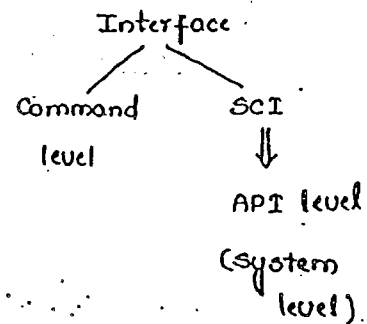
* Data structure :-

(1) Definition

(2) Representation / Implementation :-

* Flat structure
* Record structure
* Tree structure

} $\Rightarrow$ file structures



(3) Operations :-

- * Create a file
- * Open a file
- * Read / write / append / seek / modify / Truncate
- * close a file
- * Delete a file

(4) Attributes :-

* name, extension of file

* Type of file

* Size of file

* owner

* permission

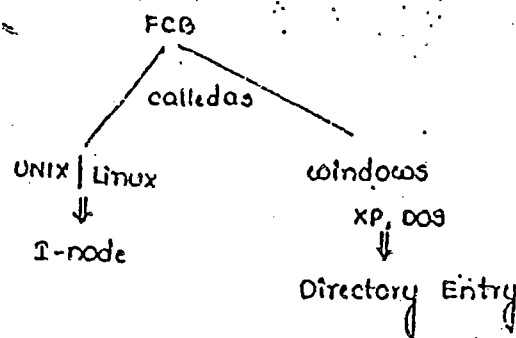
* mode of access (sequence / random)

* Time & Date stamps

* link count

} stored in
file control block (FCB)

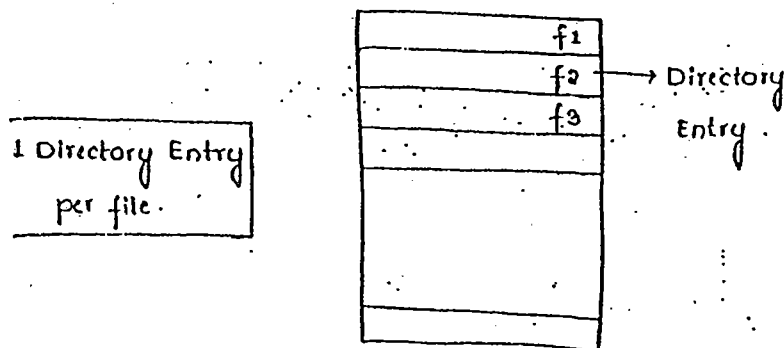
* Attributes of a file are stored in file control Block (FCB).



Directory :-

* It contains information about files i.e., metadata of files.

Abstract view of
directory (metadata of files)



Organisation of Directory Structure :-

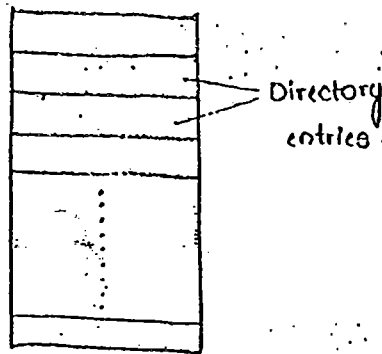
- * Single level directory
- * Two level directory
- * multi level / Tree Structures
- * Acyclic Graph.

31/07/2010

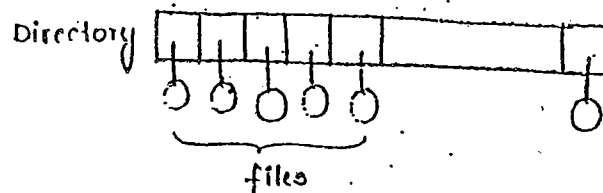
Saturday

Directory Structures :-

Abstract view of Directory



(a) Single-level Directory structure :-



* No sub-directories are present. (one entry for each file).

* All the files are managed by one directory.

Advantages :

* Simplicity

* Searching

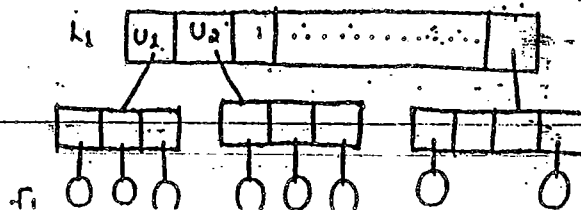
* Name conflicts

Disadvantages :-

* Only one directory, no sub-directories.

* Two different users, simultaneously can have same filename.

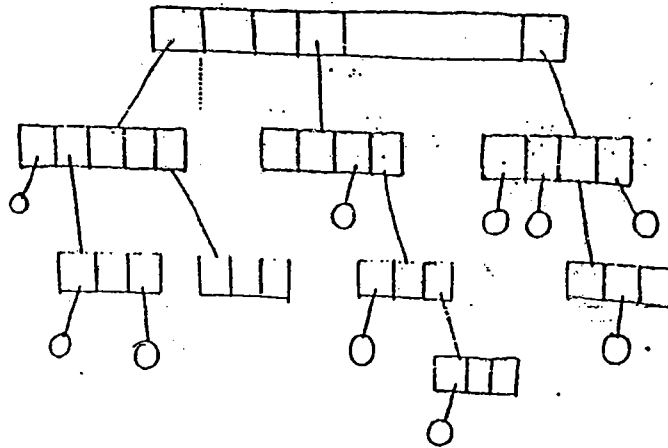
(b) Two-level Directory structure :-



Advantages :-

- * Creating different files with same name. (flexibility)

3) Tree-structured directory :-



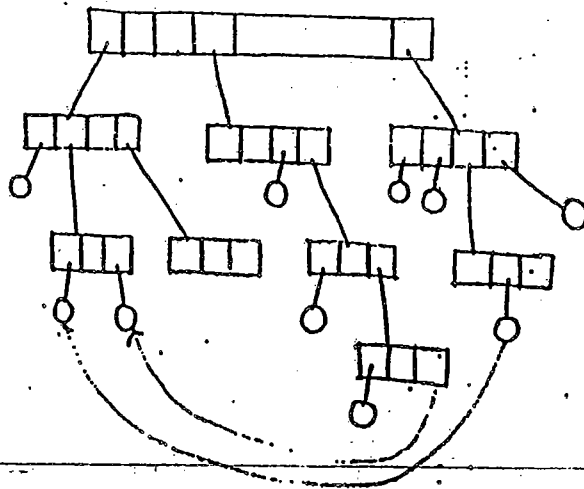
Drawback :-

* File sharing :-

* Sharing by duplication has drawbacks of :-

- * Inconsistency
- * wastage of space

To overcome duplication in file sharing, we have the concept of file sharing with links called "Directed Acyclic Graph".

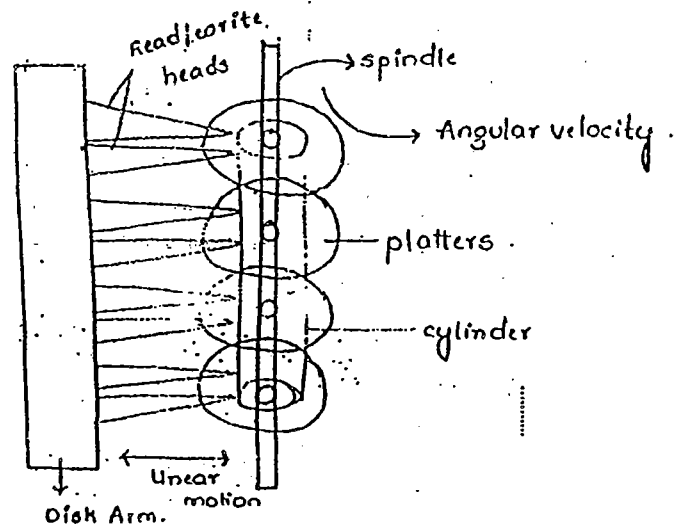


Device characteristics:

- * Major component in secondary storage is disk.

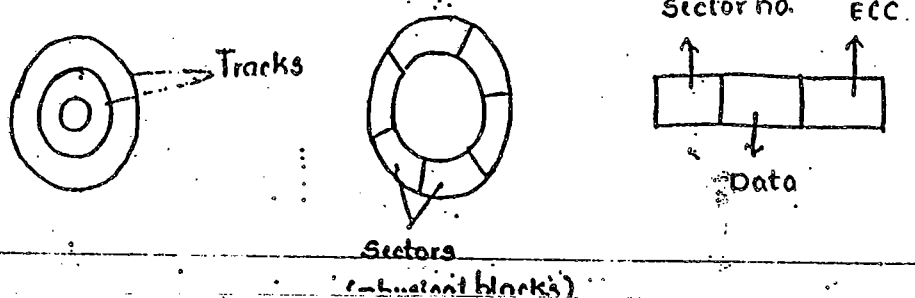
Physical and logical structure of Disk :-

Physical structure / geometry of Hard disk :-



- * Hard disk consists of set of platters, with a spindle.
- * Each surface of platters is associated with Read/write heads.
- * Two Types of motions are supported :
 - * Linear velocity (move forward & backward) with disk armature.
 - * Angular velocity with disk spindle.
- * Tracks are divided into sectors.

Sectors :-



Sectors can be represented by three fields :-

- * Sector no.
- * Data
- * Error correction code (ECC).

Error Correction code (ECC):

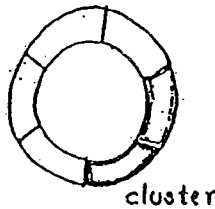
- * To detect the deadlock. By reformatting the bad sector, some sectors can be made good sector.

Cylinder :-

A group of same track number.

Cluster :

A group of one (or) more adjacent sectors is a cluster. A cluster is one of the unit of I/O transfer.



I/O Transfer :-

The amount of time taken to a sector to make a move from present area to desired area is called seek time.

Seek time:

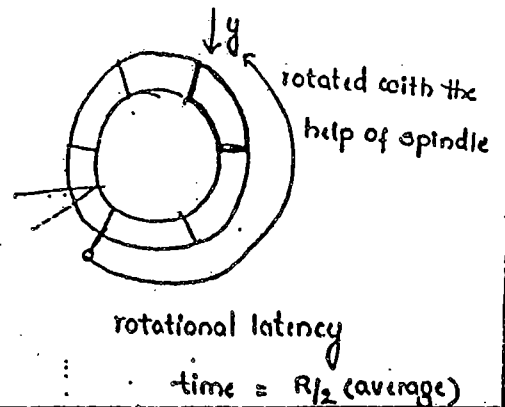
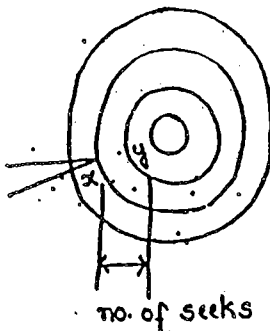
time taken to

travel from 'x'

to 'y' (or)

linear motion

from x to y



Seek time + (Rotational) latency time

$$\downarrow$$

$$\frac{R}{2} \text{ (on an average)}$$

where: R = Time for one rotation.

Transfer time (Transmission Time) :-

* The transfer time depends on two factors:

* Track size (S) bytes

* Rotation rate (RPM)

Transfer time:

* Read the data from sector

Sector size = ' x ' bytes ($x < S$) i.e., Sector size < Track size.

Rotation rate = ' r '.

Time for one rotation = ' R '.

$$R = \frac{60}{r} \text{ s}$$

$$3600 \text{ r} = 60 \text{ s}$$

$$1 \text{ r} = ?$$

$$\text{If } \frac{60}{r} \text{ s} \Rightarrow 's' \text{ bytes}$$

$$\Rightarrow \frac{60}{3600} = \frac{1}{60} \Rightarrow R = \frac{60}{r} \text{ second}$$

$$? \Rightarrow 'x' \text{ bytes (read)}$$

$$\Rightarrow \frac{x * 60}{r * \cancel{s} \text{ bytes}} \times \cancel{\text{sec}} = \frac{x * 60}{r}$$

Disk I/O Time :-

$$= \underbrace{(\text{Seek Time}) + (\text{Latency Time}) + (\text{Transfer time})}$$

$$= ST + \frac{R}{2} + \frac{x * 60}{r * S} \text{ (sec)}$$

$$\frac{60}{r} - S$$

$$? - x$$

$$\Rightarrow \frac{x * 60}{r * S}$$

Effective Data Transfer Rate (DTR) (Bytes/sec) :- no. of bytes in 1 sec.

Assume Track Size = 's' bytes

RPM = 'r'

Rotation time = $\frac{60}{r}$ sec

$\frac{60}{r}$ sec ——— 's' Bytes

1s ——— ?

$$\text{D.T.R} = \frac{r * s}{60} \text{ bytes/sec}$$

Eg:- consider a hypothetical disk with :-

16 platters

↳ 2 surfaces

Disk seek time = 30ms

↳ 2 Tracks

RPM = 4000

↳ 512 sectors

↳ 2KB

Then calculate (i) capacity (ii) I/O time (iii) DTR

Sol:-

(i) Total no. of sectors = $2K \times 512$

capacity of one surface = $2K \times 512 \times 2KB$

$$= 2^{11} \times 2^9 \times 2^{11}$$

Capacity of two surfaces = $2^5 \times 2^{11} \times 2^9 \times 2^{11} \times 2^{36}$

$$= 64GB.$$

(ii) I/O time = $\underbrace{\text{seektime} + \text{latency time} + \text{Transfer time}}$

$$\frac{R}{2} + \frac{2 * 60}{r * s} (s)$$

$$R = \frac{60}{4000} \text{ s} \Rightarrow \frac{R}{2} (\text{latency time}) = \frac{60}{8000} \text{ s}$$

Transfer time:- $\frac{60}{4000} \text{ s} \text{ --- } 1 \text{ MB}$
 $\text{? --- } 2 \text{ KB}$

$$\Rightarrow \frac{60 \times 2 \text{ KB}}{4000 \times 1 \text{ MB}} \text{ s} = x$$

$$\therefore \text{I/O time} = \underbrace{S.T + L.T + T.T}_{\frac{R}{2} + \frac{x \times 60}{8000} \text{ (s)}}$$

$$= 30 \text{ ms} + \frac{60 \times x}{8000} \text{ s} + x$$

$$\frac{80}{8000} \text{ s} \times$$

(iii) DTR :-

$$\frac{60}{4000} \text{ s} \text{ --- } 1 \text{ MB}$$

$$1 \text{ s} \text{ --- } ?$$

$$\frac{4000 \times 1}{60} \text{ MB/s}$$

Logical structure of disk (formatting process) :-

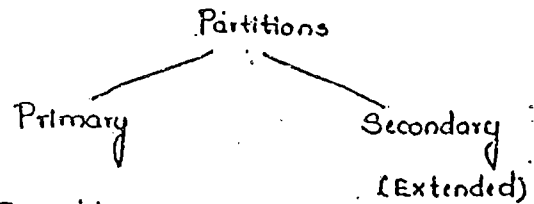
Popular file systems :-

- * NTFS
 - * Solaris
 - * FAT 32
 - * UFS
 - * XFS
- } popular.

fdisk $\Rightarrow$ create, delete, & edit the partitions

XP	LINUX	OS
NTFS	UNIX FILE SYSTEM	FAT 32

partitions
(volumes)
(c; d; e;)



In the partitioning of a system, there must be atleast 1 primary partition and the rest may or may not be of secondary.

* Bootable
(stores o.s into it)
&
userdata

* Non-bootable
 $\downarrow$
only userdata
get stored.

Computer system, which supports multiple o.s to boot is called multi-boot computer.

File format system:

we can select any file format with different o.s's.

Master Boot Record
(MBR)



Boot
process

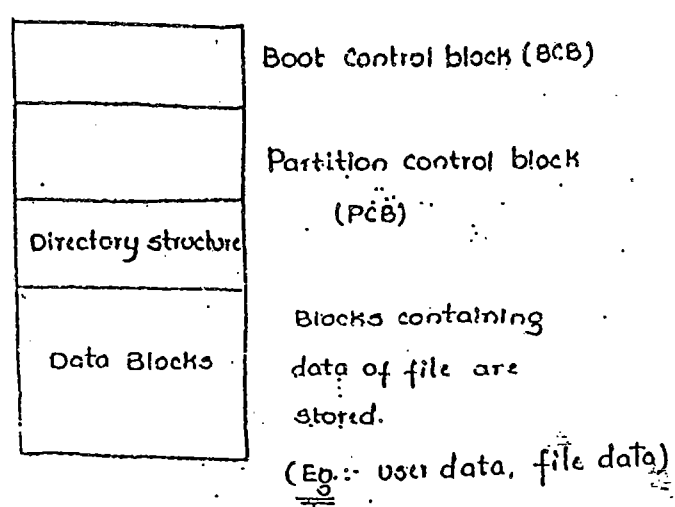
partition
table

(It represents the complete picture of partition)

(Booting the selected o.s)

(only one entry for the partition).

Partition Structures :-



BCB block is empty $\Rightarrow$ If there is non-bootable partition.

BCB can be represented in different terms :-

- * Boot Block (In UNIX)
- * ~~Master~~ Partition Boot Sector (In NTFS) (It contains boot of that sector).

PCB gives information about other blocks.

PCB can be represented in different terms :-

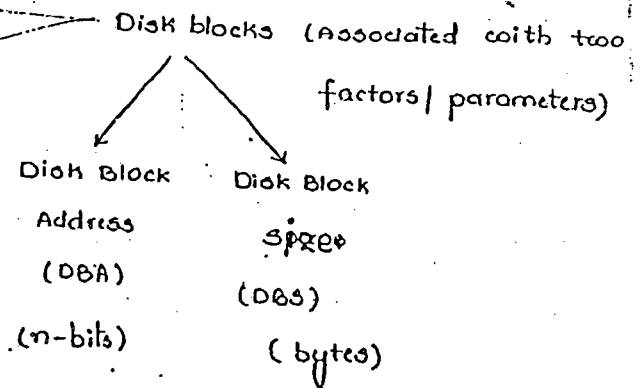
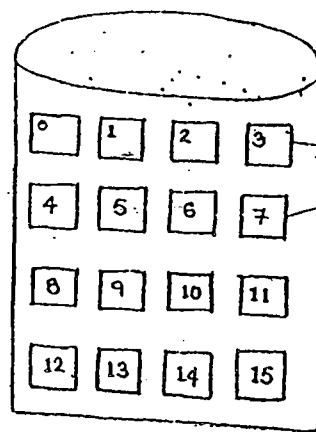
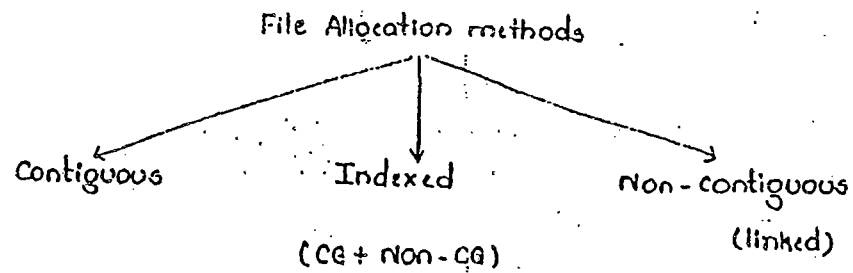
- * Superblock (In UNIX) (gives information about others)
- * Master file Table (M.F.T) (In NTFS)

Disk space is being organised in blocks.

Block $\Rightarrow$ unit of Allocation.

Allocating the disk space methods $\Rightarrow$ File Allocation methods.

Disk File Allocation methods (or) Disk Space Allocation strategies :-



consider DBA = 16 bits

DBS = 1 KB

Then, maximum file size (in bytes) = 64 MB

↓
depends on / limited by
disk size

$$\text{Disk Size} = 2^{16} \times 1 \text{ KB}$$

$$= 64 \text{ K} \times 1 \text{ KB}$$

$$= 64 \text{ MB}$$

(1) Contiguous Allocation :-

first block of disk used by file
↑
directory entry

filename	Starting DBA	file size (no. of blocks)
test.c	5	4
tmp	10	6

(5, 6, 7, 8 blocks)
(10, 11, 12, 13, 14, 15 blocks)

Atleast 'm' contiguous blocks require.

Performance :-

(1) Fragmentation :-

Internal

External

(not contiguous, but
bifercated with memory)

If last block files are
not fully utilised, then
it goes waste, which
represents internal fragr-
-entation.

(2) Increasing file size :

May / May not be possible
↓
Inflexible

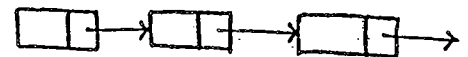
(3) Type of Access :

Sequential

Random

Both are
supported

Sequential (Array)



Random (linked list)

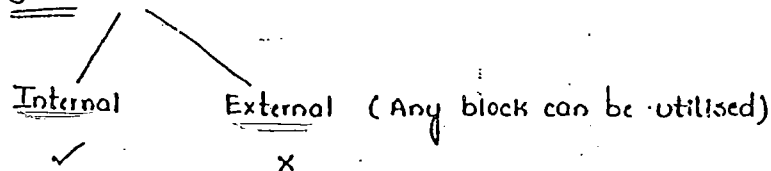
Since, it supports random access, it is faster.

(a) Non-Contiguous / Linked Allocation :-

Filename	Starting DBA	Ending DBA	
test.c	4	3	(4, 9, 6, 3)
temp	5	2	(5, 1, 2)

Performance :-

(1) Fragmentation :-



(2) Increasing file size :-

File size increment is possible, as long as free blocks are available.

(3) Types of Access :-

Sequential access only possible: No random access. So, it is slow.

Drawbacks :-

* Some disk space is consumed for storing pointers; that addresses next instruction block.

* vulnerability of links :-

↓ expose danger ↓ breakage of links

↓
Then, file gets truncated.

If any link break then truncating a file

Reason for storing ending DBA :-

* we store ending DBA, (along with starting DBA), because for the following :-

* Creating a new pointer done fastly.

i.e., To extend a new file along with the existing, it can be made faster.

* Checking file system consistency. (scandisk)

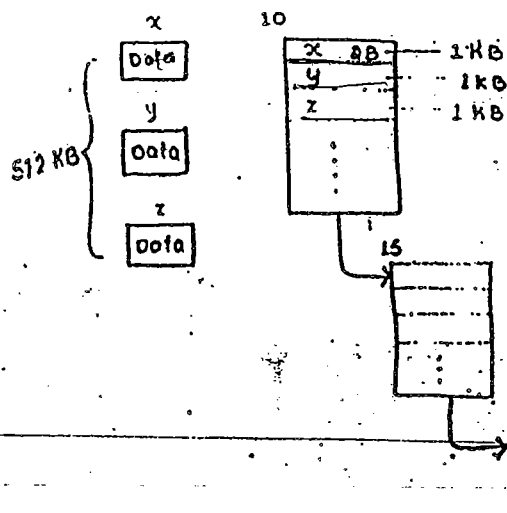
i.e., checking whether file is fully accessed or not.

(3) Indexed Allocation (Contiguous & Non-Contiguous) :-

Filename	I-Block
test.c	10
temp.	15

* A file can be allocated in both contiguous and non-contiguous order.

* Eg. 1:- DBA = 16 bits, DBS = 1 KB



$$DBA = 16 \text{ bits} = 2 \text{ Bytes}$$

$$\frac{1 \text{ KB}}{16} = 512 \text{ KB} \quad 2^{16} = 64 \text{ K}$$

$$64 \text{ K} \times 1 \text{ KB} = 64 \text{ MB}$$

512.

Using 1KB Index, we can represent 512KB no. of blocks.

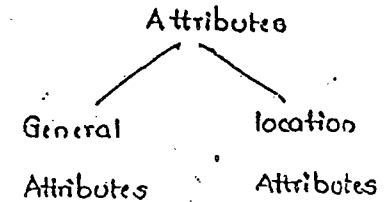
04/08/2010

Wednesday

UNIX - I-node structure

Directory entry consists of two fields :

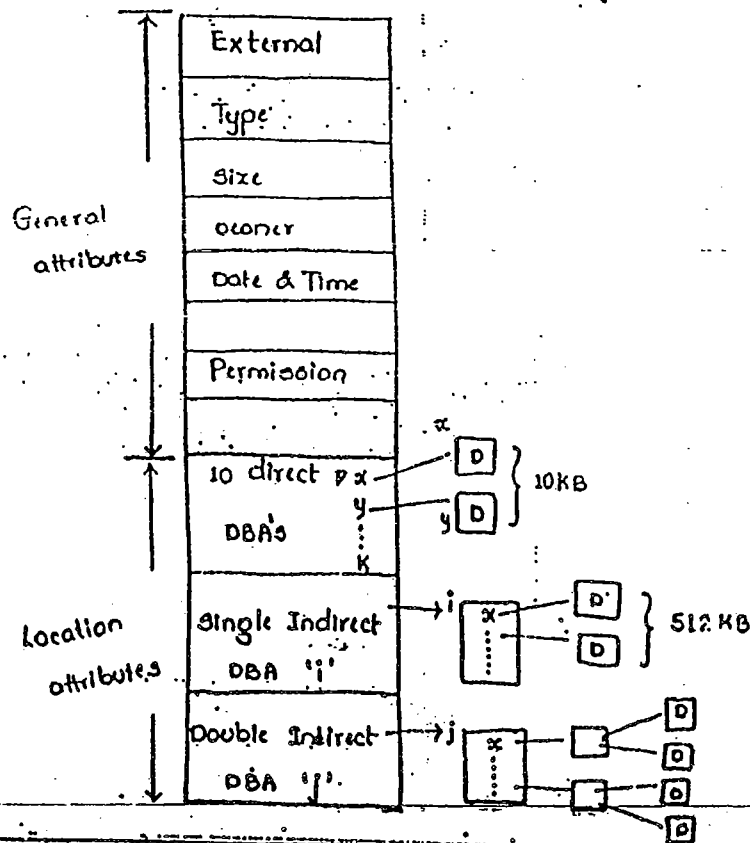
- * filename
- * I-node (contains Attributes).



(name,...)

filename	I-node (23)
test.c	23
temp	40

I-node (23) ⇒ Block of memory



* Maximum file size is limited by the maximum disk size = 64 KB

Single Indirect DBA :-

$$DBA = 16 \text{ bits} \quad DBS = 1KB$$

$$2^{16} = 64K$$

$$64K * 1KB = 64MB$$

$$512 + 10 = 522KB = 0.522MB$$

Double Indirect DBA :-

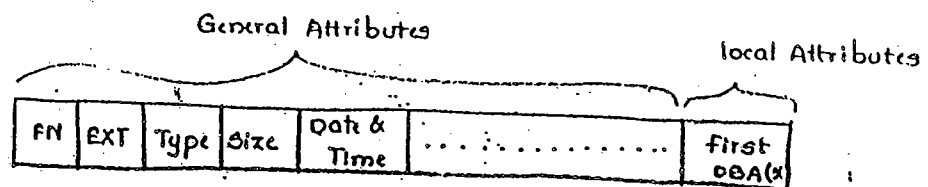
$$512 * 512KB = 2^9 \times 2^9 \times 2^{10}$$

$$= 2^{28} = 256MB$$

Total size = 256.522 MB (logical) $\Rightarrow$ not considered because max. file size is only 64

DOS directory Implementation :-

- * DOS directory implementation is complex than that of UNIX.
- * All attributes are embedded in the directory entry itself. There is no I-node in DOS implementation.



test.c Directory entry

$x \rightarrow$ first block address of data.

- * For the remaining data addresses to represent, we use file Allocation Table (FAT).

No. of Entries in FAT = No. of blocks on disk. n blocks = 0 to

(n-1)

* Only the first block address of data is being indicated as a field, and the remaining data addresses are stored in FAT table, and referred whenever required.

* To know the (block) address of data, move to 'x' address on FAT. It represents the next data address. If there is last address, then it is represented by "-1" (i.e., it is last data)

FAT: 16

0	5
1	-1
2	
3	2
4	-1
...	
x	3
y	9

next block address.

* FAT entry represents $\Rightarrow$ address of disk block. n-1

Tabular linked allocation

DISK FREE SPACE MANAGEMENT

Eg. :-

consider disk = 20 MB

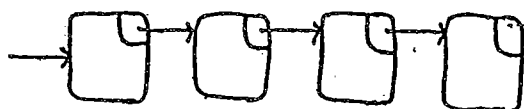
OBA = 16 bits ODS = 1 KB

$$\text{Total no. of blocks} = \frac{20 \text{ MB}}{1 \text{ KB}} = 20 \text{ K blocks}$$

Initial, 20 K blocks are free.

Approaches, to keep track of these 20 K blocks :-

(1) Free linked list :-



If any file requires the block, it deletes the particular size (eg. 10) and allocates it to the file.

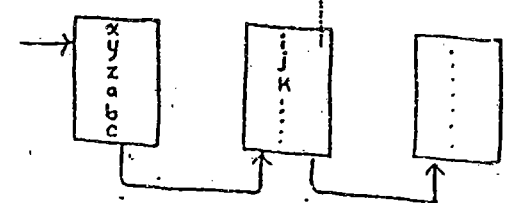
(2) Free list :

* The addresses of the files are stored in linked lists.

1 block $\Rightarrow$ 512

? $\Rightarrow$ 20K

$$\frac{20K^2}{512} = 40$$



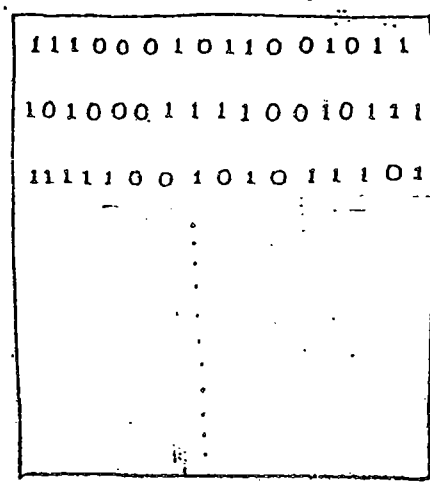
* If the disk is almost free, it requires 40 blocks.

* If the disk is almost full, it requires 1 block.

✓ (3) Bit-map method :

Associate a binary bit with each block

$\begin{cases} 0 & \text{— block is free} \\ 1 & \text{— block is in use} \end{cases}$



Size of bit map for the blocks = 20K bits.

1 block can store = 8K bits
? — 20K

$$\frac{20K}{8} = 2.5$$

$\approx$ 3 blocks.

It requires searching time (more) for a file.

So it is slower than free list.

Block size $\leq 2^{10}$

S - DCS

B - Blocks

D - DBA

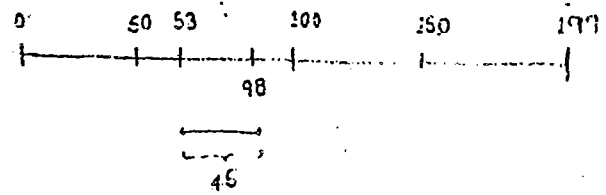
P - Free blocks

Disk Scheduling :

Ready Queue

P₀ P_b P_c

CPU



CPU
P_b

Process Queue

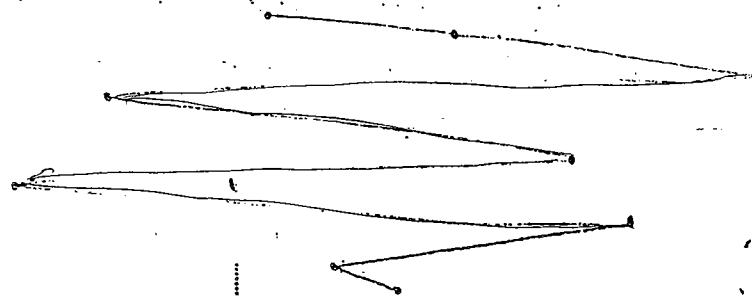
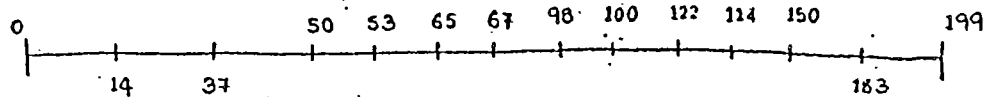
P_c P_b P₀

Device
(disk)

- Disk scheduling is used to represent which process is serviced next, when a process completes its operations.

(1) FCFS :

Requests : 98, 183, 37, 133, 14, 124, 65, 67



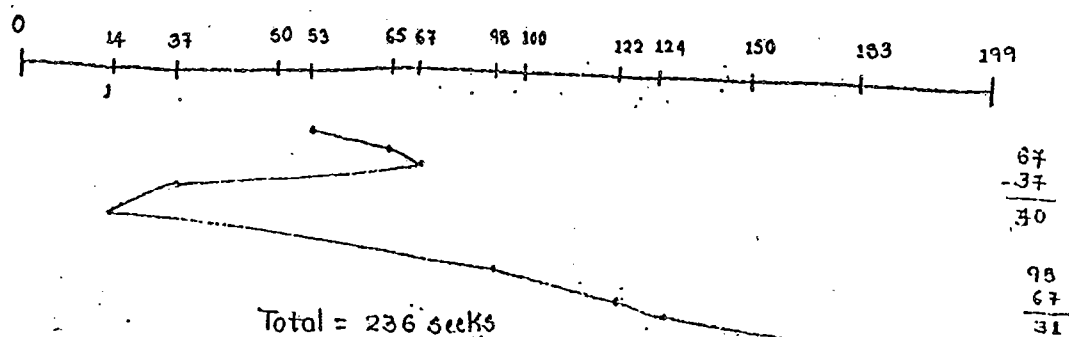
98 - read
from track
'9'

Total = 640 tracks

To reduce no. of seeks, we have SSTF.

(a) Shortest Seek Time first (SSTF) / Nearest Track Next (NTN) :-

(Greedy method)

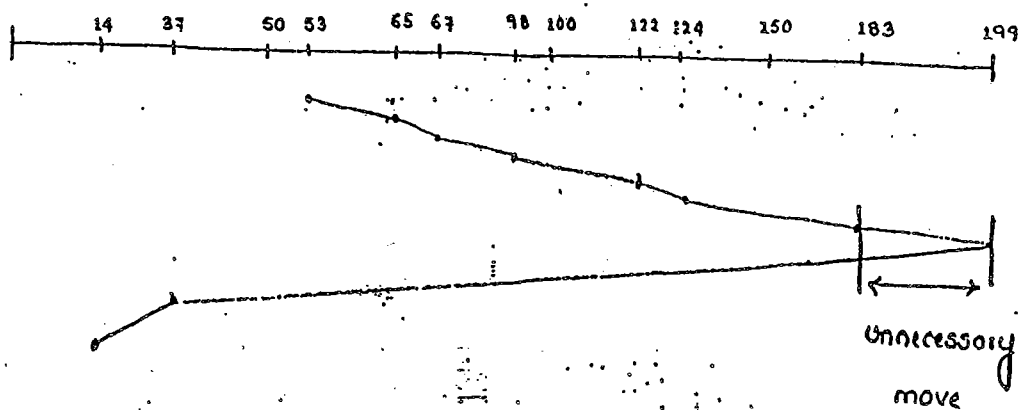


(b) SCAN / Elevator algorithm) :-

- * Read-write head $\Rightarrow$ represents directional scan.
- * Scanning through the request encountered in certain direction.

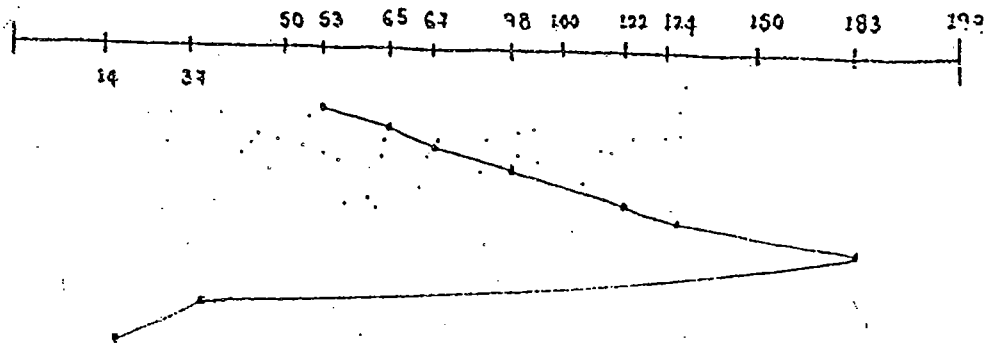
Drawback:

- * It moves to the last track unnecessarily.
- * Unnecessary causing starvation, by pending the other track in other direction.



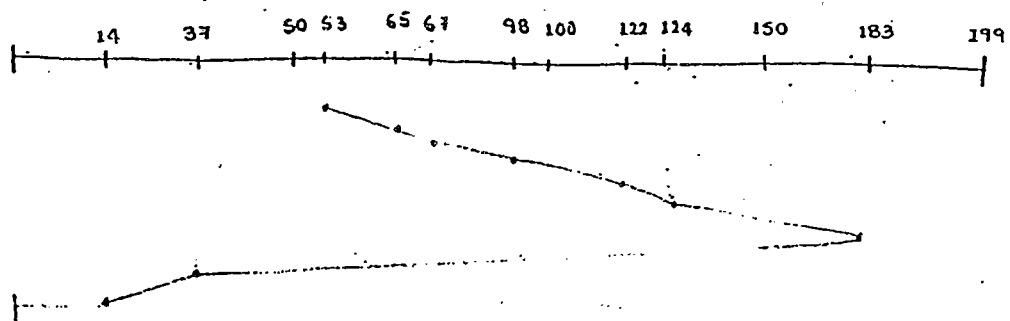
(4) Look :

Look up to last request pending and take reverse turn.



(5) Circular scan (cscan):

Eg:- Disk



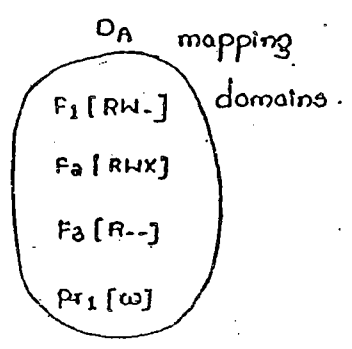
Security vs Protection

Security :

- * Making system secure from external threats. (virus, worms, trojans etc)
- * Security / protection has difference of authorised & unauthorised users.
- * Protection mechanism is implemented using "domain() method"
- * Protection Domain (objects, Rights)



refers either user/process (with Reg. permissions) :

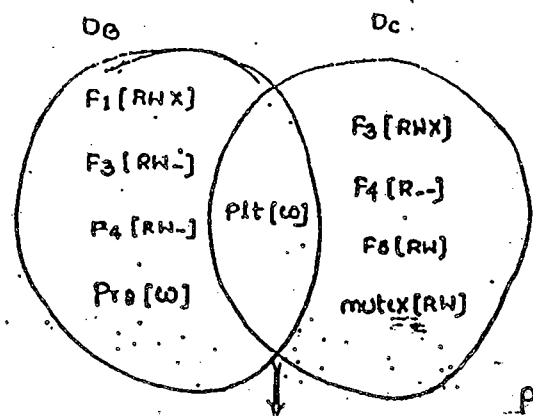


Protection :-

Internal threats
(system level)

Security :

External threats



↓
Overlapping
domain

plt ⇒ platter...

(1) Protection Domain matrix :

(Printer)

	F ₁	F ₂	F ₃	F ₄	F ₅	Pr ₁	Pr ₂	Pr ₃	Plt	mutex
D _A	RW-	RWX	R--	-	-	W	-	-	-	-
D _B	RW		RW	RW			W			
D _C			RW	R	RW					RW

$n \times m$

$n \rightarrow$ no. of domain

$m \rightarrow$ no. of objects

entry $\rightarrow$ permission.

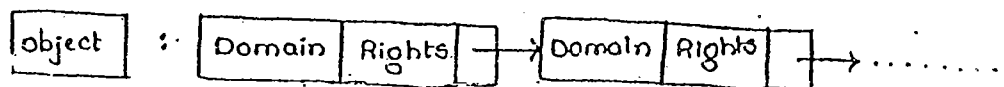
$\therefore$ They support direct access. (So, they are faster)

Drawback : space wastage (called sparse matrix).

$\downarrow$
most are null entries.

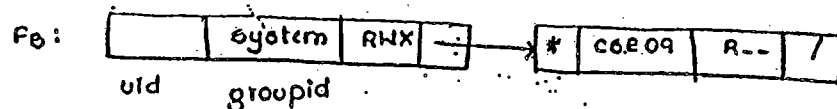
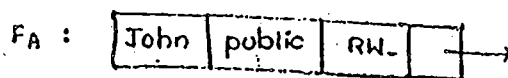
To overcome

$\downarrow$
a) Access Control list (A.C.L) mechanism :



linked lists headed by objects

Used in : UNIX / LINUX



uid groupid

(domain) $\Rightarrow$

file used by all cse people.

F₀ :

*	CSE10	RW-	
---	-------	-----	--

 →

*	CSE08		
---	-------	------	--

F_D :

*	*	R--	/
---	---	-----	---

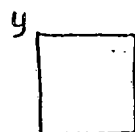
 ⇒ All users of comp.
& use people

(3) Capability list (c-list) :

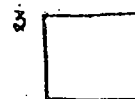
Type	Rights	Name/Address
FILE	RW-	F ₁
FILE	RWX	F ₂
FILE	R--	F ₃
PRINTER	W-	Pr ₁

→ stored in
fixed place

Capability (D_A) windows



c(D_B)



c(D_C)

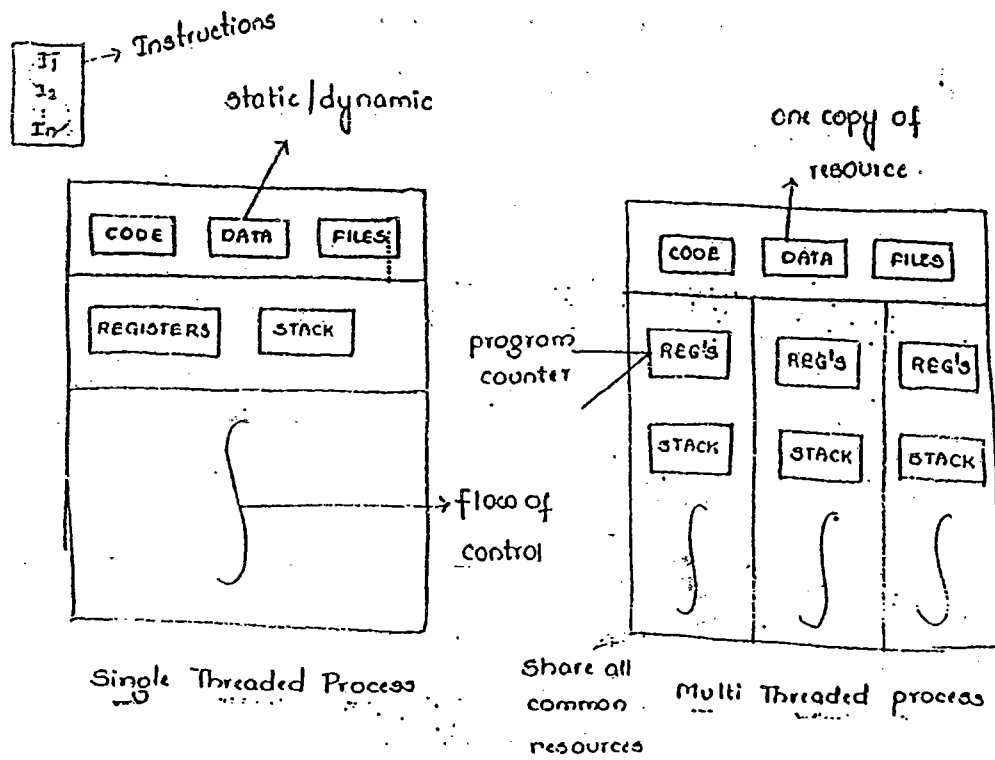
protection mgmt follows

c-list

Domain	Address
D _A	x
D _B	y
D _C	z

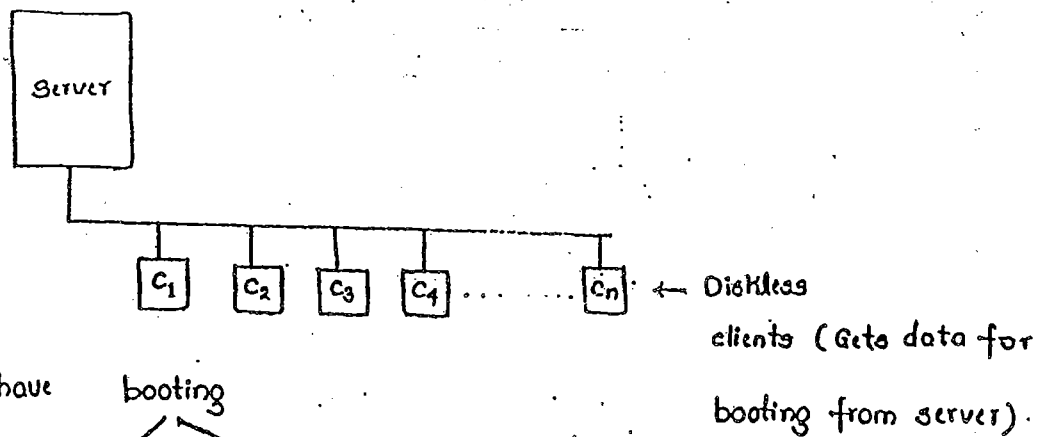
(capability)

Threads & Multithreading



Thread - light weight process.

LAN (File Server process) :-

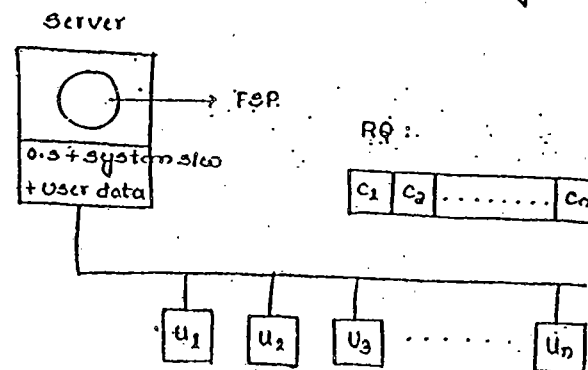
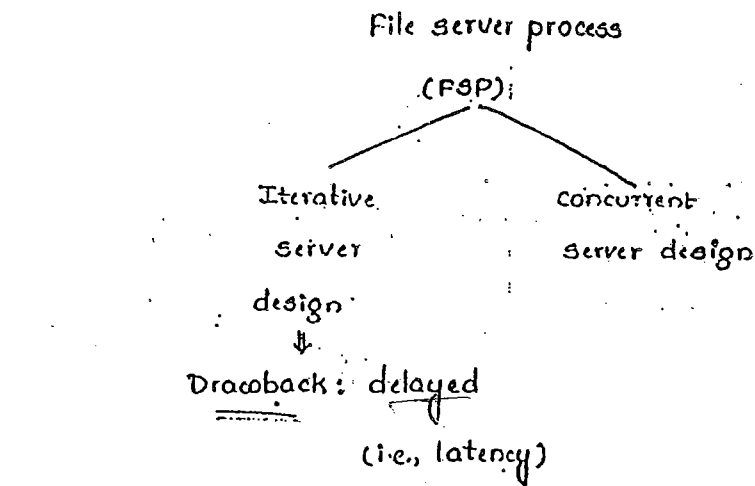


we have booting

- Local booting** (done on it's own)
- Remote booting** (done through server)

$$|TCB| < |PCB|$$

- * If server is alive, it transfers the kernel code to the client to get booted, which is called as "remote booting."



- * Concurrency is achieved through multi-process mechanism (using forks).
- * The same code of server (parent) get copied in the childs (clients), i.e., duplication of code is done at all the clients.

Drawback:

- * wastage of resources:

- * Since code is duplicated (i.e., 'n' copies of code, data, files and accessed by all the clients. Same functionality is processed by all clients, instead of single copy and get shared by them).

In non-sharing clients, every independent process must have program counter, General purpose registers and stack for each, where as a single copy of these can be accessed by all the clients in sharing.

Benefits: of multithreading

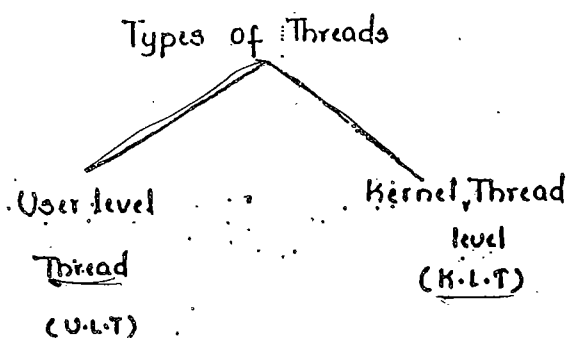
- * Resource sharing
- * cost effective : Economical
- * Improved performance.
- * Achieve parallelism (multi-CPU).
- * $|TCB| < |PCB|$

* As process have PCB's, Threads also have TCB's (Id, state, priority)

* Attributes that can't be shared, are stored in TCB (i.e., Id) and which are shared are stored in PCB.

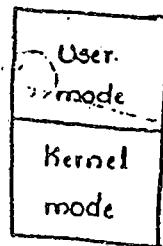
* Context-switch becomes fast and less context-switch overhead. So, there is improved performance.

* Since, it satisfies (5) & (6)th conditions, a thread is called as "Light weight process".



Threads which are created by user label

75



CPU

* Threads, which have packages, libraries.

Eg: Java threads (It won't require O.S support)

* O.S visualises it as a process.

- * It is transparent to the O.S, that it is only viewed as a process, but not as a thread.
- * Java Virtual machine (JVM) allocates the time to the threads.
- * User level Thread switching (U.L.T.S) is faster than Kernel level Thread switching (K.L.T.S), because for every processing, there is no need to go to kernel always, everything is managed at user level.

Drawback:-

- * when a process requires I/O, the total process gets blocked instead of a single thread, because of transparency.
- * To overcome this drawback, we move to kernel level threads.
- * Threading management can be handled by kernel level.
- * If one thread requires I/O, then that particular process only gets blocked but not the others.
- * context switching also done through kernel level. so, it is slightly slower compare to user level thread.

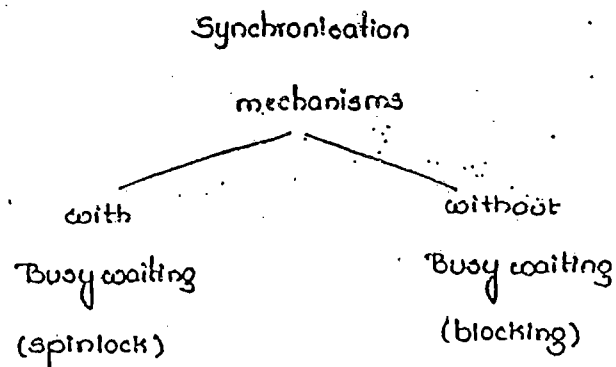
The entire man

is done by thread library packages
& environment

Pthread :

* Portable operating system interface for UNIX $\Rightarrow$ POSIX

Monitors



Spinlock $\Rightarrow$ busy waiting.

{ livelock $\Rightarrow$ states of process are always ready (on running).
Deadlock $\Rightarrow$ states of process are always blocked

monitors :

It is a collection of procedures, variables and data structures, that are all group together in a special kind of modular package.

: Procedures running outside the monitor, cannot access the monitor's internal variables & data structures. However, they can activate monitor's internal procedures.

* Monitors have an important property that "only one process can be active at any time".

Producer-consumer problem:

```
monitor producer_consumer;
```

```
begin
```

```
integer count = 0;
```

```
condition full, empty;
```

```
Procedure Enter
```

```
begin
```

```
if (count = n) wait (full);
```

```
Buffer[in] = itemp;
```

```
in = (in + 1) mod n;
```

```
count = count + 1;
```

```
if (count = 1) signal (empty); // wakeup consumer process
```

```
end
```

```
Procedure Remove
```

```
begin
```

```
if (count = 0) wait (empty);
```

```
itemc = Buffer[out];
```

```
out = (out + 1) mod n;
```

```
count = count - 1;
```

```
if (count = n - 1) signal (full);
```

```
end
```

```
end
```

```
Procedure producer
```

```
begin
```

```
while (True)
```

```
begin
```

```
Produce_item(itemp);
```

```
Producer_consumer_enter;
```

```
end
```

```
end
```

```
Procedure consumer
```

```
begin
```

```
while (True)
```

```
begin Producer_consumer_remove;
```

```
consume item/itemc;
```

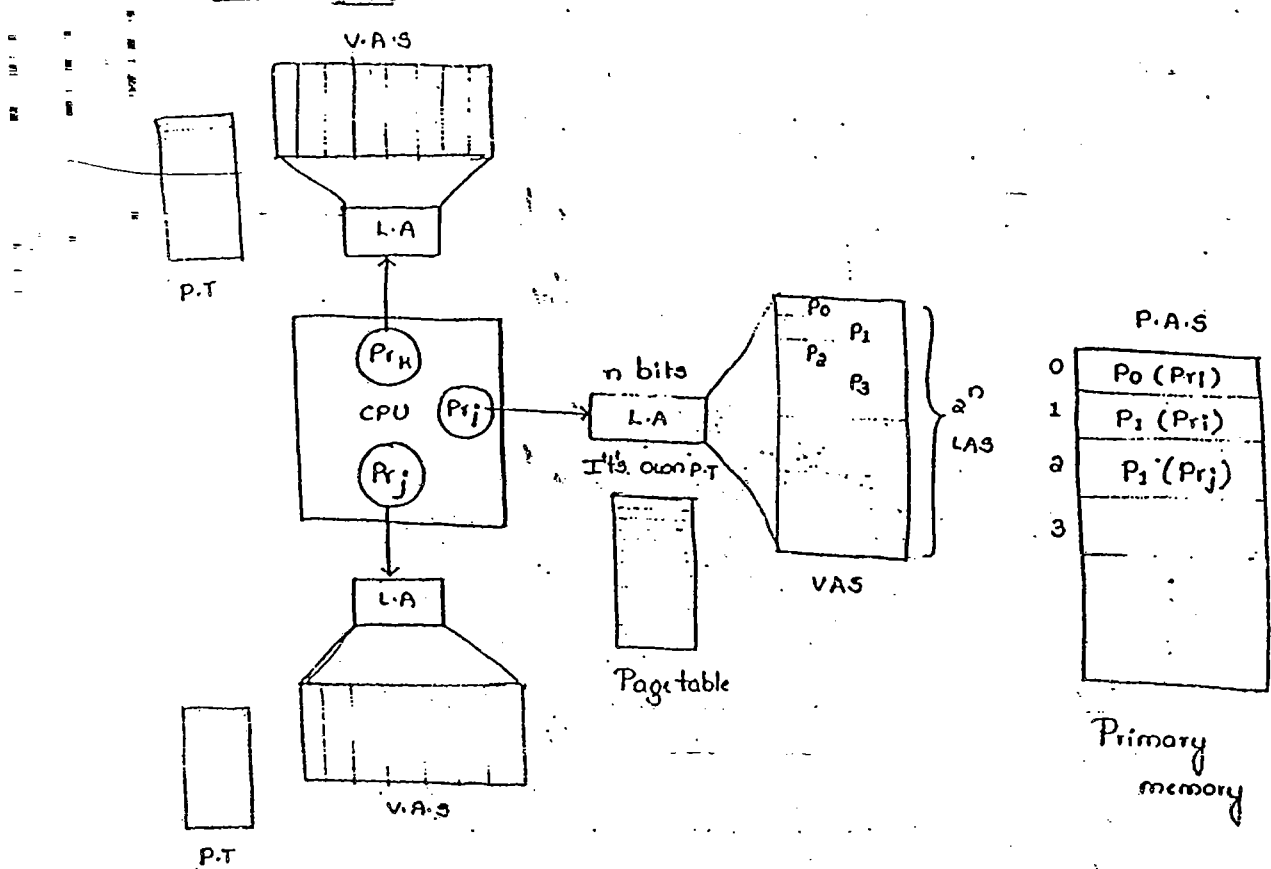
$\text{mutex} = 1 \Rightarrow$ no process is inside

$= 0 \Rightarrow$ some process is accessing

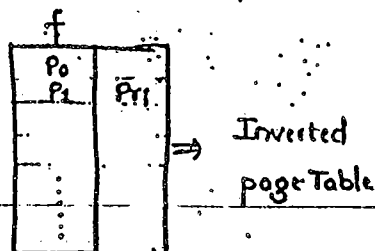
Initially $\text{mutex} = 1$;

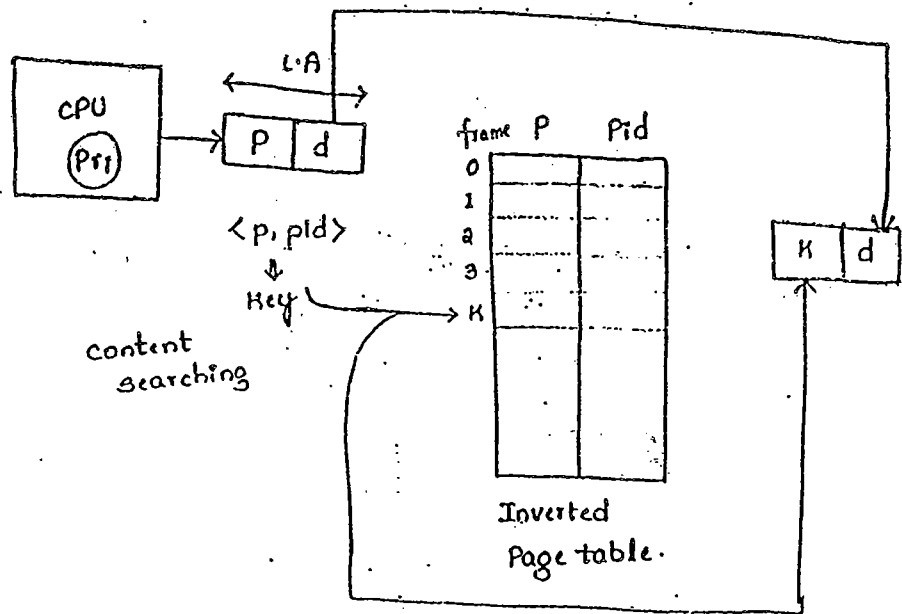
then $\text{DOWN}(\text{mutex})$;

Inverted paging :-



All the processes maintain single primary memory ~~can~~ within the page table, frames are stored within the page. The Inverted page table contains pages get stored in frames.





* The Inverted page table maintains the "key" generated by adding $\langle p, pid \rangle$, thus, by referring the key, the particular page gets accessed, and the same offset is considered. This reduces the time for searching the content.

08/07/2010

Thursday

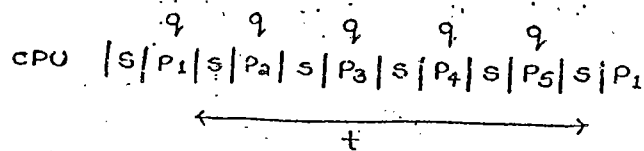
Page no : 34

(1) n -processors

context switching - 's' seconds.

$$T \cdot q = q$$

$n=5$

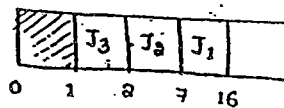


$$q \leq \frac{t - ns}{n-1} \text{ (atleast)}$$

$$q = \frac{t - ns}{n-1} \text{ (exactly)}$$

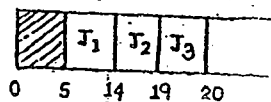
(4)

(a) $\{3, 2, 1\}, 1$



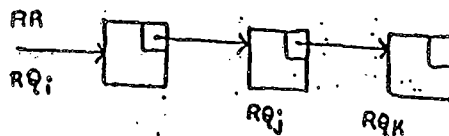
J	A.T	B.T
1	0.0	9
2	0.6	5
3	1.0	1

(d) $\{1, 2, 3\}, 5$



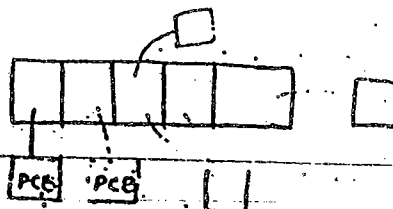
P.no: 42

(1)



priority of process increases

Drawback : Starvation.

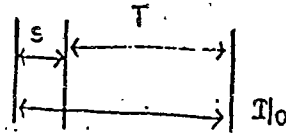


(9)

$$(a) \text{ CPU Efficiency } (\mu) = \frac{T}{T+S} \quad (TQ = \infty)$$

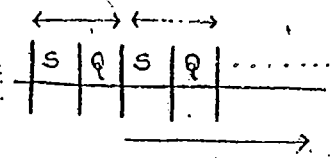
$$(b) \mu = \frac{T}{T+S} \quad (TQ > T)$$

$$(c) \mu = \frac{Q}{Q+S} \quad (S < Q < T)$$



$$(d) \mu = \frac{Q}{Q+S} \quad (Q = S)$$

$$= 50\%$$



$$(e) \mu = 0 \quad (Q = 0)$$

10) Trap & Interrupt :

Trap : Indicates error condition always

Interrupt : Indicates error condition (sometimes but not always)

02/07/2010

Thursday

Pno: 101

(6) $n=4$, $m=3$, $(r_1, r_2, r_3) = (2, 3, 8)$

n	Alloc	Request	Available
P ₁	1 1 0	1 0 0	0 0 1
P ₂	1 0 1	0 1 1	safe = $\langle P_3, P_2, P_1, P_4 \rangle$
P ₃	0 1 0	0 0 1	
P ₄	0 1 0	0 2 0	

4/07/2010

Saturday

P.no: 140

5) min. size of partition = max. path length (in sizes of modules)

All the paths are considered as per their sizes
and the maximum one is represented.

P.no: 142

(20)

$e \Rightarrow$ frame no. $PA9 = 64 \text{ MB}$

$VA9 = 32 \text{ bit}$

$PS = 4 \text{ MB}$

P.T size = $N * e$

$$N = \frac{LA9}{PS} = \frac{2^{32}}{2^{12}} = 2^{20} = 1 \text{ M.}$$

$$M = \frac{64 \text{ MB}}{4 \text{ KB}} = 2^{14} \approx 2 \text{ bytes}$$

$$m = \frac{64 \text{ MB}}{4 \text{ KB}}$$

$$= \frac{2^{26}}{2^{12}} = 2^{14}$$

$$e = N * e$$

$$= 1 \text{ M} * 20$$

$$= 2 \text{ MB.}$$

P.no: 153

(17)

$LA9 = 8 \text{ KW}$ (8 pages of 1024 words)

min size = 32K (32 frames of 1024 words)

$$LA9 = 8 * 1024$$

$$= 8 \text{ KW} = 13 \text{ hits}$$

$$PA9 = 32 * 1024 = 32 \text{ K} = 16 \text{ hits}$$

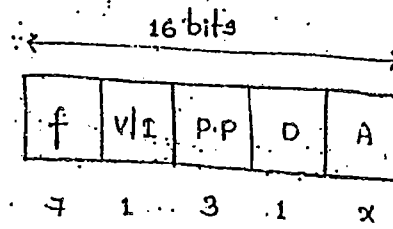
80107/2010

Friday

P.no: 150

(i) (b)

$$IAS = PAS = 2^{16} B$$



P.T. entry

$$f = \frac{2^{16}}{2^9} = 2^7 \quad \text{page size} = 512 = 2^9$$

P.no: 155

(86) (a) Stack :

Since, only top of the stack is accessed, so, there is a less no of page faults. It is good.

(b) Hashed symbol table :-

It distributes identifiers across the table. So, it generates more no. of page faults.

(c) Sequential search & Binary search :-

↓
likely to cause more page faults than seq. search.

(e) pure code :-

Read only code (non-modified code).

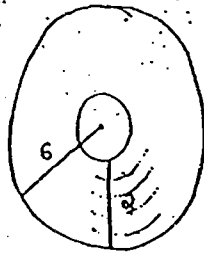
∴ It is good to have non-modified in demand paged environment.

(f) vector operations :-

Array (contiguous allocation) $\Rightarrow$ good.

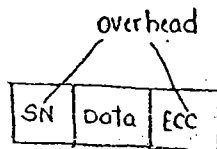
(g) Indirection :-

linked lists (non-contiguous allocation) $\Rightarrow$ bad.



$$\text{no. of tracks} = \frac{(6-2) \text{ cm}}{0.01 \text{ cm}}$$

= 400 tracks on one surface.



$$400 \times 96 = 4096 = 4 \text{ KB}$$

Each track has 96 sectors

$$(400 \times 96 \times 4 \text{ KB}) \times 2 \text{ surfaces}$$

$$\Rightarrow 2^9 \times 2^5 \times 2^{12} \times 2^3 = 2^{29} = 512 \text{ MB}$$

$$\approx 500 \text{ MB}$$

b)

$$R = \frac{60}{3600} \text{ sec}$$

$$\frac{60}{3600} \text{ s} \times 20 \times 4 \text{ KB}$$

$$1.6 \text{ sec}$$

$$3600 \times 20 \times 4 \text{ KB}$$

$$60$$

$$1.6 \text{ sec}$$

P.no: 184

(4) program size = 64 KB

$$T.S = 80 \text{ KB}$$

$$R = 80 \text{ ms}$$

$$S.T = 30 \text{ ms}$$

$$80 \text{ ms} - 30 \text{ KB}$$

$$? - 8 \text{ KB}$$

$$P.S = 2 \text{ KB}$$

$$\text{no. of pages} = \frac{64 \text{ KB}}{2 \text{ KB}} = 32 \text{ KB}$$

$$(a) \text{ Time for loading one page} = 30 \text{ ms} + 10 \text{ ms} + \frac{20 * 2}{32} \text{ ms}$$

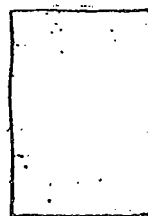
(b) page stored on track to go to that track, we have seek time

$$= 30 \text{ ms} + 10 \text{ ms} + \frac{20 * 4}{32} \text{ ms} * 16$$

010010010

wednesday(a) (a) max. no. of files $\Rightarrow$ depends on directory

$$\text{Directory} = 32 \text{ bits} \Rightarrow 4 \text{ B}$$



4KB

$$\therefore \frac{4 \text{ KB}}{4 \text{ B}} = 1 \text{ K}$$

$$(b) \text{ DBA} = 8 \text{ bits}$$

 $\therefore$ 256 blocks possible for a file of 8 bitsBut, there are 2 data blocks. so, $= 256 - 2 = 254$

$$(254 * 4 \text{ KB}) \approx 1 \text{ MB}$$

P.no: 183

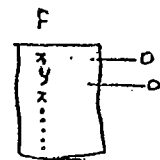
- (1)
- B - blocks
 - S - DBS
 - D - DBA
 - F - Free blocks

DBA = 0 bits
 no. of bits = B
 free = f

Bit map space consumption is 'B' bits (depends on blocks)

Size of free list (space) = $F \times D$

$$\therefore FD < B$$



S - DBS

Disk size = $B \cdot S$

B - blocks

D - DBA

$$2^D \geq B \quad (B \cdot S \leq 2^D \cdot S)$$

F - Free blocks

(maximum possible)

20MB 64MB

Block = 20K

DS = 20MB

$$2^{16} = 64KB * K = 64MB$$

DBA = 16 bits

(max.)

DBS = 1KB

2no: 179

(1) b

(2) b (since, it takes more access time, if placed anywhere).

(3) a

(4) b

(5) If there is only one process, then there is no matter whether to use any disk scheduling approach.

so $\Rightarrow 0\%$

(6) In use : 2 block
 free : $\begin{pmatrix} 0 \\ 0 \end{pmatrix} \Rightarrow$ Not so serious error but just reduces disk size

In use : 6 block
 free : $\begin{pmatrix} 1 \\ 1 \end{pmatrix} \Rightarrow$ Serious error

Since it is free, it may also given to other file, but a file is already being accessing. So, it is considered as a serious error.

To solve this error :-

copy the In use file to some other block and use it.

P.no: 37

(25) (a)

(26) (a)

P. no : 15a

(8) VA = 48 bits

PA = 32 bits

no. of pages = $8K = 2^{13}$

no. of entries = $\frac{2^{48}}{2^{13}} = 2^{35} = 32G \text{ entries}$

Inverted page table :

$= \frac{2^{32}}{2^{13}} = 2^{19} = 512K \text{ entries}$